Local EIS Pipeline

Training guide

This is the training guide for whoever runs, maintains or extends the Local EIS pipeline. It explains what the software does, how it reads its data, where it stores every result, what each script is for, and how to work on it safely. It assumes you know what impedance spectroscopy is. It does not assume you have seen the code.

What it covers	The whole package: 4 Databricks notebooks, the `runner/` package, `core/`, `pipeline/`, `checks/`, `analysis/`, `plotting/`, `readers/`, `EIS_config/`, `tests/`
Size	115 Python files (59 of them in `tests/`: 56 test modules, 2 data generators, `conftest.py`), about 43,850 lines
Verified	Test suite run on this exact package: **505 passed, 3 skipped, 12 failed** (see §21.1 for what each failure means)
Companions	`TRAINING_GUIDE_FILE_INDEX.md` (every class and function, generated from the source) · `PIPELINE_GUIDE.md` (the package's own reference manual) · `../SETUP_SHARED_FILES.md` (shared calibration files)

Contents

Part 1 — Getting started 4

1. How to use this guide 4

2. What the software does 4

2.1 The measurement 4
2.2 The calculation 4
2.3 Why it is 43,000 lines and not 400 4
2.4 The architecture: medallion layers 5

3. The package on one page 6

3.1 Folder map 6
3.2 Four notebooks, one package 6
3.3 Two pipelines behind one switch 6
3.4 How the modules find each other 6

4. Your first run on Databricks 7

4.1 One-time setup (per workspace) 7
4.2 Evaluate an order (01 Local EIS - Process) 7
4.3 Look at the results (02 Local EIS - Results) 8
4.4 After a code change 8

5. Working on a laptop (no Databricks) 8

Part 2 — Data in, data out 9

6. Where every input is read from 9

6.1 Overview 9
6.2 FAMOS card recordings 9
6.3 Datago (Spark SQL) 10
6.4 Gamry sweeps: which one belongs to the run? 10
6.5 Plate calibration files and the plate lookup 10
6.6 CSV measurements 11

7. Where every result is stored 11

7.1 The three places 11
7.2 The run folder 11
7.3 The Delta tables 12
7.4 The cache: run_identity.json and the run modes 12

8. The life of one run, call by call 13

8.1 The data objects handed between steps 13

Part 3 — The processing 15

9. Bronze: raw waveforms → raw impedance 15

9.1 The steps of bronze.run(cfg) 15
9.2 The Gamry clock (pipeline/gamry_sync.py) 16
9.3 Bronze files (<run>/bronze/) 16

10. Silver: correct, gate, model, grade 16

10.1 Before the segments: plate-level decisions 16
10.2 Point gates (gate_points) — the first gate to reject a point owns it 17
10.3 Modelling: R_ohmic, R_ct, R_mt, R_pol 17
10.4 Validation and tiers 17
10.5 Plate products 17
10.6 Silver files (<run>/silver/) 18

11. Gold: the plate as a field 18

11.1 Gold files (<run>/gold/) 18

12. Checks: is the result believable? 18

12.1 Gamry whole-cell comparison (checks/gamry_compare.py) 18
12.2 Chain frequency response (checks/frequency_response.py) 19
12.3 Plausibility (checks/plausibility.py → plausibility.csv) 19
12.4 Other checks you run by hand 19

13. The CSV pipeline 19

14. ECM fits in the runner 20

15. Formula sheet 20

Part 4 — Every script 21

16. Script-by-script guide 21

16.1 Top level 21
16.2 runner/ — the notebooks' code 21
16.3 core/ — foundations 22
16.4 pipeline/ — the processing 22
16.5 checks/ — diagnostics 23

16.6 analysis/ 23
16.7 plotting/ 23
16.8 readers/ 23
16.9 EIS_config/, docs/, tests/ 24

Part 5 — Working on it 25

17. Settings: what to change, and where 25
17.1 runner/settings.py reference 25
17.2 core/config.py: the fields you will meet most 25

18. How-to recipes 26
18.1 Evaluate a new FAMOS order 26
18.2 Re-evaluate after a code or setting change 26
18.3 A gen2 (blue Naboo) order 26
18.4 A plate mounted turned 180° 26
18.5 A broken segment 26
18.6 Build or rebuild a plate's chain_gain.csv 27
18.7 Check a new Abgleich delivery, or rebuild curr.csv/temp.csv 27
18.8 Build the frozen high-frequency response of an order 27
18.9 Evaluate CSV measurements 27
18.10 Schedule the automatic job 27
18.11 Remove a run 27
18.12 Add a setting that changes results 27
18.13 Add a parameter to the plate maps and tables 27
18.14 Add a plausibility check 28
18.15 Re-evaluate an old run without its .DAT files 28
18.16 Query results across orders 28
18.17 Move to another site or workspace 28
18.18 Ship a new version to Databricks 28

19. Development workflow and tests 28
19.1 The loop 28
19.2 Rules for changing the code 28
19.3 How the tests work 28

20. Reading a finished result 29

21. Troubleshooting and known issues 29
21.1 Test results on this package 29
21.2 The runner/tables.py gap 30
21.3 Measurement limitations (from the package's own analyses) 30
21.4 Documentation that is out of step with the code 30
21.5 Symptom → cause → fix 31

22. Training plan and self-check 31
Session 1 — The picture 31
Session 2 — Data in and out 31
Session 3 — The science 31
Session 4 — The code 31
Session 5 — Making a change safely 32
Answers 32

23. Glossary 32

Appendix A — File reference (generated) 34

(top level) 34
runner/ 35
core/ 38
pipeline/ 41
checks/ 51
analysis/ 55
plotting/ 56
readers/ 60
docs/ 62
tests/ 63

Part 1 — Getting started

1. How to use this guide

Different readers need different parts. Pick your path:

You are…	Read	Then
An operator who evaluates measurements	§2, §3, §4, §6, §7, §20	§18 recipes, §21 troubleshooting
An analyst who interprets maps and spectra	§2, §9–§15, §20	§23 glossary
A developer who changes the code	Everything, in order	§16 with the code open, §19 before your first commit
New to the project	§22 training plan — it walks you through the guide in five sessions with exercises	

Conventions used throughout:

- `code` is a file, function, setting or command. `pipeline/bronze.py:run` means the function `run` in that file.
- **Volume** means the Databricks Unity Catalog Volume `/Volumes/ps_xplatform_dev/rvadvtec_dev/ev_rvadvtec_dev` (called `EV_ROOT` in the code).
- **Order** (also *Leepa*) is the test order number, e.g. `2612090`. Datago writes it as `RO2612090`.
- **Condition** is an operating point named by its current: `45A`, `60A`, `150A`, `450A`.
- **Test** is the FAMOS test number: `Test_01` (up-ramp), `Test_02` (down-ramp), …
- Impedances inside the code are area-specific in Ω·cm²; every output file and plot uses **mΩ·cm²**.

> **Rule of thumb.** Settings live in `runner/settings.py` (what the notebooks do) and `core/config.py` (every number of the science). The notebooks themselves contain only calls. If you find yourself editing a notebook to change behaviour, stop — the change belongs in one of those two files.

2. What the software does

2.1 The measurement

A Bosch **R2-D2 segmented plate** sits on one side of a PEM fuel cell. Its 900 pads (45 × 20, each 5.60 × 6.05 mm) are wired into **72 segments**, so the plate is 252 × 121 mm = **304.92 cm²**. Each segment carries its own current through a shunt, so its local current density can be measured.

A **Gamry** potentiostat runs a galvanostatic EIS sweep on the whole cell (typically 0.15 Hz to ~12 kHz, stepped). While it does, five imc **FAMOS** acquisition cards ("Karte_1" … "Karte_5") record raw waveforms, 16 channels each, typically at 25 kHz:

- one or two **cell-voltage** channels per card (`UC1`, `UC2`; `UC2` is the reference),
- the **segment current** channels, named `1` … `72` (the segment numbers printed on the plate),
- four **plate temperature** sensors `Temp_1` … `Temp_4` at x = 0, 84, 168, 252 mm.

A test bench logs the operating point (currents, gas flows, temperatures, pressures) to an **MF4** file.

2.2 The calculation

For every segment *s* and every excitation frequency *f*:

> **Z_s(f) = K_s(T) · A_U(f) / A_s(f)**
> *A_U* is the cell-voltage phasor, *A_s* the segment's current-channel phasor, both fitted at *f*. *K_s(T)* is the segment's calibration constant (V per A/cm²) at its temperature, from `curr.csv`. The result is an area-specific impedance in Ω·cm².

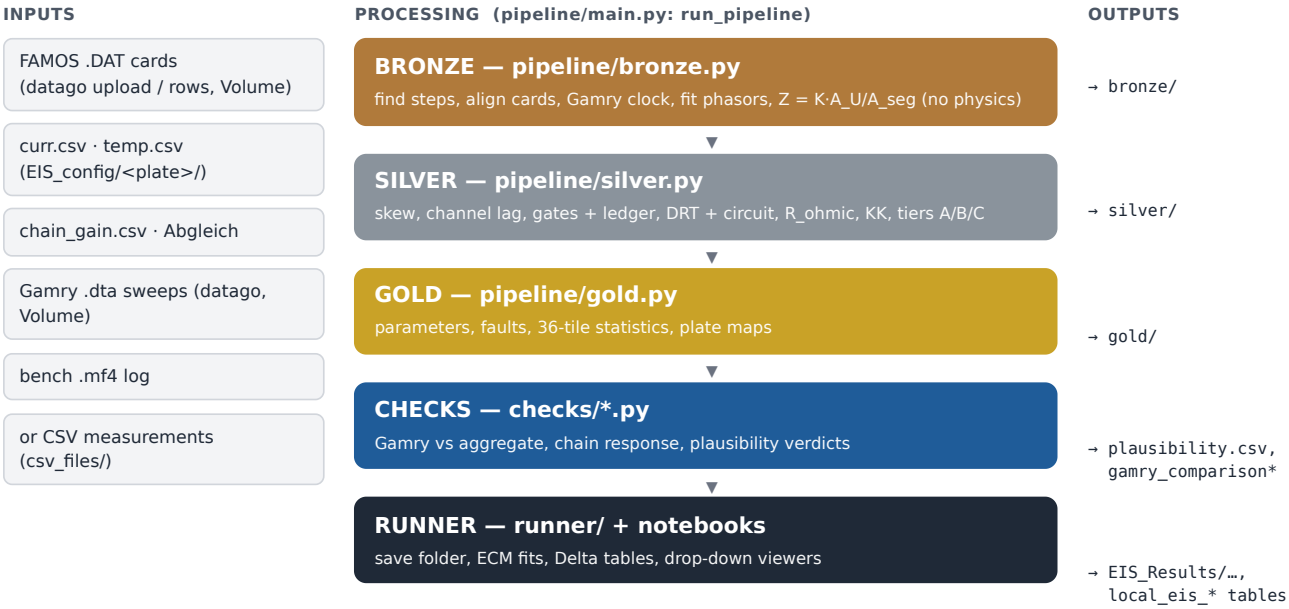
From each segment's spectrum the pipeline derives the **ohmic resistance R_ohmic** (the high-frequency intercept), **Re Z at 1 kHz**, the **charge-transfer resistance R_ct**, the **mass-transport resistance R_mt**, the **polarisation resistance R_pol = R_ct + R_mt**, the **DC current density j** and the **temperature T**. It draws each as a plate heat map, sums the 72 segments in parallel and compares the result with the Gamry whole-cell spectrum, and runs plausibility checks on the whole plate.

2.3 Why it is 43,000 lines and not 400

The ratio above is one line of code. Making it trustworthy is the rest:

Problem	What the code does about it	Where
Which stretch of a 2-minute record holds which frequency?	Detects the stepped sweep blindly, snaps it to the geometric ladder, then puts it on the Gamry's own clock	`eis_local`, `ladder_snap`, `hf_schedule`, `gamry_sync`
Five cards start at different moments (up to 5.7 s apart)	Cross-correlates the shared `UC2` channel to align the cards	`bronze.estimate_card_lags`
One multiplexed ADC samples the 16 channels one after another (2.5 µs per slot)	Structural de-skew per channel slot	`silver` (skew model)
Each segment's shunt/amplifier chain adds its own delay (±50–110 µs)	Fits and removes a per-segment lag	`channel_lag`
Each card's voltage gain differs by ~1 %	Measures it from the shared `UC2` and divides it out	`card_gain`
The amplifiers roll off above 1 kHz	Divides by the ex-situ chain response from the Abgleich bode sweeps	`gamry_dta`, `chain_gain.csv`
Some points are noise, clipped, distorted or drifting	11 point gates; every rejected point is recorded with its reason	`silver.gate_points`, `point_rejections.csv`
Is the spectrum physically valid?	Lin-KK and Z-HIT tests, quality tiers A/B/C	`eis_validation`, `silver`
Does the whole plate add up?	Parallel sum vs Gamry, DC current closure, a dozen plate-level plausibility checks	`checks/`

2.4 The architecture: medallion layers



Each layer only **adds**; nothing upstream is overwritten. Every layer writes its own folder, so a later layer can be re-run from the saved files of the earlier one (`main.py --reevaluate`) without touching the 40 GB of raw `.DAT` files.

3. The package on one page

3.1 Folder map

```
local_EIS_TOM_rebuild/
├── 01 Local EIS - Process.py      notebook: evaluate + publish          (WRITES)
├── 02 Local EIS - Results.py      notebook: plots of saved runs          (READ-ONLY)
├── 03 Local EIS - Auto.py         notebook/job: evaluate what is new     (WRITES)
├── 09 Local EIS - Maintenance.py  notebook: remove a run, rebuild tables (DESTRUCTIVE)
├── eis_paths.py                   puts core/ pipeline/ checks/ … on sys.path
├── run.py                         command line: python run.py <module> [args]
├── requirements.txt               numpy, scipy, matplotlib (+ pandas, plotly)
├── runner/       what the notebooks do: settings, datago, Session, Results, tables, auto job, views
├── core/         config (every number), utils, plate geometry, plate stats, plate conditions, site paths
├── pipeline/     main (orchestrator), bronze, silver, gold, csv_pipeline + signal processing
├── checks/       plausibility, Gamry comparison, card gain, DC closure, frequency/HF response
├── analysis/     ecm_drt (DRT and DRT-informed circuit fit)
├── plotting/     Nyquist, plate maps, drop-down viewers, style, bench (MF4) plots
├── readers/      Gamry .DTA, MF4, Abgleich, CSV sources, zstd shim
├── EIS_config/   calibration files per plate: gen1/ (green Kashyyyk), gen2/ (blue Naboo)
├── tests/        pytest suite (56 test modules), synthetic data generators, fixtures
└── docs/         this guide, the pipeline guide, analyses, figures
```

Folder	Files	Role
(top level)	6	Entry points: 4 notebooks, `eis_paths.py`, `run.py`
`runner/`	8	The notebooks' code. Paths and switches, datago, widgets, cache, ECM, tables, job, figures
`core/`	6	Shared foundations that every other folder uses
`pipeline/`	15	The processing itself
`checks/`	7	Diagnostics. None of them can stop a run
`analysis/`	1	DRT-informed equivalent-circuit fit
`plotting/`	8	Every figure
`readers/`	5	Readers for every file that is not a FAMOS card
`tests/`	59	461 test functions, 520 test cases after parametrisation

3.2 Four notebooks, one package

The notebooks are deliberately thin: each cell is one call into `runner/`. That split copies the TOM `spp_eval` pipeline: notebooks that **write** and notebooks that only **read**, each opening with a banner that says which.

```
FAMOS cards (datago / Volume) + Gamry sweeps + bench .mf4
  └─ 01 Local EIS - Process       bronze → silver → gold per order / condition / test
       ├─ EIS_Results/<order>/<condition>_Test<nn>/       the run folder
       └─ <TABLE_SCHEMA>.local_eis_*                      the same results as Delta tables
            ├─ 02 Local EIS - Results    one order's plots; every order from the tables
            └─ 03 Local EIS - Auto       job: evaluates what is not published yet
  09 Local EIS - Maintenance              removes one run (tables, optionally folder)
```

Notebook	Banner	Cells (each one call)
`01 Local EIS - Process`	writes results and tables	Setup `runner.Session(globals())` → `R.widgets()` → `R.run()` → `R.fit_ecm()` → `R.publish()` → `R.summary()`
`02 Local EIS - Results`	📊 read-only	Setup `runner.Results(globals())` → `R.widgets()` → `show_frequency_response` · `show_nyquist` · `show_median_nyquist` · `show_heatmaps` · `show_mf4` · `show_ecm` · `show_runs` · `show_trends`
`03 Local EIS - Auto`	🛠 writes (scheduled job)	Setup → job parameters `dry_run`, `max_runs`, `orders` → `runner.auto.plan(R)` or `runner.auto.run(R)`
`09 Local EIS - Maintenance`	⚠️ destructive	`tables.delete_run(spark, KEY)` (key typed twice) · optional folder delete · `tables.rebuild(spark)` (type `REBUILD`)

3.3 Two pipelines behind one switch

`source_format = "famos"` (the normal case) runs bronze → silver → gold. `source_format = "csv"` runs `pipeline/csv_pipeline.py`, a separate pipeline for the single-clock CSV logger and the bench tool's exported spectra. They share the plate geometry, the calibration reader, the physics formulas, the gold outputs and the checks. They are **two pipelines, not two readers**: FAMOS needs inter-card synchronisation, CSV has one clock and must not run it (§13).

3.4 How the modules find each other

`eis_paths.py` inserts the package root and its six sub-folders (`core`, `pipeline`, `checks`, `analysis`, `plotting`, `readers`) at the front of `sys.path`. Every module therefore imports the others by plain name — `import silver`, `from config import DEFAULT` — wherever

it lives. The notebooks, `run.py`, `pipeline/main.py` and `tests/conftest.py` all import `eis_paths` first. `runner/` is a normal package imported as `import runner`.

Layering rule (keep it when you add code): `core/` imports nothing from other folders except `plate_style` (for drawing); `pipeline/` uses `core/`, `readers/` and some `checks/`; `plotting/` only reads results; `runner/` sits on top of everything. A `core` module importing `silver` would create an import cycle.

4. Your first run on Databricks

4.1 One-time setup (per workspace)

1. **Import the package into an EMPTY workspace folder.** Never import on top of an older copy: a folder with a new `silver.py` beside an old `gold.py` gives silently wrong maps.
2. **Shared calibration files.** Make sure `/Volumes/…/ev_rvadvtec_dev/EIS_config/gen1/` and `…/gen2/` hold `curr.csv`, `temp.csv`, `chain_gain.csv` and `Abgleich/` (upload the package's `EIS_config/` folder once; see `SETUP_SHARED_FILES.md`). If the Volume copy is missing, the package's own `EIS_config/<plate>/` is used as a fallback.
3. **Permissions.** Everyone who runs the notebooks needs read access to `Famos/`, `Gamry/`, `EIS_config/` and `EIS_Results/` on the Volume, write access to `EIS_Results/`, read access to the datago tables, and `CREATE TABLE` on `ps_xplatform_dev.rvadvtec_dev` (for publishing).
4. **Cluster.** Any Databricks ML runtime: numpy, scipy, matplotlib, pandas and plotly are already installed. `asammdf` is optional (MF4 bench log); `readers/mdf4_min.py` reads Gamry `.mf4` sweeps without it.

4.2 Evaluate an order (01 Local EIS - Process)

1. Open **`01 Local EIS - Process`**, attach the cluster, run the first three cells (restart, Setup, Selection). The setup cell prints `Local EIS v35.1 -- modules from /Workspace/…`.
2. The widget bar appears. Choose:

Widget	What to choose
Order ID (Leepa)	The order, e.g. `2612090`. The list merges datago (orders with a Gamry `GALVEIS` sweep) and the FAMOS file names on the Volume
Measurement file format	`famos` (normal) or `csv`
Plate	`gen1` = green Kashyyyk, `gen2` = blue Naboo. **Choosing wrong does not fail** — it draws the right numbers on the wrong squares. The Selection cell warns when the order's file names name the other plate
Plate position	`standard` (as drawn) or `rot180` (plate mounted turned 180°, inlet and outlet segments swapped)
CSV folder	Only for `csv`: the campaign folder under `csv_files/`
Conditions	`ALL` or a subset, e.g. `45A,450A`
FAMOS test	`01` (up-ramp), `02` (down-ramp), … (read from the file names)
Exclude segments	Empty, unless a segment is known broken; it is then not read at all
Rebuild from neighbours	Segments whose own data is discarded and rebuilt from adjacent segments
Fill unmeasured segments	`no` (default) or `yes`
Run mode	`cache` = use the saved result, run only what is missing; `rerun` = recompute and **overwrite**

3. The Selection cell prints what it resolved — read it:

```
Order       2612090   (famos, gen1 green / Kashyyyk)
Plate in    standard   (gas and coolant ports fixed to the cell)
Conditions  45A, 60A, 150A, 450A   Test_01
Run mode    cache   (saved results are used)
Gamry       /tmp/datago_gamry/RO2612090   build V26_112
Chain gain  /Volumes/…/EIS_config/gen1/chain_gain.csv
Results     /Volumes/…/EIS_Results/2612090
```

   `!!` lines are warnings (plate mismatch, missing calibration files). A missing `curr.csv`/`temp.csv` stops the run with the exact path where the file must go.

4. **Run all.** The *Evaluate* cell prints one line per condition — `saved` (loaded) or `run` (computed, 5–25 min each) — then the pipeline log, the plausibility verdicts and `45A: 66 segments, 412 s -> /Volumes/…/EIS_Results/2612090/45A_Test01`.
5. *ECM fits* fits the standard and the DRT-informed circuit to the aggregate and every segment; *Publish* writes the run into the Delta tables; *Summary* shows one line per run (segments, Gamry sweep, failed/warned checks, FAMOS source).

4.3 Look at the results (02 Local EIS - Results)

Open `02 Local EIS - Results`, choose a saved order folder (e.g. `2612090` or `2612091_gen2`), its conditions and its test, and run all. Each cell draws **one plot with drop-downs** and saves it as `EIS_Results/<order>/plots/<name>.html`. Nothing is evaluated here; `R.run()` refuses.

4.4 After a code change

Always run once with **Run mode =** `rerun`. The cache compares a digest of the *settings*, not of the code (§7.4). A saved result made by another runner version is still used under `cache`; the Evaluate cell says so (`saved by runner v34 …; rerun to evaluate with v35.1`).

5. Working on a laptop (no Databricks)

Everything except datago, the Delta tables and the widgets runs locally.

```
cd local_EIS_TOM_rebuild
python -m venv .venv && . .venv/bin/activate
pip install numpy scipy matplotlib pandas plotly pytest     # + asammdf zstandard for MF4 bench logs

python run.py main --self-test                       # geometry, estimators, CSV readers: no data needed
python -m pytest tests -q                            # 520 test cases, ~2 min

# one full FAMOS run from local files
python pipeline/main.py --dat /data/Famos --curr-cal EIS_config/gen1/curr.csv \
       --temp-cal EIS_config/gen1/temp.csv --gain EIS_config/gen1/chain_gain.csv \
       --gamry /data/Gamry --leepa 2612030 --condition 45A --out out/2612030/45A

# silver + gold again from a saved run's bronze/ folder (no .DAT needed)
python run.py main --reevaluate EIS_Results/2612090/45A_Test01 --out out/re45A

# a synthetic plate with known truth (cards + Gamry .DTA + calibration)
python tests/make_synth_sweep.py /tmp/synth
```

`runner.Session` also works without Databricks: `Session(globals(), famos_root=…, results_root=…, config_root=…)` with `dbutils = None` uses `R.select(leepa_id=…, conditions=…, run_mode=…)` instead of widgets, and `R.html()` writes the viewer to `<scratch>/last_view.html` instead of displaying it. The figures in `runner/views.py` are pure functions of a run folder, so they can be called from any script.

Part 2 — Data in, data out

6. Where every input is read from

6.1 Overview

All Volume paths are under `EV_ROOT = /Volumes/ps_xplatform_dev/rvadvtec_dev/ev_rvadvtec_dev`. The paths are set once, in `runner/settings.py`.

Input	Where it is read from (first hit wins)	Format	Read by
FAMOS card recordings	`FAMOS_FROM_DATAGO = True`: datago's original upload → datago rows rebuilt as `.DAT` → Volume `Famos/` (per card). Off: Volume `Famos/` only	imc FAMOS `.DAT`, v1 or v2 dialect, 16 channels	`eis_local.FamosFile` (via `bronze`)
Gamry whole-cell sweeps	datago (every sweep of the order, written back as `.dta`) → Volume `RO<order>_Gamry/`, `<order>_Gamry/`, `Gamry/`	Gamry `.DTA` (`ZCURVE` table) or `.mf4` export	`gamry_dta.read_dta`, `gamry_compare`, `mdf4_min`
Bench log	datago (longest `.mf4` of the order that is not an `_Anfang`/`_Ende` snapshot) → Volume `Gamry/*RO<order>*.mf4` (largest)	ASAM MDF4	`bench_plots.read_mf4`, `gamry_compare.read_bench_log` (needs `asammdf`)
Current calibration `curr.csv`	`EIS_config/<plate>/` on the Volume → `<package>/EIS_config/<plate>/` → (gen1 only) your own workspace folder	72 rows `c0;c1`	`eis_local.PlateCalibration`
Temperature calibration `temp.csv`	same lookup	4 rows `c0;c1`	`eis_local.PlateCalibration`
Chain response `chain_gain.csv`	same lookup; built once from `Abgleich/bode/` when missing	`segment,freq_hz,gain_real,gain_imag` with `#` comments	`utils.load_gain`
Abgleich delivery	`EIS_config/<plate>/Abgleich/` (only when it has a `bode/` folder)	`coefficients/`, `bode/*.DTA`, `Step<k>_<T>Grad.csv`	`gamry_dta.read_bode_folder`, `abgleich.read_step`
HF response (optional)	`EIS_Results/<order>/hf_response/hf_response.csv` (`HF_RESPONSE = "auto"`)	per segment/card response above 1 kHz	`silver`
CSV measurements (alternative)	`Volume csv_files/<campaign>/<condition folder>`	bench-tool spectra or R2-D2 logger sweeps	`csv_source`

6.2 FAMOS card recordings

File names. Each card of each condition and test is one file. The pipeline accepts five naming conventions (`core/config.py: FAMOS_PATTERNS`), tried in order:

```
Leepa_{leepa}_Current_{cond}_Test_{test}_Karte_*.DAT       e.g. Leepa_2612030_Current_45A_Test_01_Karte_3.DAT
Leepa_RO{leepa}_Current_{cond}_Test_{test}_Karte_*.DAT
Leepa_RO{leepa}_Current_{cond}_Test_*_Karte_*.DAT
RO{leepa}-*_Current_{cond}_Test_{test}_Karte_*.DAT          e.g. RO2612091-01_Current_450A_Test_01_Karte_1.DAT
{leepa}_Current_{cond}_Test_{test}_Karte_*.DAT
```

`bronze.discover_files(cfg)` takes the **first pattern that matches** and refuses a file set that spans more than one current. The widgets read the conditions and tests from the same names (`Session.conditions_and_tests`: `_Current_<cond>_Test_<nn>_`).

Two FAMOS dialects. `eis_local.FamosFile(path)` is a factory (`__new__`) that returns a `FamosV1` or a `FamosV2` object with the same interface (`channel(name)`, `position`, `segment_names`, `uc_names`, `temp_names`):

- **v1** packs every channel name into one `|CP` field as `7,32,<name>`; float32 samples.
- **v2** (DASYLab 14) writes one `|CN` block per channel with a per-channel byte offset; float64 samples.

It checks the parse is plausible (`_famos_plausible`) because the v1 reader does not raise on a v2 file, tries both orders, and reports every attempt when nothing works. Files smaller than `MIN_FAMOS_BYTES` are refused as placeholders (campaign folders carry empty files for unconnected cards). `famos_start_time(path)` reads the trigger time from the `|NT` header field.

Channel classification (`bronze.inventory_channels`): names made of digits → segment; `UC*` → voltage reference (fixed to `ref_channel = "UC2"`, by name, never by "largest signal"); `Temp*` → temperature. The header order of the channels is recorded as each channel's **ADC slot** — silver's skew correction needs it.

FAMOS from datago (`FAMOS_FROM_DATAGO = True`, default in this package). For each run `Session.famos_dir(cond)` builds a scratch folder `/tmp/eis_<uid>/datago_famos/RO<order>/<cond>_Test<nn>/` with every card, each from the **first source that passes its checks**:

```
for every card of this condition + test:
  1. datago.raw_card     the ORIGINAL uploaded .DAT (path from the datago file record,
                         or DATAGO_RAW_FILE_TEMPLATE, or found by name under Famos/)
                         → exact, any FAMOS version, the only datago source of a v2 card
  2. datago.famos_card   the card's rows from values_delta, sorted per channel and
                         written as a FAMOS-v1 .DAT (float32, |NT start time); used ONLY if
                           - eis_local.channel_integrity: one distinct signal per channel
                           - datago.compare_with_dat: same samples as the Volume .DAT (when it exists)
                         channel (slot) order taken from a Volume .DAT of the same card
  3. Volume .DAT         symlinked (or copied) into the folder — FAMOS_VOLUME_FALLBACK = True
  else                   card left out, with a message naming the missing file
```

Why the checks: datago's importer of **FAMOS-v2** cards ignores the per-channel byte offset and stores `UC2` under all 16 channel names (RO2612085 450 A Karte_3). Such a card is rejected and read from the Volume. Samples datago holds twice under the same time are kept once (RO2611976 was ingested twice). The scratch folder is deleted after the run (about 350 MB per card), and `run_identity.json` records which source every card came from (`famos_source`).

6.3 Datago (Spark SQL)

`runner/datago.py` reads four Delta tables (names in `runner/settings.py`). Every function takes `spark` explicitly; nothing works without a Spark session.

Setting	Table	Used for
`DATAGO_META_TBL`	`ps_xplatform_dev.rvadvtec_ops.datago_advtec_metadata`	`measurement_id` ↔ `orderId` (`RO2612090`)
`DATAGO_FILES_TBL`	`…datago_advtec_metafiles_delta`	one row per uploaded file: `file_id`, `raw_file_name`, `measurementBegin`, bucket/folder columns
`DATAGO_GP_TBL`	`…datago_advtec_generalproperties`	Gamry header: `measurement_type` (`GALVEIS`), `STARTTIME`, `IACREQ`, `FREQINIT`, `FREQFINAL`, `PTSPERDEC`, begin/end
`DATAGO_VAL_TBL`	`…datago_advtec_values_delta`	the samples: `file_id`, `channel`, `time_value`, `value`

Key functions: `orders(spark)` lists orders that have a `GALVEIS` sweep; `files(spark, leepa)` lists an order's files and classifies each (`classify`: name with `CurrVal_<n>` → `gamry` + condition; `.dat` → `famos`; other `.mf4` → `bench`); `gamry_dir(spark, leepa, volume_root)` writes **every** sweep of the order as a `.dta` with its `STARTTIME` into `/tmp/datago_gamry/RO<order>/` (sweeps with fewer than `DATAGO_MIN_SWEEP_POINTS = 10` points are taken from the Volume instead); `bench_channels(spark, leepa)` returns the bench log's channels; `famos_cards`, `raw_card`, `famos_card` serve §6.2.

Switches (all in `runner/settings.py`): `DATAGO_FIRST` (master switch), `GAMRY_FROM_DATAGO`, `BENCH_FROM_DATAGO`, `FAMOS_FROM_DATAGO`, `FAMOS_VOLUME_FALLBACK`, `DATAGO_RAW_FILE_TEMPLATE`, `DATAGO_LOCAL_TZ = "Europe/Berlin"` (the Gamry's `STARTTIME` is local time).

6.4 Gamry sweeps: which one belongs to the run?

An up/down ramp measures every current twice (RO2612086: `HFR_103` at 08:39 and `HFR_111` at 09:34, both 60 A), and a shared `Gamry/` folder holds sweeps of many cells. `gamry_compare.find_cell_sweeps(root, order_id, version)` keeps only files naming **this order or this build token** (`V26_092`, read from file names by `version_of`; `Session.plate_version` takes the most frequent token among the order's file names, or `PLATE_VERSION_OVERRIDE`). Then `select_sweep` picks one:

- `GAMRY_MATCH = "file"` (default): by file name — order/build, current, and the test number (`Test_01` = the first sweep in HFR-number order). Time stamps are not used.
- `GAMRY_MATCH = "time"`: only a sweep overlapping the FAMOS recording's header stamps (within 30 min).

Synchronisation itself never uses time stamps: the cards are aligned by `UC2`, the Gamry by its frequency steps (`gamry_sync`, §9). Every plot draws the sweep the run was compared with.

6.5 Plate calibration files and the plate lookup

`core/site_paths.py: resolve(plate, root, package_dir)` finds the plate files. Lookup order, first hit wins:

1. `<EIS_CONFIG_ROOT>/<plate>/` — the shared Volume folder (`$EIS_CONFIG_ROOT` overrides it on the command line)
2. `<package folder>/EIS_config/<plate>/` — so a freshly imported package runs even before the upload
3. gen1 only: the current user's own workspace folder (old location; never another person's)

The plates are never mixed. A gen2 run never borrows a gen1 file: the two calibrations differ by about 1.45× in absolute level, so every impedance would be scaled wrongly without any error.

File	Content	Formula
`curr.csv`	72 rows `c0;c1`, one per segment (row *k* = segment *k*)	$K(T) = c0 + 10^{-3}\cdot c1\cdot T$ [V per A/cm²]; j = u/K
`temp.csv`	4 rows `c0;c1`, one per sensor	$T = (u - c0)/c1$ [°C]
`chain_gain.csv`	`segment,freq_hz,gain_real,gain_imag` (# `convention=pipeline`: gain = 1/H, Z is divided by it)	—
`Abgleich/Step<k>_<T>Grad.csv`	per segment: four sensor voltages and five DC (i_s, u_s) points at plate temperature T	`abgleich.fit_coefficients` rebuilds `curr.csv` / `temp.csv` from them

File	Content	Formula
`Abgleich/bode/*.DTA`	72 Gamry sweeps (100 kHz → 1 Hz) of each segment's current chain, no cell	`gamry_dta.write_gain_csv` turns them into `chain_gain.csv`

Building `chain_gain.csv`. When a plate has no `chain_gain.csv` but has `Abgleich/bode/`, the Selection cell builds it once (`Session._chain_gain`) and writes it into the shared folder so the next user finds it. It first cross-checks the bode index against `curr.csv` (`gamry_dta.cross_check_abgleich`); on the Naboo delivery the pairing does not hang together (r = +0.41), so the index-free **plate median** is written (`shared` mode), which still removes the common −11° roll-off at 4.5 kHz.

> **In this repository** `EIS_config/gen1/` and `gen2/` contain `curr.csv`, `temp.csv`, `chain_gain.csv` and the `Abgleich/` Step files and coefficients, but **not** the `Abgleich/bode/` sweeps or the plate drawings (`DATA_NOT_IN_GIT.md`). They are only needed to rebuild `chain_gain.csv`, which is present for both plates.

6.6 CSV measurements

With **Measurement file format = `csv`** the widgets list campaign folders under `CSV_ROOT = <Volume>/csv_files` (`csv_source.campaigns`) and map each operating-point folder to a condition by its current (`csv_source.condition_map`: 449 A → `450A`). Two kinds of folder are recognised:

- **Bench-tool spectra** (`is_spectra_dir`): a folder like `Spectrum2_65degC_449A/` holding `z<n>.csv`, one spectrum per segment, already evaluated (`read_spectra_dir`).
- **R2-D2 logger sweep** (`read_r2d2_sweep`): `metadata.csv` plus `p1.csv, p2.csv, …`, one point file per frequency. The second header row, `timeshifts`, is the acquisition instant of each column inside a row in µs; the `s` columns are already current density in A/cm² and the temperatures already °C, so `curr.csv` is **not** applied again.

`csv_source.detect_dialect(path)` also reads three generic layouts: wide time series (`time_s, u_cell, s1..s72`), long time series (`time_s, segment, u_s`) and ready-made spectrum tables (`segment, freq_hz, Z`). The result is always one `CsvMeasurement` object. For a CSV run the order is named `csv_<campaign>`.

7. Where every result is stored

7.1 The three places

Place	What	Lifetime
Scratch `/tmp/eis_<uid>/`	Where each run is computed (`<order folder>/<condition folder>/`), rebuilt FAMOS cards (`datago_famos/`), runs restored from the tables (`from_tables/`), viewers when the Volume is not writable	Lost with the cluster
Run folders `EIS_Results/` on the Volume	The saved result of every run, copied from scratch when the run finishes (`Session._save`, replacing what was there)	Permanent; this is the source of truth
Delta tables `ps_xplatform_dev.rvadvtec_dev.local_eis_*`	A queryable copy of the result files of every published run	Permanent; replaced per run on publish

7.2 The run folder

```
EIS_Results/
└── <order>[_gen2][_rot180]/                e.g. 2612090/, 2612091_gen2/, 2612095_rot180/
    ├── <condition>_Test<nn>[_<mode>]/      ONE RUN, e.g. 450A_Test01/  (csv runs: <condition>/)
    │   ├── config_used.json                every setting of this run (Config.to_dict)
    │   ├── run_identity.json               settings digest, runner version, FAMOS source per card, elapsed, written
    │   ├── run_manifest.json               summaries of every stage, plate, elapsed time
    │   ├── plausibility.csv                the plate-level verdicts (§12.3)
    │   ├── gamry_comparison.csv            aggregate vs Gamry: one row (incl. bench current at the sweep)
    │   ├── gamry_comparison_curves.csv     both curves on the shared band
    │   ├── gamry_reference_full.csv        the whole Gamry sweep as recorded (mΩ·cm²)
    │   ├── gamry_comparison.png
    │   ├── frequency_response/             ex-situ + in-situ chain response (png, csv, summary.json)
    │   ├── bronze/                         §9.3
    │   ├── silver/                         §10.6
    │   ├── gold/                           §11.4
    │   └── ecm/                            ecm_fits.json, ecm_parameters.csv  (written by R.fit_ecm)
    ├── plots/                              the drop-down viewers of the order (written by the show_* cells)
    │   ├── nyquist.html  median_nyquist.html  heatmaps.html  frequency_response.html
    │   ├── mf4_bench_log.html  ecm_fits.html
    │   └── <condition>_Test<nn>/           plate_*.png, ecm_aggregate.png, ecm_parameters.csv, ecm_fits.json
    └── hf_response/                        optional frozen HF response of the order (R.build_hf_response)
        ├── hf_response.csv  hf_response_segments.csv  hf_response_validation.csv
        └── history/                        files it replaced
EIS_Results/_overview/trends.html          plate parameters against current, every order (02 Results)
```

Naming rules (`Session.order_dir`, `Session.cond_name`):

- gen2 adds `_gen2`, a rotated plate adds `_rot180`, both: `<order>_gen2_rot180`.
- A non-default `EVALUATION_MODE` (`permissive`/`strict`) adds `_<mode>` to the run folder.
- A folder counts as a result only if `silver/spectra_clean.csv` (or `csv/spectra_clean.csv`) exists (`Session._usable`).
- The **run key** is the path under `EIS_Results`, e.g. `2612090/450A_Test01`. It is the key of every table row.

7.3 The Delta tables

`runner/tables.py` copies result files of a run folder into tables named `<TABLE_SCHEMA>.<TABLE_PREFIX><suffix>` = `ps_xplatform_dev.rvadvtec_dev.local_eis_<suffix>`:

Table	Copied from	One row per
`local_eis_runs`	`run_identity.json` + manifests (`tables.run_record`)	run: digest, runner, settings JSON, FAMOS source, cards, Gamry sweep and build, segments measured/inferred/substituted, plausibility fail/warn counts, elapsed, folder, publication time
`local_eis_segment_meta`	`bronze/segment_meta.csv`	segment: card, K, T, u_dc, j_dc, slots
`local_eis_bronze_spectra`	`bronze/raw_spectra.csv`	segment × frequency (raw)
`local_eis_silver_spectra`	`silver/spectra_clean.csv`	segment × frequency (corrected)
`local_eis_silver_reconstructed`	`silver/spectra_reconstructed.csv`	rebuilt segment × frequency
`local_eis_silver_segments`	`silver/segments_summary.csv`	segment: tier, R_ohmic, flags…
`local_eis_gold_segments`	`gold/plate_summary.csv`	segment: every parameter, class, fault
`local_eis_gold_plate`	`gold/plate_statistics.csv`	parameter: 36-tile statistics
`local_eis_gold_cell`	`silver/cell_aggregate.csv`	frequency: the parallel sum
`local_eis_gamry_reference`	`gamry_reference_full.csv`	Gamry point
`local_eis_gamry_comparison`	`gamry_comparison.csv`	comparison
`local_eis_checks`	`plausibility.csv`	check
`local_eis_ecm`	`ecm/ecm_parameters.csv`	spectrum × fit

Every row carries the key columns `run_key`, `order_id`, `plate`, `orientation`, `condition`, `current_a`, `test_id`, `eval_mode`; tables are partitioned by `order_id`. **Publishing replaces a run's rows** (`replaceWhere` on `run_key`), so the tables never hold two versions of one run; the `runs` row is written last, so a run listed there has all its tables. Before writing, column names are made Delta-safe and the key columns put first (`tables.tidy`), then every column is given the type it already has in the table (`tables._align`): pandas guesses a CSV column's type per file (`L_nH` = 12, 15, 9 reads as integers in one run, 12.4 as decimals in the next) and Delta cannot merge two types. A column that cannot take the table's type without losing values is **widened in the table first** (bigint → double, number → string), which rewrites that table once. (`PIPELINE_GUIDE.md` §14 describes a newer rule that refuses such a write instead; the `tables.py` shipped here widens.)

Example query:

```
SELECT order_id, test_id, current_a, median AS R_ohmic_median
FROM ps_xplatform_dev.rvadvtec_dev.local_eis_gold_plate
WHERE parameter = 'R_ohmic'
ORDER BY order_id, current_a;
```

> **Known gap in this package** (§21.2). The settings, the notebooks and the docs describe a 14th table, `local_eis_run_files` (every file of a run byte for byte), with `tables.restore()`, `tables.rebuild()` and `tables.file_bytes()`. The `runner/tables.py` shipped here does not contain them. Publishing the 13 tables above works; restoring a run from the tables and the Maintenance "Rebuild all tables" cell do not.

7.4 The cache: `run_identity.json` and the run modes

Every saved run carries `run_identity.json`:


```
{ "digest": "3f9a1c07b2",
  "settings": { "plate": "gen1", "plate_orientation": "standard", "test_id": "01",
                "f_min_hz": 0.15, "full_band": true, "min_snr_db": -10.0,
                "channel_lag": "correct", "gamry_sync": "guide", "skew_model": "structural",
                "...": "...", "gain_file_sha": "a1b2c3d4e5f6", "hf_response_file_sha": "" },
  "runner": "v35.1", "condition": "450A", "test_id": "01", "order": "2612090",
  "cards": 5, "famos_source": {"Karte_1": "datago (original file)", "Karte_3": "Volume"},
  "elapsed_s": 412.3, "written": "2026-10-21T14:02:11" }
```


The digest is a SHA-256 over the values of `session.IDENTITY_KEYS` (the settings that decide what a run keeps) **plus the content hashes of the chain-gain and HF-response files**. It is *not* a hash of the code.

Run mode	Saved folder usable?	What happens
`cache`	yes	loaded; the Evaluate cell prints `saved` and, if the digest differs, `<-- made with OTHER settings (…); choose rerun`, or `saved by runner vXX`
`cache`	no, but the run is published	(`CACHE_FROM_TABLES`) restore from the tables — currently fails over to computing, see §21.2
`cache`	no	computed in scratch, then copied to the saved folder
`rerun`	—	computed in scratch, then the saved folder is **deleted and replaced**

8. The life of one run, call by call

The numbered steps are the real call order when `01 Local EIS - Process` evaluates one condition with Run mode `rerun`. Search for `def <name>` in your editor to follow along.

#	Where	What happens
1	Notebook *Setup*	finds its own workspace folder, puts it on `sys.path`, `import eis_paths`, `import runner`, `R = runner.Session(globals(), package_dir=…)` (picks up `dbutils`, `spark`, `displayHTML`; creates `/tmp/eis_<uid>`)
2	`Session.widgets`	removes old widgets; `orders()` (datago + Volume); creates the dropdowns; `conditions_and_tests()`; `geom.use_plate(plate, orientation)`; `site_paths.resolve()`; `_chain_gain()`; `plate_version()`; `gamry_dir()` (datago → Volume); `gamry_compare.find_bench_log()`; prints the selection
3	`Session.run`	for each condition: skip if no files for this test; `saved_dir`, scratch dir; `_config(cond, scratch)` builds `cfg = DEFAULT.replace(…)` from the widgets and `runner/settings.py`; `identity(cfg)`
4	`Session.run`	cache decision (§7.4); for FAMOS-from-datago `famos_dir(cond)` builds the card folder; `bronze.discover_files(cfg)` counts cards (one card → `min_ref_channels = 1`)
5	`main.run_pipeline(cfg)`	`geom.use_plate()` **before anything reads the geometry**; CSV → `csv_pipeline.run_csv` (§13); writes `config_used.json`
6	`bronze.run` / `bronze.save`	`discover_files` → `inventory_channels` (FamosFile) → `PlateCalibration.load` → `estimate_card_lags` → `plate_temperatures` → `consensus_schedule` (detect, ladder snap, Gamry sync, window repair) → `refine_frequencies` (tone estimation) → `card_gain.measure_reference` → `process_card` per card → `bronze/*.csv`
7	`silver.run` / `silver.save`	`dc_share_faults` → skew model per card → `channel_lag.estimate` → card phase → `process_segment` per segment (gates, weights, DRT, circuit fit, R_ohmic, lin-KK, Z-HIT, tier) → DC closure → rebuilt segments → cell aggregate → `silver/*.csv`
8	`gold.run` / `gold.save`	`collect_parameters` → `fit_field` (only for missing segments) → `label_faults` → 36-tile statistics → `plate_<param>_interp.png`, `nyquist.png/.html`, `plate_summary.csv`, `plate_statistics.csv`
9	`main.whole_cell_and_plausibility`	`gamry_compare.run` (+ `gamry_comparison.png`) → `frequency_response.run` → `plausibility.check_run` → `plausibility.csv`; `run_manifest.json`
10	`Session.run`	deletes the rebuilt-card scratch folder; writes `run_identity.json`; `_save(scratch, saved)` copies the run to `EIS_Results/…`
11	`Session.fit_ecm`	`views.ecm_fits(run_dir)` → standard ECM (`standard_eis`) + DRT-informed ECM (`ecm_drt`) for the aggregate and every segment → `ecm/ecm_fits.json`, `ecm/ecm_parameters.csv`
12	`Session.publish`	`tables.publish(spark, run_dir)` for every run → 13 tables, runs row last
13	`Session.summary`	`tables.run_record()` per run → one display table

8.1 The data objects handed between steps

Every arrow between layers carries one of these dataclasses. Knowing them is the fastest way into the code.

Object	Defined in	Holds	Flows
`Config` (frozen)	`core/config.py`	every setting of one run; `DEFAULT` is the default instance	built by `Session._config` / CLI, read by every stage
`Segment`, `Plate`	`core/r2d2_geometry.py`	pads of a segment, area, centroid; one plate generation	geometry, everywhere
`PlateCalibration`	`pipeline/eis_local.py`	`c0, c1` per segment and sensor; `K(seg, T)`, `temperature()`	bronze
`Step`	`pipeline/eis_local.py`	one excitation step: freq, start, stop, amplitude, SNR, THD, stationarity	bronze schedule
`ChannelInfo`, `CardInfo`	`pipeline/bronze.py`	one channel (card, ADC slot, kind); one card (fs, n_ch, reference)	bronze inventory
`BronzeSpectrum`	`pipeline/bronze.py`	raw spectrum of one segment: freq, Z_raw, SNRs, THD, drift, samples/step, slot, K, T, u_dc	bronze → silver
`BronzeRun`	`pipeline/bronze.py`	schedule, channels, cards, spectra, digests; `load(run_dir)` rebuilds it from CSV	bronze → silver

Object	Defined in	Holds	Flows
`SkewModel`	`pipeline/silver.py`	per-card ADC skew: basis, dt0, k_slot, applied	silver
`ChannelLag`	`pipeline/channel_lag.py`	per-segment τ, residual, status	silver
`SilverSpectrum`	`pipeline/silver.py`	corrected + modelled spectrum, R_ohmic ± sd, R_pol, DRT, KK/Z-HIT, tier, flags	silver → gold
`SilverRun`	`pipeline/silver.py`	spectra, skew, DC closure, cell aggregate, rejection ledger	silver → gold, checks
`SegmentRecord`, `GoldRun`	`pipeline/gold.py`	one segment on the finished plate (class, tier, values, sd, fault); all records + statistics	gold → save, manifest
`CsvMeasurement`	`readers/csv_source.py`	whatever a CSV held, in one shape	csv_source → csv_pipeline
`GamrySweep`, `CellSweep`	`readers/gamry_dta.py`, `checks/gamry_compare.py`	a Gamry `.DTA` sweep	chain gain, comparison
`Check`, `Report`	`checks/plausibility.py`	one verdict / all verdicts	`plausibility.csv`

Part 3 — The processing

The defaults below are the ones **the notebooks use**: `runner/settings.py` overrides several library defaults of `core/config.py` (marked "runner"). Where they differ, the runner value wins for every notebook run; a command-line run uses `config.py` unless you pass the flag.

Setting	`config.py` default	Runner value	Effect
`full_band` / band	`False` (0.15 Hz – 2.4 kHz)	`True` (0.15 Hz – 0.49·fs, 12.25 kHz at 25 kHz)	evaluated band
`skew_model`	`"none"`	`"structural"`	multiplexer slot delay removed
`gamry_sync`	`"report"`	`"guide"`	Gamry frequencies + windows re-located in their Gamry slot
`hf_use_ensemble`, `hf_ladder_prune`	`False`, `False`	`True`, `True`	top-decade step detection
`f_hi_frac_fs`	0.45	0.49	detection ceiling
`min_snr_db`	10	−10	bronze off-grid step gate
`window_min_snr_db`	−3	−4	analysis-window SNR floor at the top steps
`channel_lag`, `card_gain`	`correct`, `correct`	`correct`, `correct`	—
`min_ref_channels`	2	2 (1 if the run has one card)	consensus schedule

9. Bronze: raw waveforms → raw impedance

`pipeline/bronze.py`, helped by `eis_local.py`, `tone_estimation.py`, `ladder_snap.py`, `hf_schedule.py`, `gamry_sync.py` and `checks/card_gain.py`.

Contract. Bronze writes only measured values, or values that follow algebraically from the calibration. No de-skew, no inductance removal, no smoothing. That discipline is what makes `--reevaluate` possible: silver can be re-run in seconds from saved bronze tables instead of re-reading tens of GB of `.DAT` files. The only things bronze changes are the chain gain, the card voltage gain and an obvious polarity flip.

9.1 The steps of `bronze.run(cfg)`

1. **Find the card files** — `discover_files(cfg)`: first matching `FAMOS_PATTERNS` entry, sorted so the card order is deterministic.
2. **Inventory the channels** — `inventory_channels`: opens each card once with `FamosFile`, classifies channels, records each channel's ADC slot and each card's sample rate.
3. **Calibration** — `PlateCalibration.load(curr_cal, temp_cal)`. A segment missing from `curr.csv` is imputed from the plate median and flagged `K_imputed` (tier capped at B).
4. **Align the cards** — `estimate_card_lags`. The cards are armed separately (one recording had card 3 5.7 s late).
   - Each card's `UC2` is band-passed to 0.5–300 Hz and cross-correlated (FFT) with the anchor card's; the anchor is the card the others agree with best (`_pick_anchor`).
   - A lag is **applied** only if |r| ≥ 0.5 (`align_min_corr`) and the peak stands ≥ 15 robust σ above the background (`align_min_prominence`); a weaker lag is accepted when another card independently reproduces it (`_agreeing_cards`).
   - Cards at another sample rate are resampled for the correlation; windows are converted with `to_own(idx) = round(idx·ratio) + lag`.
   - Clock drift (ppm, `_blockwise_lag_diagnostic`) and pairwise closure (`_pairwise_closure`) are reported, not applied.
5. **Plate temperatures** — `plate_temperatures`: the four `Temp_*` channels through `temp.csv`, interpolated linearly in x to every segment centroid (fallback 58.4 °C without sensors).
6. **One consensus schedule** — `consensus_schedule`: *which stretch of the record holds which frequency.*
   1. Per card, `eis_local.detect_schedule` scans a log grid of trial frequencies on the reference: demodulate, keep envelope peaks 4× above the median, grow a dwell window, fit with `fit4` (IEEE 1057, frequency free), polish the window, de-duplicate, fill gaps. With `hf_use_ensemble` the top decade is detected on the stacked segment channels (`hf_schedule`), because the cell removes the tone from the voltage channel exactly where its impedance is lowest.
   2. Every card's steps go onto the common time base and are clustered by frequency (`grid_tol` = 1 %). A step is accepted if ≥ 2 cards saw it **or** it lies on the fitted geometric grid.
   3. **Ladder snap** (`ladder_snap.snap_steps`): duplicates merge, frequencies snap to the exact rungs f_k = 10^((k+offset)/ppd). **Window repair** (`repair_windows`) interpolates windows that break the time order; a step is refused if > 25 % would need it.
   4. **Gamry clock** (`gamry_sync`, mode `guide`): see 9.2.
   5. Overlapping neighbour windows are cut at the midpoint.
7. **Exact frequency per card and step** — `refine_frequencies` → `tone_estimation.card_frequency`: a multichannel maximum-likelihood search (variable projection over all channels of a card) gives one frequency per card and step, within ±1 FFT bin of

the schedule.

8. **Card voltage gain** — `card_gain.measure_reference`: each card's `UC2` phasor against the median card, from the AC steps 1–300 Hz with SNR ≥ 30 dB. With `card_gain = "correct"` it is divided out of Z.
9. **Phasors and impedance** — `process_card`, for every segment and step: read the window, fit voltage and segment phasors at the step frequency (`tone_estimation.phasors_at`: least squares on cos, sin, offset, ramp), **Z = K(T)·A_ref/A_seg**, record THD, χ^2 stationarity (drift), SNR of reference/segment/combined, samples per step; divide by the chain gain (`utils.gain_at`) and the card's voltage gain; flip the sign if Re Z is negative over the lowest decade (`_fix_polarity`).
10. **Duplicates** — if two cards report the same segment, the one with the higher median SNR wins.

9.2 The Gamry clock (`pipeline/gamry_sync.py`)

The FAMOS cards record the cell's response to the **Gamry's own sweep**, so the `.dta` knows each step's exact frequency and the second it ended (on 2612030/60 A every FAMOS window ends at t_Gamry + 99.24 s, spread 0.44 s).

Function	What it does
`align`	every confidently detected window votes for an offset (window end − Gamry time); the median is accepted if ≥ 5 voters agree within 1.5 s; each Gamry point is judged `ok`, `misplaced` or `missing`
`apply_frequencies`	steps judged `ok` take the Gamry's exact frequency (the ladder was up to 1 % off)
`relocate` (`guide`)	misplaced, missing, interpolated and empty windows are searched again inside their Gamry slot; accepted only if two independent array tests agree (CFAR channel count and rank-1). A prediction alone is never accepted
`confine_windows`	low frequencies (interval ≥ 3 s): the window sits strictly between two Gamry step ends, with a guard and half a period of settling
`corroborate_card_lags`	a card lag alignment refused is applied when the Gamry clock independently implies the same lag

It refuses (and changes nothing) when fewer than 5 confident steps agree on one offset. It needs the raw `.DAT`, so `--reevaluate` cannot apply it to saved spectra.

9.3 Bronze files (`<run>/bronze/`)

File	Content
`raw_spectra.csv`	segment, card, freq_hz, z_re/z_im, snr_ref/seg/comb_db, thd, drift, n_samples, on_grid
`segment_meta.csv`	per segment: card, area, centroid, channel_slot, ref_slot, fs, K, K_imputed, T, u_dc, j_dc
`schedule.csv`	one row per step: freq, start/stop sample, amplitude, SNR, THD, drift, `window_source` (detected / gamry / gamry_step / interpolated), coverage
`card_alignment.csv`	per card: lag (samples, s), correlation, prominence, applied, ratio, rescued, clock ppm
`card_reference.csv`	per card: UC gain against the median card
`gamry_sync.csv`	per Gamry point: offset residual, verdict, relocation
`gamry_card_sync.csv`	card lags corroborated by the Gamry clock
`frequency_coverage.csv`	how much of the plate carries each frequency
`channels.csv`	every channel of every card with its slot
`bronze_manifest.json`	summary: cards, steps, coverage, sensor temperatures, recording window, digests

10. Silver: correct, gate, model, grade

`pipeline/silver.py` with `channel_lag.py`, `eis_measurement_model.py`, `eis_validation.py`, `neighbours.py`, `standard_eis.py` and `checks/hf_response.py`. **Nothing is deleted silently**: every point that is dropped is recorded in a ledger with the gate that dropped it.

10.1 Before the segments: plate-level decisions

- **Channel faults** (`dc_share_faults`): a segment whose DC current density is < 0.5× or > 2× its neighbours (`segment_fault_ratio`) is left out (e.g. segment 33 reading 18 % of its neighbours) — rebuilt from neighbours when "Fill unmeasured" is on.
- **Structural skew** (`choose_global_basis`, `fit_structural_skew`): one converter scans the channels in turn, so slot *p* is sampled p/(n_ch·fs) later — 2.5 µs per slot at 16 channels and 25 kHz, i.e. up to ~150° at 12 kHz across a card. Z is de-skewed with `Z·exp(+j2πf·dt)`, `dt = k·(slot_seg − slot_ref)`; *k* is fitted per card within 0.5–1.6× nominal. The common delay `dt0` is held at 0 (`fit_common_delay = False`): it cannot be told apart from a series inductance.
- **Channel lag** (`channel_lag.estimate`, mode `correct`): each segment's shunt/trace/amplifier chain adds its own delay (−57… +76 µs on 2612030). Per segment, the phase of `Z_seg / Z_plate-median` between 50 Hz and the coherent limit is fitted with a pure delay; three iterations as the plate median is rebuilt. τ is applied only if the rms residual ≤ 10° and |τ| ≤ 250 µs; otherwise the segment is flagged (`not_first_order`, `too_large`) and forced to tier C. A delay **every** channel shares is invisible here (it is measured against the plate median).

- **Card phase above 1 kHz** (`card_hf_phase`, `gamry_card_phase`, full band only): each card's common phase above the coherent limit, calibrated against the condition's Gamry sweep.
- **HF response** (`HF_RESPONSE = "auto"`): when `EIS_Results/<order>/hf_response/hf_response.csv` exists, each segment is divided by its frozen response above 1 kHz (built across conditions by `R.build_hf_response()`, validated leave-one-current-out).

10.2 Point gates (`gate_points`) — the first gate to reject a point owns it

#	Ledger name	Rule (config field, default)
1	`not_finite`	f > 0 and Z finite
2	`outside_band`	`f_min_hz` (0.15) ≤ f ≤ `f_eval_max()` (0.49·fs on the full band)
3	`snr`	backstop only: SNR ≥ −40 dB (`silver_snr_gate_db`, `silver_snr_floor_db`)
4	`off_tone`	off (`off_tone_drop_db = None`)
5	`thd`	THD ≤ 0.10 (`max_thd`)
6	`drift`	drift ≤ 0.25 (`max_drift`)
7	`background`	coherent background at the step frequency before the sweep ≤ 5 % (`background_max_pct`) — mains, switching tones, cell oscillation
8	`magnitude`	|Z| inside a plausible range
9	`uncertainty`	CRLB σ_rel = √(1/(N·γ)) ≤ 0.60 (`sigma_rel_max`) — **the gate that really decides**
10	`cycles`	≥ 3 cycles in the window (`min_cycles_per_dwell`)
11	`zmag_outlier`	$\log_{10}$|Z| within 4.5 robust sd of a 7-point running median
12	`passivity`	after the timing corrections: Re Z > 0 for f ≥ 1 Hz (`passivity_gate_min_hz`)

A segment with fewer than 8 surviving points (`min_points_per_spectrum`) is dropped and its verdict recorded. `silver/segment_reach.csv` names, per segment, the highest kept frequency and the gate that stopped it.

10.3 Modelling: R_ohmic, R_ct, R_mt, R_pol

`param_method = "drt"` (the code default; the runner does not change it) with `ecm_compare = True`:

- **Bayesian DRT** (`bayesian_drt`): a Gaussian-process prior on the distribution of relaxation times, closed-form posterior, every point weighted by its Cramér–Rao uncertainty. R_pol is the area of the DRT; R_ct is the part faster than `tau_split_kinetic_s` (10^{-2} s), R_mt the slower part. The DRT's own R_∞ is reported as `R_ohmic_drt`.
- **R_ohmic** (`r_ohmic_method = "auto"`): on the full band the high-frequency **real-axis crossing** (Im Z changes sign), interpolated; when the band ends before a crossing, Re Z extrapolated to Im Z = 0 along a straight line through the top points (`standard_eis.hf_intercept`, `silver.r_ohmic_axis_fit`). `R_ohmic_topband` (inverse-variance mean of Re Z over the top 0.35 decade) is written for comparison.
- **Standard circuit beside it** (`pipeline/standard_eis.py`): Z = R_ohmic + jωL + R_ct/(1+(jωτ_ct)^n_ct) + R_mt/(1+(jωτ_mt)^n_mt), complex non-linear least squares with modulus weighting, R_ohmic held at the intercept, bounds R ≥ 0, L ≥ 0, 0.5 ≤ n ≤ 1, τ_mt ≥ 3·τ_ct, several starting points; only the capacitive part is fitted (the low-frequency inductive loop at high current has no element). Written as `R_ct_ecm`, `R_mt_ecm`, `R_pol_ecm`, `ecm_rms_pct`. Set `param_method = "ecm"` to make the circuit the primary method.

> The package `README.md` says R_ct/R_mt/R_pol are "computed the standard way … no DRT". The code default is the DRT with the circuit beside it. Trust the code (`core/config.py: param_method`), and check `config_used.json` of a run to see which was used.

10.4 Validation and tiers

- **lin-KK** (`eis_validation.lin_kk`, Boukamp with Schönleber's μ criterion): the maximum residual relative to |Z|.
- **Z-HIT** (`eis_validation.z_hit`): reconstructs ln|Z| from the phase; it sees pure delays, which KK cannot.
- **Tier** (`process_segment`): **A** = KK ≤ 2 % (`kk_tol`) and CV(R_ohmic) ≤ 10 %; **B** = KK ≤ 6 % and CV ≤ 25 %; **C** otherwise, or a non-physical R_ohmic. An imputed calibration caps A at B; an unreliable channel lag forces C. Gold adds **D** for segments with no data of their own (inferred).

10.5 Plate products

- **DC closure** (`dc_closure`): I = Σ j_dc·A over the measured segments, scaled to 304.92 cm², against the set/bench current.
- **Rebuilt segments** (`build_filled_spectra` → `neighbours.fill_spectrum`): the area-weighted mean of adjacent segments' model spectra (pads sharing an edge, up to `fill_max_hops = 2` rings). Only for segments listed under *Rebuild from neighbours*, or every gap with *Fill unmeasured = yes*. Labelled `substituted` in gold (written to `spectra_reconstructed.csv`), never given a quality tier (gold tier D).

- **Cell aggregate** (`cell_aggregate_full`): the segments in parallel, Y(f) = Σ A_s/Z_s(f); ASR over the covered area = A_used/Y, full-plate ASR = A_cell/Y.

10.6 Silver files (`<run>/silver/`)

File	Content
`spectra_clean.csv`	**the corrected spectra**: segment, freq_hz, z_re/z_im (mΩ·cm²), model and its sd, sigma_rel
`segments_summary.csv`	per segment: tier, R_ohmic (+sd, how), R_pol, R_ct/R_mt (DRT and ECM), L, dt applied, chain τ and status, T, j_dc, KK %, Z-HIT %, SNR, THD, points used/dropped, flags
`point_rejections.csv`	**the ledger**: one row per segment and point, kept or the gate that rejected it
`segment_reach.csv`	per segment: highest kept frequency and the gate that stopped it (`blocked_by`)
`channel_lag.csv`	per segment: τ (µs), applied, residual, status
`skew_model.csv`	per card: basis, dt0, slot µs, applied
`cell_aggregate.csv`	the parallel sum, covered-area and full-plate ASR, coverage
`card_factors.csv`	per card: |Z| and j_dc against the plate
`spectra_reconstructed.csv`	rebuilt segments, with their donors
`silver_manifest.json`	tier counts, DC closure, skew per card

11. Gold: the plate as a field

`pipeline/gold.py` answers the question the rig exists for: **where** on the plate is something wrong, and **what** is it.

1. **Parameters** (`collect_parameters`): R_ohmic, R_ct, R_mt, R_pol (`split_processes`), `ReZ_1kHz` (`re_at`: interpolated in log f between the two kept points around 1 kHz, never extrapolated), `Z_mag_100Hz` and `phase_100Hz` from the model, `j_dc`, `T_degC`, `chain_tau_us`, `sigma_rel`, each with its sd.
2. **Spatial field** (`fit_field`): Gaussian-process regression (Matérn 5/2; 70 mm along the flow, 35 mm across) over the segment centroids. **Measured segments keep their own values**; the field only fills segments without data (class `inferred`, tier D), and only if ≤ 60 % are missing (`max_inferred_fraction`). The runner passes `infer_missing_segments = False`, so in notebook runs gaps stay empty unless rebuilt from neighbours.
3. **Fault labels** (`label_faults`): ratios to the plate median — drying, flooding, starvation, contact_loss (`config.FAULT_RULES`).
4. **Statistics over 36 equal tiles** (`plate_stats.tile_stats`): the 45 × 20 pads split into 9 × 4 tiles of 5 × 5 pads (8.47 cm²); segment k = 1…36 stands for tile k, so mean and median are equal-area and the dense edge segmentation cannot pull them. A tile without a measured value is left out and the count says so ("35 of 36 tiles").
5. **Maps**: one 2D interpolated plate map per parameter, `plate_<param>_interp.png`, for R_ohmic, ReZ_1kHz, R_ct, R_mt, R_pol, j_dc, Z_mag_100Hz, phase_100Hz, T_degC (`config.heatmap_params`), on fixed colour limits (`config.HEATMAP_LIMITS`) so one colour means one value in every condition. Plus `nyquist.png` / `nyquist.html` (all segments, coloured inlet → outlet).

11.1 Gold files (`<run>/gold/`)

File	Content
`plate_summary.csv`	**one row per segment, all 72**: class (`measured` / `substituted` (rebuilt from neighbours) / `inferred` / `excluded` / `bad`), tier, position, area, fault, flags, every parameter with its sd (resistances in mΩ·cm²; exception `R_hf_extra` in Ω·cm²)
`plate_statistics.csv`	per parameter: tiles used, mean, median, sd, CV, min, max
`plate_*_interp.png`	the interpolated plate maps
`nyquist.png`, `nyquist.html`	all segments; the HTML also has |Z| and phase panels
`gold_manifest.json`	statistics, class counts, spatial trends, DC closure

12. Checks: is the result believable?

`pipeline/main.py: whole_cell_and_plausibility` runs three things after gold, for FAMOS and CSV runs alike. **None of them can stop a run**; each is wrapped so a failure becomes a message.

12.1 Gamry whole-cell comparison (`checks/gamry_compare.py`)

The 72 segments summed in parallel must reproduce what the Gamry measured on the whole cell at the same moment — the only end-to-end check, because the Gamry shares none of the calibration, geometry, synchronisation or chain. `Z_Gamry,ASR = Z_Gamry[Ω] · 304.92 cm²` is compared with `silver/cell_aggregate.csv` **on the shared band only**: magnitude and phase difference, rms, and each side's HFR. Outputs: `gamry_comparison.csv`, `gamry_comparison_curves.csv`, `gamry_reference_full.csv`, `gamry_comparison.png`. The plots draw the Gamry **as recorded** (every point, exact frequencies, no band limit, no inductance removed).

How to read a disagreement: a constant real offset → a resistance in one path and not the other; a scale at all frequencies → calibration/area; a phase growing with f → timing. Agreement does not mean the map is uninformative: the harmonic mean is dominated by the low-impedance segments.

12.2 Chain frequency response (`checks/frequency_response.py`)

- **Ex-situ**: each amplifier's bench response G_d(f) from the chain-gain file or the `bode/` folder.
- **In-situ**: each segment against the plate median, from `bronze/raw_spectra.csv` (no timing correction) and `silver/spectra_clean.csv` (after it).

Writes `frequency_response/` (`freq_response_exsitu.png`, `freq_response_insitu.png`, CSVs, `summary.json`) and two plausibility checks.

12.3 Plausibility (`checks/plausibility.py` → `plausibility.csv`)

Check	Function	Tests	Pass / warn / fail
coverage	`coverage`	measured **area** fraction	≥ 60 % / ≥ 25 % / below
measured block	`describe_block`	whole plate, block 1–36, or 37–72	whole or 1–36 pass, else warn
current closure	`current_closure`	plate current (mean j over 36 tiles × 304.92 cm²) vs set/bench current	≤ 10 % / ≤ 30 % / above
passivity	`passivity`	Re Z > 0 everywhere after correction	pass / – / any Re Z ≤ 0
card alignment	`card_alignment_check`	every card's lag applied	pass / – / a card not aligned
channel lag	`channel_lag_check`	share of segments with an unreliable lag	none / some, or a lag ≥ 30 µs / ≥ 20 %
card voltage gain	`card_voltage_check`	UC gain of each card vs the median card	≤ tolerance / not removed or not flat / > 10 %
card impedance factor	`card_factor_check`	a card that looks like a pure scale error	pass / warn
R_ohmic smoothness	`neighbour_smoothness`	neighbour contrast vs 200 random shuffles	≤ 0.6 / ≤ 0.85 / above
R_ohmic along flow	`flow_trend`	correlation of R_ohmic with position along the flow	|r| ≥ 0.15 pass, else warn
R_s matched-band closure	`matched_series_resistance`	top-band Re Z, aggregate vs Gamry, same frequencies	≤ tol / ≤ 2·tol / above
aggregate vs whole cell	`aggregate_vs_reference`	median |Z_agg|/|Z_Gamry| − 1 on the shared band	same rule
chain response (ex-situ)	`frequency_response`	amplifier flat within 1 % at 5–500 Hz, segments within 2°	pass / warn
segment response (in-situ)	`frequency_response`	phase spread over segments at the top of the band	≤ 5° / ≤ 10° / above

12.4 Other checks you run by hand

Module	Command	Answers
`checks/dc_closure.py`	`python run.py dc_closure RUN_45A RUN_60A … -o OUT [--bench x.mf4 --gamry DIR]`	do segment currents add up to the bench current at every operating point; is the gap a scale (a − 1, in every Z too) or an offset (b)
`checks/card_gain.py`	(inside bronze/silver)	per-card voltage gain; per-card impedance factors in four bands
`checks/hf_response.py`	`R.build_hf_response()` in a notebook	the frozen per-segment HF response across conditions
`checks/validate_hf.py`	runs inside `python run.py main --self-test`	the high-frequency chain against synthetic spectra with known truth

13. The CSV pipeline

`pipeline/csv_pipeline.py: run_csv(cfg)`, reached from `run_pipeline` when `cfg.source_format == "csv"`.

Deliberately not done: no inter-card synchronisation, no clock-drift regression, no structural skew fit. One logger writes one row per instant; there is no second clock, so there is no delay to estimate, and an estimator pointed at a quantity that does not exist returns noise with a plausible uncertainty.

Done instead:

- the **channel scan** of the R2-D2 logger is corrected analytically from the printed `timeshifts` (segment 1 at 0 µs, cell voltage at ~80 µs: 29° at 1 kHz) — exact, no fit (`apply_scan_skew`);
- non-uniform time bases are fitted with `csv_source.fit_phasor_nonuniform`;
- the tone is found on the segment-averaged spectrum and the record is windowed to the **burst** (point files carry 0.25 s lead-in and 0.4 s lead-out with a ~999 Hz artefact); a record whose strongest common tone is under 10× the noise floor is refused;

- a point whose analogue frequency is above Nyquist is reported (aliased), and `cfg.csv_tones` can pass the sweep's own frequency list;
- the channel lag stage runs as in silver; the standard ECM gives the parameters.

Outputs: `csv/` (spectra, ECM parameters, cell aggregate), plus `silver/spectra_clean.csv`, `silver/cell_aggregate.csv`, `silver/channel_lag.csv`, `gold/plate_summary.csv`, `gold/plate_statistics.csv` and the plate maps in the same format as the FAMOS path, so every viewer, check and table works on both. Then the Gamry comparison and plausibility run as for FAMOS.

14. ECM fits in the runner

`R.fit_ecm()` (notebook 01) and `R.show_ecm()` (notebook 02) call `runner/views.py: ecm_fits(run_dir)` for the **aggregate and every segment**:

- **Standard ECM** (`views.fit_standard` → `standard_eis.fit_spectrum`): R_ohmic + L + two ZARCs, complex NLLS.
- **DRT-informed ECM** (`views.fit_drt` → `analysis/ecm_drt.py`): Tikhonov/NNLS DRT → its peaks → Rs + jωL + fast arc [+ slow arc], arcs sorted by τ and reported as `fast`/`slow`, one or two arcs chosen by AICc, weighting by the CRLB σ when known, points above `ECM_DRT_F_MAX_HZ = 1000` left out, Rs held at the pipeline's R_ohmic (`ECM_DRT_FIX_RS`), optional TLM fast element (`ECM_DRT_FAST = "auto"`).

Saved as `RUN/ecm/ecm_fits.json` and `RUN/ecm/ecm_parameters.csv` (reused unless `refit=True`); the viewer shows Nyquist, DRT γ(τ), |Z| and residual for Condition × Spectrum × Fit (standard / DRT-informed / both).

15. Formula sheet

Units: area-specific impedance in Ω·cm² inside the code and mΩ·cm² in every output file; currents in A; areas in cm²; temperatures in °C.

Quantity	Formula	Code
Segment calibration	K(T) = c0 + 10⁻³·c1·T (V per A/cm²)	`eis_local.PlateCalibration.K`
Sensor temperature	T = (u − c0)/c1; segment T linear in x between the 4 sensors	`PlateCalibration.temperature`, `r2d2_geometry.segment_temperatures`
Phasor	least squares y = a·cos ωt + b·sin ωt + c + d·t → A = a − jb	`tone_estimation.phasors_at`
Local impedance	**Z = K(T)·A_U/A_seg**, then ÷ chain gain(f), ÷ card voltage gain	`bronze.process_card`
ADC de-skew	Z ← Z·exp(+j2πf·k·(slot_seg − slot_ref)), k ≈ 1/(n_ch·fs)	`silver.process_segment`
Channel lag	fit arg(Z/Z_plate-median) = −2πfτ; Z ← Z·exp(+j2πfτ)	`channel_lag.estimate`
Point uncertainty	σ_rel = √(1/(N·γ)), γ = linear SNR, N = samples per step (CRLB)	`utils.sigma_rel_from_snr`
DC current density	j = u_dc/K(T) (A/cm²)	`SilverSpectrum.j_dc`
DRT model	Z = R_∞ + jωL + Σ γ_m/(1 + jωτ_m), γ ≥ 0	`silver.bayesian_drt`
R_ohmic	Im Z = 0 crossing, or Re Z = a + b·Im Z through the top points, read at Im Z = 0	`standard_eis.hf_intercept`, `silver.r_ohmic_axis_fit`
Standard circuit	Z = R_ohmic + jωL + R_ct/(1+(jωτ_ct)^n_ct) + R_mt/(1+(jωτ_mt)^n_mt)	`standard_eis.fit_spectrum`
R_pol	R_ct + R_mt (graphical check: Re Z(f_min) − R_ohmic)	silver / `standard_eis`
Arc peak frequency	f = 1/(2πτ)	`standard_eis`
Re Z at 1 kHz	linear in ln f between the two kept points around 1 kHz	`gold.re_at`
Cell aggregate	Y(f) = Σ A_s/Z_s(f); Z_ASR = A_used/Y (covered) or A_cell/Y (full plate)	`silver.cell_aggregate_full`
Gamry ASR	Z_Gamry[Ω] · 304.92 cm²	`gamry_compare.compare`
Plate current	I = mean(j over the 36 tiles) · 304.92 cm²; DC closure I = Σ j·A · A_cell/A_meas	`plausibility.current_closure`, `silver.dc_closure`
Tile statistics	mean, median, sd (ddof = 1), CV = sd/mean over measured segments 1–36	`plate_stats.tile_stats`
lin-KK residual	max over f of |Z − Z_KK|/|Z|	`eis_validation.lin_kk`
Cathode λ	λ = flow / (I/(4F)/x_O₂ · 22.414 · 60)	`plate_conditions.cathode_stoichiometry`

Part 4 — Every script

16. Script-by-script guide

Each entry says **what the file is for**, **what it reads and writes**, its **key functions**, and **when you would touch it**. The complete list of every class and function, with dependencies, is in the generated companion `TRAINING_GUIDE_FILE_INDEX.md`.

How to run a module from the command line: modules with a `main(argv)` go through the launcher, `python run.py <module> [args]` (`main`, `frequency_response`, `plausibility`, `dc_closure`, `hf_response`). The readers' own command lines (`gamry_dta.py`, `abgleich.py`, `csv_source.py`, `r2d2_geometry.py`, `utils.py`) are run as files; set the import path first so they find their neighbours:

```
export PYTHONPATH=core:pipeline:checks:analysis:plotting:readers     # from the package root
python readers/abgleich.py EIS_config/gen1/Abgleich --curr-cal EIS_config/gen1/curr.csv --temp-cal EIS_config/gen1/temp.csv
```

16.1 Top level

`01 Local EIS - Process.py` — *notebook, writes.* Setup → widgets → `R.run()` → `R.fit_ecm()` → `R.publish()` (when `PUBLISH_TABLES`) → `R.summary()`. Reads every input of §6; writes run folders and tables. Touch it only to add a cell that calls a new `Session` method.

`02 Local EIS - Results.py` — *notebook, read-only.* `runner.Results` lists saved order folders and published runs; the cells draw frequency response, Nyquist/Bode, median Nyquist, heat maps, MF4 bench log, ECM fits, the runs table and the trends across orders. Writes only the viewer pages under `EIS_Results/<order>/plots/` (scratch if not writable) and `EIS_Results/_overview/trends.html`.

`03 Local EIS - Auto.py` — *notebook/job, writes.* Widgets `dry_run` (`yes` = show the plan only), `max_runs` (default `AUTO_MAX_RUNS = 8`), `orders`. Calls `runner.auto.plan(R)` or `runner.auto.run(R)`. Meant to be scheduled as a Databricks Job (e.g. nightly).

`09 Local EIS - Maintenance.py` — *notebook, destructive.* Removes one `run_key` from every table (typed twice to confirm) and optionally its folder; "Rebuild all tables" (type `REBUILD`) calls `tables.rebuild`, which is **missing** in this package (§21.2).

`eis_paths.py` — puts the package root and `core/ pipeline/ checks/ analysis/ plotting/ readers/` on `sys.path` once, at import time. Imported first by every notebook, `run.py`, `pipeline/main.py` and `tests/conftest.py`. Touch it when you add a new top-level code folder.

`run.py` — command-line launcher: `python run.py <module> [args]` imports the module with `importlib` and calls its `main(argv)`. Only works for modules whose `main` takes `argv` (not `validate_hf`, `eis_local`).

`requirements.txt` — numpy, scipy, matplotlib (+ pandas, plotly for the viewers), unpinned on purpose: Databricks ML runtimes already carry them.

16.2 `runner/` — the notebooks' code

`runner/settings.py` — every fixed setting of the notebooks: Volume paths (`EV_ROOT`, `FAMOS_ROOT`, `GAMRY_ROOT`, `EIS_CONFIG_ROOT`, `CSV_ROOT`, `RESULTS_ROOT`), Gamry search folders, datago tables and switches, the fixed evaluation (band, SNR, `CHANNEL_LAG`, `GAMRY_SYNC`, `GAMRY_MATCH`, `CARD_GAIN`, `SKEW_MODEL`, …), table schema/prefix, publish and results-source switches, the auto job, plot sizes, ECM options, and `RUNNER_VERSION`. **The first file to edit when a path moves or a setting changes** (§17.1).

`runner/datago.py` — reads an order's files from datago with Spark SQL (§6.3): order list, file list and classification, sample values, Gamry sweeps written back as `.dta` (`gamry_dir`), the bench log (`bench_row`, `bench_channels`), FAMOS uploads (`famos_cards`, `famos_key`), rebuilding a card as a FAMOS-v1 `.DAT` (`famos_card`, `write_famos_v1`, `default_slot_order`), finding the original upload (`raw_card`, `raw_file_candidates`, `file_record`) and checking a rebuilt card against the Volume (`compare_with_dat`). Writes to `/tmp/datago_gamry/` and the run's scratch folder. Touch it when datago tables or naming change.

`runner/session.py` — class `Session`, the brain of notebooks 01 and 03.

- *Discovery*: `orders`, `conditions_and_tests`, `plate_version`, `plate_hint`, `famos_dir` (card source per card, §6.2), `gamry_dir`.
- *Widgets*: `widgets()` creates and reads them and resolves every file; `select(**values)` does the same without widgets (jobs, tests).
- *Run*: `_config` (widgets + settings → `Config`), `identity` (settings digest), `run()` (cache/rerun, scratch → saved folder), `_save`, `restore_run`.
- *After*: `fit_ecm`, `publish`, `summary`, `build_hf_response`.
- *Plots*: `show_nyquist`, `show_median_nyquist`, `show_heatmaps`, `show_frequency_response`, `show_mf4`, `show_ecm` — each draws one viewer and saves it under `plots/`.
- `IDENTITY_KEYS` lists the settings that make up the digest. **Add a new result-changing setting here** (§18.12).

`runner/results.py` — class `Results(Session)`, notebook 02. Its widgets list saved order folders (`result_set`), conditions and test; `run()` and `publish()` raise. `run_dirs` restores from the tables when `RESULTS_SOURCE = "tables"` (falls back to folders), `show_runs` lists the runs table, `show_trends` plots plate median ± sd of every gold parameter against current, one curve per order and test.

`runner/tables.py` — the Delta tables (§7.3): `FILES` (table suffix → result file), `run_key` (key columns from a folder path), `run_record` (the runs row), `tidy`, `frames`, `_align` (column types), `_write` (`replaceWhere` on `run_key`), `publish`, `delete_run`, `read`, `runs`. Touch it to add a table: add the file to `FILES`.

runner/auto.py — the incremental job. `plan(R)` lists every (order, condition, test) with FAMOS data whose run is not in the runs table, or (`AUTO_RERUN_CHANGED`) is there with another settings digest; `run(R)` evaluates the first `max_runs` with `rerun`, fits the ECM and publishes. Plate per order from `AUTO_PLATE`, else the order's file names (`Naboo` = gen2, `Kashyyyk` = gen1), else gen1; orientation from `AUTO_ORIENTATION`. A failed run is not published and is retried next time.

runner/views.py — the figures, as pure functions of a run folder (no widgets, no Spark): `nyquist_figure`, `median_figure`, `frequency_response`, `bench_figure_whole`, `channels_from_values`, `ecm_fits`/`fit_standard`/`fit_drt`/`ecm_figure`/`ecm_viewer`/`ecm_png`. Touch it to change a plot.

runner/__init__.py — exports `Session`, `Results`, `condition_key`, `settings`, `tables`, `auto`.

16.3 `core/` — foundations

core/config.py — every tunable number of the science in one frozen dataclass `Config`, default instance `DEFAULT`. Also the physical constants (plate 252 × 121 mm, `A_CELL_CM2 = 304.92`), `FAMOS_PATTERNS`, port layout (`COOLANT_INLET_END`, `PLATE_VIEW_MIRRORED`), `HEATMAP_LIMITS`, `FAULT_RULES`. Methods: `replace(**kw)`, `preset("permissive"|"strict")`, `f_hi(fs)`, `f_eval_max()`, `to_dict`, `save`, `from_json`, `from_env`, `from_cli`. Frozen so a config cannot change in the middle of a run. **Every new number goes here as a field with a comment**, never as a constant inside a module.

core/utils.py — shared toolbox: sine fits (`fit3`, `fit4`, `fit7_joint`), THD, stationarity, Cramér–Rao uncertainty (`crlb_phasor`, `sigma_rel_from_snr`), |Z| outlier detection, delay helpers, geometric-grid fit, complex interpolation, gain files (`read_pairs`, `load_gain`, `gain_at`), console-safe logging (`get_logger`, `banner`, `section`), `write_json`, `write_table`, `segment_areas(cfg)`. Anything used by more than one layer lives here.

core/r2d2_geometry.py — geometry of both plates: which of the 900 pads belong to which of the 72 segments (`gen1` green Kashyyyk, verified; `gen2` blue Naboo, reconstructed), areas, centroids, label positions, temperature interpolation, orientation (`standard`/`rot180`). `use_plate(plate, orientation)` switches the active plate **for the whole process** (module-level state `SEGMENTS`). Also `self_check`, `symmetry_report`, `plot_map`, `write_csv`. Touch it only with the coordinate drawing in hand.

core/plate_stats.py — mean, median, sd, CV, min, max over the 36 equal 5 × 5-pad tiles (segment k = tile k): `tile_of`, `tile_values`, `tile_stats`, `summary_line`, `statistics_rows`, `write_statistics` (`plate_statistics.csv`).

core/plate_conditions.py — turns port measurements into per-segment fields with a provenance label: temperature (measured by 4 sensors), pressure (interpolated inlet → outlet), relative humidity (modelled water balance), cathode stoichiometry. Used by `main`.

core/site_paths.py — finds the plate calibration files for any user (§6.5): `resolve`, `config_root`, `plate_dirs`, `matches_plate`, `describe`, `missing_message`.

16.4 `pipeline/` — the processing

pipeline/main.py — the orchestrator and command line. `run_pipeline(cfg, stop_after)` selects the plate, then bronze → silver → gold → `whole_cell_and_plausibility` and writes `run_manifest.json`; `reevaluate(run_dir, out_dir, **overrides)` re-runs silver → gold → checks from saved bronze CSVs; `self_test()`; `main(argv)` with ~40 flags (`--dat`, `--curr-cal`, `--temp-cal`, `--gain`, `--gamry`, `--leepa`, `--condition`, `--plate`, `--source`, `--csv`, `--reevaluate`, `--self-test`, `--print-config`, `--stop-after`, `--channel-lag`, `--card-gain`, …).

pipeline/bronze.py — layer 1 (§9). Dataclasses `ChannelInfo`, `BronzeSpectrum`, `CardInfo`, `BronzeRun`; functions `discover_files`, `inventory_channels`, `estimate_card_lags` (+ alignment diagnostics), `plate_temperatures`, `consensus_schedule`, `refine_frequencies`, `process_card`, `run`, `save`, `load`.

pipeline/eis_local.py — the FAMOS readers (`FamosV1`, `FamosV2`, factory `FamosFile`, `famos_start_time`, `channel_integrity`), the calibration (`PlateCalibration`), blind schedule detection (`detect_schedule`, `demod_envelope`, `dwell_window`, `fill_gaps`, `Step`), basic sine fits, and an older stand-alone evaluator (`evaluate`, `main`). Touch it when a new FAMOS dialect appears.

pipeline/tone_estimation.py — one exact frequency per card and step by multichannel maximum likelihood (variable projection), and `phasors_at` (all channels' phasors at that frequency).

pipeline/ladder_snap.py — snaps a recovered schedule onto its own geometric ladder f_k = 10^((k+offset)/ppd) and repairs windows that break the time order of a monotonic sweep.

pipeline/hf_schedule.py — recovers the top decade of the sweep: detection on the stacked segment channels (whose tone amplitude is flat in f), ladder extension with predict-and-verify, and the array tests used to verify any relocated window (`cfar_channel_count`, rank-1).

pipeline/gamry_sync.py — puts the FAMOS record on the Gamry's clock (§9.2): `read_timeline`, `align`, `apply_frequencies`, `relocate`, `confine_windows`, `corroborate_card_lags`, `recording_window`.

pipeline/silver.py — layer 2 (§10). `SkewModel`, `SilverSpectrum`, `SilverRun`, `FilledSpectrum`; skew model (`choose_global_basis`, `fit_structural_skew`), DRT (`bayesian_drt`), R_ohmic estimators, `gate_points`, `process_segment`, card phase (`card_hf_phase`, `gamry_card_phase`), `dc_closure`, `cell_aggregate_full`, `build_filled_spectra`, `dc_share_faults`, `run`, `save`. The biggest file (2,728 lines); every correction and judgement lives here, separate from raw data.

pipeline/channel_lag.py — per-segment chain lag against the plate median (`estimate`, `ChannelLag`, `correction`); statuses `ok`, `not_first_order`, `too_large`, ….

pipeline/eis_measurement_model.py — Boukamp/Agarwal measurement model (Voigt elements with fixed τ): KK-compliant filtering and R_0 without extrapolation by eye; used by silver.

`pipeline/eis_validation.py` — instrument-independent quality tests: `lin_kk` (Schönleber μ criterion) and `z_hit`; delay estimate from Z-HIT.

`pipeline/standard_eis.py` — the textbook circuit fit: `hf_intercept` and `fit_spectrum` (R_ohmic + L + two ZARCs, CNLS, bounds, multi-start).

`pipeline/neighbours.py` — adjacency from shared pad edges, neighbour outliers (`scalar_outliers`, `spectrum_outliers`), and `fill_spectrum` (rebuild a segment from its rings).

`pipeline/gold.py` — layer 3 (§11). `SegmentRecord`, `GoldRun`; `split_processes`, `collect_parameters`, `re_at`, `fit_field`, `label_faults`, `plate_heatmap`, `plate_heatmap_interactive`, `run`, `save`.

`pipeline/csv_pipeline.py` — the CSV path (§13): `run_csv`, `apply_scan_skew`, phasor fits on bursts, the CSV ECM, gold-compatible outputs.

16.5 `checks/` — diagnostics

`checks/plausibility.py` — the plate-level verdicts (§12.3): `Check`, `Report`, the individual checks, `check_run` (from a `SilverRun`), `check_from_disk` (from a folder), `save` → `plausibility.csv`; CLI `python run.py plausibility RUN_DIR`.

`checks/gamry_compare.py` — the whole-cell reference against the local aggregate (§12.1), plus Gamry/bench file discovery used everywhere: `find_cell_sweeps`, `select_sweep`, `version_of`, `find_bench_log`, `read_bench_log` (`BenchLog.state_at`), `is_sweep_mf4`, `mf4_sweep_as_dta`, `compare`, `run`, `plot`.

`checks/card_gain.py` — does every card measure the same cell voltage with the same gain? `measure_reference` (bronze, from the shared `UC2`), `impedance_factors` (silver, per card in four bands), `uc_gain_pct`.

`checks/frequency_response.py` — ex-situ and in-situ chain response (§12.2); `python run.py frequency_response RUN_DIR --bode ABGLEICH/bode`.

`checks/hf_response.py` — the frozen per-segment/card response above 1 kHz across conditions, validated leave-one-current-out; `build(runs, out_dir, mag)`; applied by silver through `cfg.hf_response_file`.

`checks/dc_closure.py` — multi-run: plate current vs bench current over conditions, `I_meas = a·I_bench + b`, and UC2 vs Gamry Vdc (series resistance in the sense path); `python run.py dc_closure RUN… -o OUT`.

`checks/validate_hf.py` — synthetic self-validation of the high-frequency chain with the acquisition skew present; part of `--self-test`.

16.6 `analysis/`

`analysis/ecm_drt.py` — the DRT (Tikhonov, non-negative) and the DRT-informed equivalent-circuit fit ("method B"): arcs sorted by τ (`fast`/`slow`), one or two arcs by AICc, CRLB weighting, optional TLM fast element, mains harmonics skipped. Used by `runner/views.py`.

16.7 `plotting/`

`plotting/plate_style.py` — one look for every plate map: plotly's Jet scale, text sizes, view direction, port positions (AIR OUT / COOLANT / H_2 IN at x = 0; H_2 OUT / COOLANT / AIR IN at x = 252 mm), orientation. Touch it to change how *every* map looks.

`plotting/plate_figure.py` — static plate heat map in the style of the bench viewer (frame, gasket, ports, sensors, true staircase outlines), fixed colour limits from `config.HEATMAP_LIMITS`; draws gold's `plate_*_interp.png`.

`plotting/plate_plotly.py` — the interactive plate map (45 × 20 pad heat map, value in each segment, hover, parameter dropdown).

`plotting/plate_maps.py` — static maps with number and value written in each segment and a robust (5–95 %) colour range.

`plotting/nyquist.py` — the per-segment Nyquist/Bode figure, coloured by position along the air path, with the Gamry sweep drawn as recorded (`gamry_frame`, `comparison_figure`).

`plotting/viewers.py` — one plot on screen chosen from drop-downs: `selector_html`, `by_condition`, `plate_maps`, `panels`, `img_html`, `save_page` (a page that also works as a saved file).

`plotting/figure_panels.py` — splits a multi-panel Plotly figure into full-width figures, one per panel, keeping legends and dropdowns.

`plotting/bench_plots.py` — the MF4 bench parameters as one interactive figure (parameter dropdown, toggling legend, unified hover, range slider); `read_mf4` (needs `asammdf`), `bench_figure`, `mf4_start_time`, `CHANNEL_GROUPS`.

16.8 `readers/`

`readers/gamry_dta.py` — reads any Gamry `.DTA` (`read_dta` → `GamrySweep`) and turns the Abgleich `bode/` sweeps into `chain_gain.csv` (`read_bode_folder`, `cross_check_abgleich`, `median_response`, `write_gain_csv`, `chain_tau`). CLI: `python readers/gamry_dta.py ABGLEICH/bode --curr-cal ABGLEICH/coefficients/curr.csv -o chain_gain.csv [--shared]`.

`readers/abgleich.py` — the plate calibration from the raw `Step<k>_<T>Grad.csv` files: `read_step`, `read_campaign`, `fit_coefficients` (rebuild `curr.csv`/`temp.csv`), `verify` (does a delivered pair reproduce the Step data), `implied_area`. CLI shown at the top of §16.

`readers/csv_source.py` — every CSV layout (§6.6): `detect_dialect`, `read` (→ `CsvMeasurement`), R2-D2 logger (`read_r2d2`, `read_r2d2_sweep`), bench-tool spectra (`read_spectra_dir`), campaign discovery (`campaigns`, `condition_map`), non-uniform phasor

fits.

`readers/mdf4_min.py` — a minimal MDF4 reader for the small sorted Gamry `.mf4` exports, so they work without `asammdf`.

`readers/_vendor/zstd.py` — a stand-in for the PyPI package `zstd`, which `asammdf` imports unconditionally and which has no Windows wheels for Python ≥ 3.11; maps to `zstandard`.

16.9 `EIS_config/`, `docs/`, `tests/`

`EIS_config/gen1/`, `EIS_config/gen2/` — `curr.csv`, `temp.csv`, `chain_gain.csv`, `Abgleich/` (Step files, `coefficients/`); gen2 also has a `README.md` on the Naboo delivery. Upload to the Volume once.

`docs/` — `PIPELINE_GUIDE.md` (reference manual), `GEN2_PLATE_AND_CSV_PIPELINE.md`, `csv_vs_famos.md`, `famos_vs_csv_analysis.md`, `spectra_recheck.md`, figures, this guide, its file index and `docs/tools/make_file_index.py` (regenerates the index).

`tests/` — `conftest.py` (imports `eis_paths`), `make_synth_famos.py` (synthetic FAMOS cards with known impedance, multiplexer skew and an open HF arc), `make_synth_sweep.py` (a descending stepped sweep on several cards armed at different moments, plus the matching Gamry `.DTA` and calibration files), `fixtures/r2d2_sample/` (a real R2-D2 logger excerpt), `data/*.mf4` (a real Gamry `.mf4` export), and 56 test modules (§19).

Part 5 — Working on it

17. Settings: what to change, and where

There are exactly two places for settings. Never put a number in a notebook or as a constant inside a module.

You want to change…	Edit	Notes
A Volume path, a datago table, a source switch, the fixed evaluation of the notebooks, table names, the auto job, plot sizes	`runner/settings.py`	Read by the notebooks only
A number of the science (gates, thresholds, model choices, band limits, colour limits, fault rules)	`core/config.py` (a `Config` field or a module constant)	Read by every stage, the CLI and the tests
A setting for one command-line run	a flag of `pipeline/main.py`, or `--config run.json` (any `Config` field)	Flags win over the JSON file. (`Config.from_cli` also reads `EIS_<FIELD>` environment variables, but `main.py` does not use it)
One setting in a notebook session, for an experiment	`R.sel` / `R._config` in an extra cell — and do not save the result over a real one	Prefer a CLI run in a scratch folder

17.1 `runner/settings.py` reference

Group	Setting = value	Meaning
Version	`RUNNER_VERSION = "v35.1"`	Written into every `run_identity.json`; a saved run from another version is announced under `cache`
Paths	`EV_ROOT`, `FAMOS_ROOT = EV_ROOT/Famos`, `GAMRY_ROOT = EV_ROOT/Gamry`, `EIS_CONFIG_ROOT = EV_ROOT/EIS_config`, `CSV_ROOT = EV_ROOT/csv_files`, `RESULTS_ROOT = EV_ROOT/EIS_Results`	Where everything is
Gamry	`GAMRY_SEARCH_ROOT_TEMPLATES`, `GAMRY_VERSION_SOURCES`, `PLATE_VERSION_OVERRIDE = {}`	Where an order's sweeps are looked for; force a build token per order
Datago	`DATAGO_META_TBL`, `DATAGO_FILES_TBL`, `DATAGO_GP_TBL`, `DATAGO_VAL_TBL`, `DATAGO_LOCAL_TZ`, `DATAGO_MIN_SWEEP_POINTS = 10`	§6.3
Sources	`DATAGO_FIRST = True`, `GAMRY_FROM_DATAGO = True`, `BENCH_FROM_DATAGO = True`, `FAMOS_FROM_DATAGO = True`, `FAMOS_VOLUME_FALLBACK = True`, `DATAGO_RAW_FILE_TEMPLATE = ""`	§6.2
Band	`F_MIN_HZ = 0.15`, `F_MAX_HZ = 30000`, `FULL_BAND = True` (False: 0.15 Hz – 1.2 kHz)	Evaluated band
Gates / mode	`MIN_SNR_DB = -10`, `EVALUATION_MODE = "default"` (`permissive`/`strict` add `_<mode>` to the folder)	
Corrections	`CHANNEL_LAG = "correct"`, `GAMRY_SYNC = "guide"`, `GAMRY_MATCH = "file"`, `CARD_GAIN = "correct"`, `UC_SERIES_MOHM_CM2 = 0.0`, `FREQ_RESPONSE = "report"`, `HF_RESPONSE = "auto"`, `HF_RESPONSE_MAG = "segment"`	§9, §10
HF detection	`SKEW_MODEL = "structural"`, `HF_USE_ENSEMBLE = True`, `HF_LADDER_PRUNE = True`, `F_HI_FRAC_FS = 0.49`, `WINDOW_MIN_SNR_DB = -4.0`	Values validated on 2612087; part of the identity
Maps	`HEATMAP_STYLE = "interpolated"`, `CSV_MIRROR_X = False`	
Cache	`RUN_MODES = ("cache", "rerun")`	
Tables	`TABLE_SCHEMA = "ps_xplatform_dev.rvadvtec_dev"`, `TABLE_PREFIX = "local_eis_"`, `PUBLISH_TABLES = True`, `RESULTS_SOURCE = "tables"`, `CACHE_FROM_TABLES = True`	§7.3
Auto job	`AUTO_ORDERS = ()`, `AUTO_MAX_RUNS = 8`, `AUTO_PLATE` (2612087, 2612090 → gen1; 2612091, 2612092 → gen2), `AUTO_ORIENTATION` (2612090 → rot180; the others standard), `AUTO_RERUN_CHANGED = True`	§18.10
Plots	`PANEL_HEIGHT = 560`, `PANEL_WIDTH = 1150`, `NYQUIST_COLOUR = "flow"`, `MF4_INCLUDE_ALL_CHANNELS = True`, `MF4_MAX_POINTS = 4000`	
ECM	`ECM_DRT_F_MAX_HZ = 1000`, `ECM_DRT_FIX_RS = True`, `ECM_DRT_FAST = "auto"`, `ECM_SMOOTH_POINTS = 160`	§14

17.2 `core/config.py`: the fields you will meet most

Group	Field = default	Meaning
Plate	`plate = "gen1"`, `plate_orientation = "standard"`, `equal_areas = False`, `areas_file`	which plate, how mounted
Source	`source_format = "famos"`, `csv_path`, `csv_dialect = "auto"`, `csv_tones`, `csv_scan_rate_hz`	§13

Group	Field = default	Meaning
Inputs	`dat_dir`, `out_dir`, `curr_cal`, `temp_cal`, `gain_file`, `gamry_dir`, `bench_log`, `gamry_version`, `hf_response_file`, `abgleich_bode_dir`	filled by the runner
Identity	`leepa`, `condition`, `test_id = "01"`	
Band	`f_min_hz = 0.15`, `f_max_hz = 30000`, `eval_f_max_hz = 2400`, `full_band = False`, `full_band_frac_fs = 0.49`, `f_hi_frac_fs = 0.45`, `coherent_f_max_frac_fs = 0.16`	runner: full band
Detection	`ppd = 12`, `grid_tol = 0.01`, `min_ref_channels = 2`, `min_snr_db = 10`, `snr_floor_db = -3`, `hf_use_ensemble`, `hf_ladder_prune`, `ladder_snap = True`	
Alignment	`ref_channel = "UC2"`, `align_cards = True`, `align_min_corr = 0.5`, `align_min_prominence = 15`, `align_max_lag_s = 30`, `align_f_lo_hz = 0.5`, `align_f_hi_hz = 300`	
Gamry clock	`gamry_sync = "report"`, `gamry_match = "file"`, `window_confine = True`, `window_min_snr_db = -3`, `window_guard_s = 0.5`, `window_settle_periods = 0.5`	runner: `guide`
Phasors	`phasor_method = "card_ml"`, `freq_refine = False`, `fit_at_exact_frequency = False`	
Gates	`sigma_rel_max = 0.60`, `max_thd = 0.10`, `max_drift = 0.25`, `min_cycles_per_dwell = 3`, `zmag_outlier_mad = 4.5`, `zmag_outlier_win = 7`, `passivity_gate_min_hz = 1.0`, `background_max_pct = 5.0`, `min_points_per_spectrum = 8`	§10.2
Timing	`skew_model = "none"`, `fit_common_delay = False`, `channel_lag = "correct"`, `channel_lag_max_resid_deg = 10`, `channel_lag_max_us = 250`, `card_hf_phase = True`, `gamry_card_phase = True`	runner: structural
Card gain	`card_gain = "correct"`, `card_gain_min_snr_db = 30`, `card_gain_f_lo_hz = 1`, `card_gain_f_hi_hz = 300`, `uc_hf_equalise = True`, `uc_reference_card = "Karte_1"`	
Model	`param_method = "drt"`, `ecm_compare = True`, `drt_method = "gp"`, `drt_n_tau_per_decade = 10`, `r_ohmic_method = "auto"`, `uncertainty_model = "crlb"`, `kk_tol = 0.02`, `mu_crit = 0.85`	§10.3
Gold	`tau_split_ohmic_s = 1e-4`, `tau_split_kinetic_s = 1e-2`, `spatial_enable = True`, `spatial_len_x_mm = 70`, `spatial_len_y_mm = 35`, `infer_missing_segments = True`, `max_inferred_fraction = 0.60`, `heatmap_params`, `heatmap_style`	runner: no inference
Segments	`exclude_segments`, `substitute_segments`, `fill_missing_from_neighbours = False`, `fill_max_hops = 2`, `segment_fault_ratio = 0.5`	widgets
Output	`write_csv`, `write_json`, `write_html`, `write_png` (all True), `verbose = True`	
Module constants	`A_CELL_CM2 = 304.92`, `FAMOS_PATTERNS`, `COOLANT_INLET_END = "xW"`, `PLATE_VIEW_MIRRORED = False`, `HEATMAP_LIMITS`, `FAULT_RULES`	

`python run.py main --print-config` prints the effective configuration of a command-line run.

18. How-to recipes

18.1 Evaluate a new FAMOS order

1. Check the files exist: the order's `.DAT` files in `Famos/` or datago, its Gamry sweeps, and that the plate is known (`Naboo` in the bench file name = gen2).
2. 01 Process: Order, `famos`, Plate, Plate position, Conditions `ALL`, Test, Run mode `cache` → Run all.
3. Read the Selection printout (Gamry folder and build, chain gain, `!!` warnings) and the Evaluate log (card sources, `FAILED` lines).
4. Open `plausibility.csv` of each run (§20), then 02 Results for the plots.

18.2 Re-evaluate after a code or setting change

Run 01 Process with **Run mode = `rerun`** for the affected orders, or let the auto job do it: with `AUTO_RERUN_CHANGED = True`, every published run whose settings digest differs from the current one is re-evaluated (code changes do not change the digest — use `rerun` for those).

18.3 A gen2 (blue Naboo) order

Choose **Plate = gen2**. Its calibration comes from `EIS_config/gen2/`; results go to `EIS_Results/<order>_gen2/`. The run stops with a clear message if `gen2/curr.csv` or `temp.csv` is missing — it never borrows the gen1 files.

18.4 A plate mounted turned 180°

Choose **Plate position = rot180**. Ports stay fixed to the cell; maps, the inlet → outlet Nyquist colours, neighbours and the spatial fill follow where the segments really were. Segment numbers, areas, calibration rows and temperatures are unchanged. Results go to `<order>_rot180/`. For the auto job: `AUTO_ORIENTATION = {"2612095": "rot180"}`. CLI: `--plate-orientation rot180`.

18.5 A broken segment

- **Exclude segments** = `33`: not read at all; the segment stays empty on the maps.

- **Rebuild from neighbours** = `33`: its own data is discarded and the spectrum rebuilt from adjacent segments (class `substituted`).
- A segment whose DC current disagrees with its neighbours by more than 2× is left out automatically (`dc_share_faults`).

18.6 Build or rebuild a plate's `chain_gain.csv`

Automatic: delete (or rename) `EIS_config/<plate>/chain_gain.csv` on the Volume; the next Selection cell rebuilds it from `Abgleich/bode/`. By hand:

```
export PYTHONPATH=core:pipeline:checks:analysis:plotting:readers
python readers/gamry_dta.py /path/Abgleich/bode --curr-cal /path/Abgleich/coefficients/curr.csv -o chain_gain.csv
#   add --shared when the bode index does not match curr.csv (the Naboo delivery)
```

The file's content hash is part of every run's identity, so runs made with the old file are announced under `cache` as made with other settings.

18.7 Check a new Abgleich delivery, or rebuild `curr.csv`/`temp.csv`

```
python readers/abgleich.py /path/Abgleich --curr-cal /path/curr.csv --temp-cal /path/temp.csv   # verify (JSON report)
python readers/abgleich.py /path/Abgleich --refit-to /path/out                                  # rebuild curr.csv + temp.csv from the
Step files into /path/out
```

The report gives linearity, the temperature coefficient (TCR ≈ 0.4 %/K for copper) and the implied area per segment; an outlier segment is named.

18.8 Build the frozen high-frequency response of an order

Needs two or more conditions evaluated on the full band. In 01 Process, select all conditions, run, then add a cell `R.build_hf_response()`. It writes `EIS_Results/<order>/hf_response/hf_response.csv` (the replaced file goes to `history/`) and refuses, writing nothing, for runs whose band ends below 5 kHz. Re-run the conditions with `rerun` to apply it (`HF_RESPONSE = "auto"`).

18.9 Evaluate CSV measurements

Put the campaign under `<Volume>/csv_files/<campaign>/` (one folder per operating point: bench-tool `Spectrum<n>_<T>degC_<I>A/z<n>.csv`, or an R2-D2 sweep folder with `metadata.csv` + `p<n>.csv`). In 01 Process choose **Measurement file format = csv**, the **CSV folder**, the plate, and run. Results go to `EIS_Results/csv_<campaign>[_gen2]/<condition>/`. CLI: `python pipeline/main.py --plate gen2 --source csv --csv /path --curr-cal … --temp-cal … --out out/run`.

18.10 Schedule the automatic job

1. Open `03 Local EIS - Auto`, run with `dry_run = yes`: the plan lists every run key with reason `new` or `settings changed`.
2. Create a Databricks Job on the notebook (e.g. nightly) with parameters `dry_run = no`, `max_runs = 8`, `orders` empty (= `AUTO_ORDERS` or every order).
3. Set `AUTO_PLATE` / `AUTO_ORIENTATION` in `runner/settings.py` for orders whose file names do not say the plate. Each run takes 5–25 min; the rest follow at the next execution.

18.11 Remove a run

`09 Local EIS - Maintenance`: type the run key (`2612090/450A_Test01`) twice; optionally delete the folder. No undo. (The "Rebuild all tables" cell needs `tables.rebuild`, missing in this package — §21.2.)

18.12 Add a setting that changes results

1. Add a field with a comment to `Config` in `core/config.py` (never a module constant).
2. Use it via `cfg.<field>` in the stage.
3. If the notebooks should set it: add it to `runner/settings.py` and pass it in `Session._config`.
4. **Add its name to `IDENTITY_KEYS` in `runner/session.py`**, otherwise `cache` serves results computed without it as if they were current.
5. Add a test that plants the effect in synthetic data and asserts it (§19.2).

18.13 Add a parameter to the plate maps and tables

Compute it per segment in `gold.collect_parameters` (with its sd), add its name to `Config.heatmap_params`, give it fixed colour limits in `config.HEATMAP_LIMITS`. It then appears in `plate_summary.csv`, `plate_statistics.csv`, the static and interactive maps, the 02 heat-map viewer and the `gold_segments`/`gold_plate` tables automatically.

18.14 Add a plausibility check

Write a function in `checks/plausibility.py` that returns a `Check` (`name`, `verdict` = `PASS/WARN/FAIL/NA`, `detail`, `value`, and `rests_on` — what would have to be true for the verdict to be wrong); call it in `check_run` and `check_from_disk`, wrapped so that it can never stop a run. Add a test that plants the fault and asserts the verdict.

18.15 Re-evaluate an old run without its `.DAT` files

```
python run.py main --reevaluate EIS_Results/2612030/45A_Test01 --out out/45A_re [--channel-lag report] [--gain g.csv]
```

`bronze.load()` rebuilds bronze from `bronze/*.csv`; silver, gold, the Gamry comparison and plausibility run with the current code. Name the output folder after the condition so the comparison finds its sweep. Do not pass a gain file to a run whose bronze already applied one. The Gamry clock (`gamry_sync`) cannot be applied this way.

18.16 Query results across orders

Use SQL on the tables (§7.3), or `R.show_runs()` / `R.show_trends()` in 02 Results. Every table has `run_key`, `order_id`, `plate`, `orientation`, `condition`, `current_a`, `test_id`, `eval_mode`.

18.17 Move to another site or workspace

Change the paths and table names in `runner/settings.py` (`EV_ROOT` and the derived roots, `DATAGO_*_TBL`, `TABLE_SCHEMA`). `core/site_paths.py` has its own `EV_ROOT` default for command-line runs; override it with `EIS_CONFIG_ROOT`.

18.18 Ship a new version to Databricks

```
git archive --format=zip --prefix=local_eis/ HEAD:local_EIS_TOM_rebuild > Local_EIS.zip
```

Import the zip into an **empty** workspace folder, bump `RUNNER_VERSION`, and run the affected orders with `rerun`.

19. Development workflow and tests

19.1 The loop

```
git checkout EIS-final && git pull
cd local_EIS_TOM_rebuild
# … change the code …
python run.py main --self-test               # 1 min, no data
python -m pytest tests -q                    # 2 min, 520 cases
python -m pytest tests/test_channel_lag_stage.py -q   # or just the area you touched
python docs/tools/make_file_index.py         # refresh the generated file index
git add -A && git commit -m "…" && git push
```

19.2 Rules for changing the code

- **A new tunable number** goes into `core/config.py` as a `Config` field with a comment — never as a constant in a module.
- **A new result-changing setting** goes into `IDENTITY_KEYS` (`runner/session.py`).
- **Areas** always through `utils.segment_areas(cfg)`; geometry always through `geom.SEGMENTS` after `geom.use_plate()`.
- **A dropped point gets a ledger reason** (the `gate()` pattern in `silver.gate_points`).
- **A new check must not be able to stop a run**: wrap it and report a verdict.
- **Bronze stays raw**: corrections belong in silver, so they can be switched off and compared.
- **Respect the layering** (§3.4): `core` never imports `pipeline`; `plotting` only reads results.
- **Every fix gets a test** in `tests/` that plants the defect in synthetic data and asserts it is caught.
- **After a change, run the affected orders with `rerun`** — the cache does not see code changes.

19.3 How the tests work

`tests/conftest.py` imports `eis_paths`, so every test imports modules by plain name. The tests use pytest fixtures (`tmp_path`, `monkeypatch`), parametrisation, a fake Spark session (`test_tables_results_auto.py`), and synthetic data with known truth:

- `tests/make_synth_famos.py` writes FAMOS cards with a known impedance, a per-channel multiplexer skew and an open high-frequency arc;
- `tests/make_synth_sweep.py` writes a descending stepped sweep on several cards, each armed at a different moment and each channel one ADC slot later, plus the matching Gamry `.DTA` and calibration files — used for end-to-end runs including the Gamry synchronisation;
- `tests/fixtures/r2d2_sample/` is a real R2-D2 logger excerpt; `tests/data/*.mf4` a real Gamry `.mf4` export.

Area	Test modules (examples)
Card alignment and clocks	`test_card_alignment`, `test_card_alignment_corroboration`, `test_alignment_diagnostics`, `test_prominence_guard`, `test_sync_timestamps`
Step detection and frequencies	`test_hf_schedule`, `test_frequency_refine`, `test_tone_estimation`, `test_window_confine`, `test_coverage_and_provenance`, `test_gamry_sync`
FAMOS and datago	`test_famos_dialects`, `test_file_discovery`, `test_channel_integrity`, `test_reference_channel`, `test_runner_datago`, `test_runner_datago_famos`, `test_datago_original_card`
Corrections and gates	`test_card_gain`, `test_channel_lag_stage`, `test_hf_response`, `test_rejection_ledger`, `test_segment_exclusion`, `test_audit_corrections`, `test_full_band`
Models and validation	`test_standard_eis`, `test_ecm_drt`, `test_drt_nonnegativity`, `test_hfr_estimators`
Plate products	`test_aggregate_and_fill`, `test_plate_stats`, `test_plausibility`, `test_gold_fill_and_silver_files`, `test_site_paths_and_plate`, `test_plate_orientation`
Gamry and CSV	`test_gamry_compare`, `test_gamry_selection`, `test_gamry_sweep_choice`, `test_gamry_match_by_file`, `test_gamry_by_bench_name`, `test_csv_pipeline`, `test_spectra_dir`
Plots and runner	`test_plate_figure`, `test_plate_maps`, `test_plate_style`, `test_viewers`, `test_figure_panels`, `test_bench_plots`, `test_runner_session`, `test_tables_results_auto`, `test_tables_delta`, `test_tables_spark_types`

The full list with the number of tests and the first docstring line of each module is in the file index.

20. Reading a finished result

Open, in this order:

1. **`plausibility.csv`** — any `fail`? Each row has the check, verdict, value and a detail sentence.
2. **`gold/plate_statistics.csv`** — plate median and CV per parameter, and how many of 36 tiles were measured.
3. **`silver/segments_summary.csv`** — tiers (A/B/C) and flags per segment.
4. **`silver/segment_reach.csv`** — why a spectrum stops where it does (`blocked_by` names the gate).
5. **`bronze/gamry_sync.csv`** — were the windows where the Gamry says (`ok` / `misplaced` / `missing`)?
6. **`bronze/card_alignment.csv`** — was every card aligned (`applied`), with what prominence?
7. **`gamry_comparison.csv`** — aggregate vs Gamry: magnitude ratio, phase difference, HFR of each.

Typical values on 2612030 for orientation: segment R_ohmic 44–72 mΩ·cm² (plate CV ~6 % after the channel-lag correction), Gamry whole-cell HFR 44–45 mΩ·cm², charge-transfer arc peak 110–170 Hz, mass-transport arc 3–10 Hz at 450 A. The aggregate reads 7–18 % above the Gamry in |Z| with phase within ±3° up to 1 kHz (§21.3).

Verdict	What to do
`card alignment: fail`	A card's segments sit on the wrong windows. Check `bronze/card_alignment.csv`; the Gamry clock may corroborate the lag (`gamry_card_sync.csv`)
`channel lag: warn/fail`	Segments with `not_first_order`/`too_large` in `silver/channel_lag.csv` are tier C; look for a wiring or card problem
`current closure: fail`	Calibration (`curr.csv` K), area or missing segments; run `dc_closure` across conditions to separate scale from offset
`coverage: warn/fail`	Too little area measured; statistics are less representative
`aggregate vs whole cell: fail`	Calibration, plate choice or chain response; compare at low frequency (scale) and high frequency (timing)

21. Troubleshooting and known issues

21.1 Test results on this package

Run on the package exactly as delivered (Python 3.13, numpy, scipy, matplotlib, pandas, plotly, pyarrow installed): **505 passed, 3 skipped, 12 failed.**

Failing test(s)	Cause	Severity
`test_card_alignment.py::test_noise_alone_produces_no_prominent_peak`	Documented, intentional: pure noise reaches prominence 8.5 against the gate of 15, the test asks for a 2× margin and gets 1.8×. Raising the gate could hide real card offsets	None — leave it
9 tests in `test_tables_results_auto.py`	`runner/tables.py` is older than the code and tests around it (§21.2)	Real: affects table restore and rebuild
`test_plate_orientation.py::test_gen2_naboo_calibration_ships_with_the_package`	`EIS_config/gen2/Abgleich/bode/` is not in the package (`DATA_NOT_IN_GIT.md`)	Only matters to rebuild gen2 `chain_gain.csv`

Failing test(s)	Cause	Severity
`test_runner_session.py::test_the_notebooks_are_only_calls_with_a_banner`	The banner of `01 Local EIS - Process` reads "writes results and tables" without the "🛠 Data engineering —" heading the test expects	Cosmetic

Without plotly, pandas or pyarrow installed, 17 further tests fail with `ModuleNotFoundError` — install them before judging a test run.

21.2 The `runner/tables.py` gap

The settings (`RESULTS_SOURCE`, `CACHE_FROM_TABLES`), the notebooks (01, 02, 09), `runner/session.py`, `runner/results.py`, the docs and the tests all expect a `local_eis_run_files` table and three functions in `runner/tables.py`: `restore()`, `rebuild()` and `file_bytes()`. The `tables.py` shipped in this package has none of them (it also widens column types instead of refusing a clash). Effects:

Feature	Behaviour now
01 Publish (`R.publish`)	Works: the 13 result tables + runs table are written; `run_files` is not
01 `cache` when a folder is gone but the run is published	`restore_run` prints "not restored from the tables (AttributeError…)" and the run is **recomputed**
02 Results with `RESULTS_SOURCE = "tables"`	Restore fails, falls back to the saved folder; a run whose folder is gone cannot be shown
09 Maintenance "Rebuild all tables"	Crashes with `AttributeError: module 'runner.tables' has no attribute 'rebuild'`
09 Maintenance "remove a run"	Works

Until `tables.py` is brought up to date, keep the run folders under `EIS_Results/` — they are the only complete copy — and consider `RESULTS_SOURCE = "folders"`.

21.3 Measurement limitations (from the package's own analyses)

1. **The top of the band.** Above ~1 kHz each card's phase is anchored to the Gamry (`gamry_card_phase`), so the HF crossing (R_ohmic) is not independent of the Gamry there until an injected per-channel calibration sweep exists.
2. **The aggregate is 7–18 % above the Gamry in |Z|** (2612030: 45 A +8 %, 450 A +18 %), phase within ±3° up to 1 kHz. About 13 mΩ·cm² is a series resistance in the UC sense path (~44 µΩ, `dc_closure`); `UC_SERIES_MOHM_CM2` stays 0 because subtracting it breaks the HF crossing of many segments. The rest is open — a simultaneous dummy-load sweep through both instruments settles it.
3. **Low-frequency scatter at the air outlet at high current** (150/450 A, below ~20 Hz, up to ~10 %) is in the measurement (the cell changes during the long steps); quote R_mt there with the larger uncertainty.
4. **Missing steps** (2.4 Hz, 9.45 Hz are sometimes weak) are left out, not guessed.
5. **Datago FAMOS-v2 cards** hold `UC2` under every channel name; such cards are read from the Volume. The datago slot order is approximate when no Volume `.DAT` of the card exists.
6. **Tile statistics** treat segments 1–36 as their whole 5 × 5 tile even where the gen1 segment is smaller.
7. **Units**: `R_hf_extra` in `plate_summary.csv` is in Ω·cm²; every other resistance there in mΩ·cm².

21.4 Documentation that is out of step with the code

Where	Says	The code
`README.md` "final version"	FAMOS/Gamry/bench from datago default `False`	`runner/settings.py`: all `True`
`README.md`	R_ct/R_mt/R_pol "the standard way … no DRT"	`param_method = "drt"` with the circuit beside it (`ecm_compare`)
`docs/PIPELINE_GUIDE.md` §2	Run modes `cache-only`, widget "Write results to cache"	`RUN_MODES = ("cache", "rerun")`, no such widget
`docs/PIPELINE_GUIDE.md` §5, §17	tree under `databricks/local_eis/`; `_BUILD_MARKERS`, `_CACHE_IDENTITY_KEYS` in the notebook	package root is `local_EIS_TOM_rebuild/`; the identity keys are `runner/session.py: IDENTITY_KEYS`; there is no build-marker check
`docs/README.md`	links `FIXES.md`, `README_2612025.md`, `EIS_PIPELINE_DEVELOPMENT_PLAN.md`, `test_plans/`	not in the package
`README.md`	`python diagnose_channel_lag.py …`	no such script in the package
`run.py` docstring style	`python run.py <module>` for any module	only modules with `main(argv)`; `validate_hf` and `eis_local` have `main()` without arguments

21.5 Symptom → cause → fix

Symptom	Cause and fix
`ModuleNotFoundError: No module named 'config'` (or `silver`, `eis_local` …)	`eis_paths` not imported. Notebooks: run the Setup cell. Scripts: `python run.py …` or `export PYTHONPATH=core:pipeline:checks:analysis:plotting:readers`
`plate gen2: curr.csv, temp.csv not found`	Upload `EIS_config/gen2/` to the Volume (`SETUP_SHARED_FILES.md`)
Selection prints `!! the files of RO… name plate gen2 … but gen1 is selected`	Wrong Plate widget. Choosing wrong does not fail — it draws on the wrong squares
A condition shows `skip no Test_02 files`	That test does not exist for the condition; choose another Test
`no files:` in the Evaluate cell	No `.DAT` matched `FAMOS_PATTERNS` in `Famos/` (or datago). Check the names
`Karte_n: datago REJECTED -- …`	Usually a FAMOS-v2 card; it is read from the Volume. If "left out", put the `.DAT` named in the message under `Famos/` and rerun
`Gamry sweeps NOT from datago: …`	Datago switched off, no Spark, or no sweep there; the Volume folder is used
No Gamry comparison	No sweep for the condition, or the build token does not match the order's; set `PLATE_VERSION_OVERRIDE`
`card alignment: fail`	§20
A spectrum stops early	`silver/segment_reach.csv`, column `blocked_by`
`<-- made with OTHER settings (…)`	The saved result predates a settings change; run with `rerun`
Results did not change after a code update	The cache key does not include the code; run with `rerun`
Tables refuse a write / type error	A column changed type between runs; see §7.3 and the `DROP TABLE` / Maintenance route
`AttributeError: … tables … rebuild/restore`	§21.2
MF4 cell: `no bench .mf4` / asammdf error	Install `asammdf` (+ `zstandard`), or check `Gamry/*RO<order>*.mf4` exists

22. Training plan and self-check

Five sessions of about two hours. Each has a goal, what to read, what to do, and questions to answer without looking; answers are at the end.

Session 1 — The picture

- **Read** §1–§3, `README.md`, `SETUP_SHARED_FILES.md`.
- **Do** Import the package into an empty workspace folder. Open the four notebooks and say for each whether it writes. Open `runner/settings.py` and find where the Volume root and the results root are set.
- **Questions**
  1. Write the formula for Z_seg and name every symbol.
  2. Which notebook would you give to a colleague who must not change anything?
  3. Why is choosing the wrong plate dangerous even though nothing fails?

Session 2 — Data in and out

- **Read** §6–§8.
- **Do** Evaluate one condition of a known order in 01 Process with `cache`, then the same with `rerun`. Find the run folder on the Volume, open `run_identity.json` and `config_used.json`. Find the run in `local_eis_runs` with SQL.
- **Questions** 4. In which order does the pipeline try to get a FAMOS card when `FAMOS_FROM_DATAGO = True`? 5. Where does a gen2 run that was mounted rotated 180° save its results? 6. What goes into the settings digest, and what does not?

Session 3 — The science

- **Read** §9–§15, then `docs/PIPELINE_GUIDE.md` §4.
- **Do** For one run, open `bronze/schedule.csv`, `bronze/gamry_sync.csv`, `silver/point_rejections.csv` and `silver/segment_reach.csv`. Find the segment whose spectrum stops lowest and name the gate.
- **Questions** 7. Why does bronze not de-skew? 8. Which gate "really decides", and what does it measure? 9. What can the channel-lag stage not see?

Session 4 — The code

- **Read** §16 with the code open, and the file index.
- **Do** On a laptop: install, `python run.py main --self-test`, `python -m pytest tests -q`. Generate a synthetic plate with `tests/make_synth_sweep.py` and run `pipeline/main.py` on it. Run `python run.py main --reevaluate` on a saved run folder.
- **Questions** 10. Which function selects the plate, and why must it run first? 11. Where would you add a new number that changes a gate? 12. Which module would you change to alter how every plate map looks?

Session 5 — Making a change safely

- **Read** §17–§21.
- **Do** On a branch: add a setting (e.g. a new threshold) following §18.12 end to end, with a test. Run the suite. Regenerate the file index. Open a pull request.
- **Questions** 13. You changed `silver.py` and ran 01 Process with `cache`. Did the results change? Why? 14. What two things must a new plausibility check never do? 15. Which features are affected by the `tables.py` gap?

Answers

1. Z = K(T)·A_U/A_seg: K from `curr.csv` at the segment temperature (V per A/cm²), A_U the cell-voltage phasor, A_seg the segment current-channel phasor, both at the excitation frequency; result Ω·cm².
2. `02 Local EIS - Results` (read-only; `run()` refuses).
3. gen1 and gen2 group the edge pads differently; the run completes but attributes each channel's current to the wrong piece of plate, mis-weighting every area sum and drawing values on the wrong squares.
4. datago original upload → datago rows rebuilt and checked → Volume `.DAT` (per card), else the card is left out.
5. `EIS_Results/<order>_gen2_rot180/<condition>_Test<nn>/`.
6. The values of `IDENTITY_KEYS` plus the content hashes of the chain-gain and HF-response files; not the code version.
7. Bronze stores only measured values so silver can be re-run and its corrections switched off or changed without re-reading the raw data.
8. `uncertainty`: the Cramér–Rao relative standard deviation σ_rel = √(1/(N·γ)) ≤ 0.60.
9. A delay every channel shares — it measures each segment against the plate median.
10. `r2d2_geometry.use_plate()`, called at the top of `run_pipeline` (and in `Session.widgets/run`): every stage reads areas and centroids from the module-level `geom.SEGMENTS`.
11. A `Config` field in `core/config.py` (with a comment), and its name in `IDENTITY_KEYS` if the notebooks set it.
12. `plotting/plate_style.py`.
13. No. `cache` loads the saved result; the digest covers settings, not code. Use `rerun`.
14. Stop the run (it must be wrapped and report a verdict), and silently drop data (a dropped point needs a ledger reason).
15. Restoring runs from the tables (01 `cache` with the folder gone, 02 with `RESULTS_SOURCE = "tables"`), the `run_files` table, and 09's "Rebuild all tables".

23. Glossary

Term	Meaning
Abgleich	the bench calibration delivery: `curr.csv`, `temp.csv`, bode sweeps, temperature Step files
ADC slot	a channel's position in the converter's scan order; multiplexed channels are sampled one after another
Aggregate	the segments in parallel: Z = A_cell / Σ(A_s/Z_s), compared with the Gamry
ASR	area-specific resistance or impedance, Ω·cm² (mΩ·cm² in outputs)
Bronze / silver / gold	the three layers: raw, corrected and modelled, plate field
Card / Karte	one FAMOS acquisition card, 16 channels; five per recording
Condition	operating point named by its current: 45A, 60A, 150A, 450A
CFAR	constant false-alarm-rate detection test (used to verify relocated windows)
CRLB	Cramér–Rao lower bound: the best possible phasor uncertainty for a given SNR and length
Datago	the Bosch Delta tables holding uploaded measurement files and samples
Digest	the 10-character settings hash in `run_identity.json` that the cache compares
DRT	distribution of relaxation times γ(τ)
ECM / ZARC / CPE	equivalent-circuit model; R in parallel with a constant-phase element
FAMOS v1 / v2	the two imc FAMOS file dialects (v2 = DASYLab 14)
gen1 / gen2	green Kashyyyk plate / blue Naboo plate
HFR / R_ohmic	high-frequency (ohmic) resistance, the real-axis intercept
Ledger	`silver/point_rejections.csv`: for each point, kept or which gate removed it
Leepa / order	the test order number (2612090); `R02612090` in datago
lin-KK / Z-HIT	spectrum validity tests (Kramers–Kronig; Hilbert transform)
Phasor	complex amplitude of a sine at a known frequency
Run key	the run folder path under `EIS_Results`, e.g. `2612090/450A_Test01`; the key of every table row
rot180	plate mounted turned 180° in the cell
Test	FAMOS test number: Test_01 up-ramp, Test_02 down-ramp, …

Term	Meaning
Tier A/B/C/D	segment quality grade (D = no data of its own)
Tile	one of the 36 equal 5 × 5-pad blocks used for plate statistics
UC2	the cell-voltage channel used as the reference on every card
Volume	the Unity Catalog Volume `/Volumes/ps_xplatform_dev/rvadvtec_dev/ev_rvadvtec_dev`

Appendix A — File reference (generated)

Generated by `docs/tools/make_file_index.py` from the source with Python's `ast` module, so it lists every file, class and function that exists in this package and nothing else. The explanations of *why* each file exists are in `TRAINING_GUIDE.md` Part 4; this is the look-up table beside it.

116 Python files (59 in `tests/`), **44,187 lines** including the four notebooks.

For each file: size, the package modules it imports (*depends on*), the modules that import it (*used by*, tests counted separately), third-party libraries, the first paragraph of its docstring and every top-level class and function with the first line of its docstring.

(top level)

Entry points: the four Databricks notebooks, the sys.path helper and the command-line launcher.

`01 Local EIS - Process.py`

75 lines · 0 classes · 0 functions

Depends on	eis_paths, runner
Used by	—
Third-party	—
Standard library	pathlib, sys

`02 Local EIS - Results.py`

97 lines · 0 classes · 0 functions

Depends on	eis_paths, runner
Used by	—
Third-party	—
Standard library	pathlib, sys

`03 Local EIS - Auto.py`

60 lines · 0 classes · 0 functions

Depends on	eis_paths, runner
Used by	—
Third-party	—
Standard library	pathlib, sys

`09 Local EIS - Maintenance.py`

77 lines · 0 classes · 0 functions

Depends on	eis_paths, runner
Used by	—
Third-party	—
Standard library	pathlib, shutil, sys

`eis_paths.py`

40 lines · 0 classes · 1 functions

Puts the pipeline's folders on sys.path, so every module keeps importing its neighbours by plain name (`import config`, `import silver`, ...) wherever it lives in the tree:

Depends on	only standard / third-party libraries
Used by	01 Local EIS - Process, 02 Local EIS - Results, 03 Local EIS - Auto, 09 Local EIS - Maintenance, main, run (+ 1 test files)
Third-party	—
Standard library	pathlib, sys

Class / function	What it does (first docstring line)
`add(root)`	Insert root and its sub-folders at the front of sys.path (once).

`run.py`

35 lines · 0 classes · 1 functions

run.py -- command-line entry point for any pipeline module.

Depends on	eis_paths
Used by	—
Third-party	—
Standard library	importlib, sys

Class / function	What it does (first docstring line)
`main(argv)`	—

runner/

What the notebooks do, as importable code: settings, datago access, the Session behind 01/03, the read-only Results behind 02, the Delta tables, the automatic job, the figures.

`runner/__init__.py`

22 lines · 0 classes · 0 functions

What the Local EIS notebooks do, as importable code:

Depends on	auto, results, session, settings, tables
Used by	—
Third-party	—
Standard library	—

`runner/auto.py`

132 lines · 0 classes · 6 functions

The incremental job: evaluate what has not been evaluated yet.

Depends on	datago, settings, site_paths, tables
Used by	**init**
Third-party	—
Standard library	pathlib

Class / function	What it does (first docstring line)
`plate_of(R, leepa)`	gen1 / gen2 of an order: AUTO_PLATE, else its file names, else gen1.
`orientation_of(leepa)`	—
`published(spark)`	{run_key: digest} of the runs table ({} without Spark or table).
`_select(R, leepa, test_id, conditions, run_mode)`	—
`plan(R, orders, say)`	What the job would evaluate, in order (oldest order first).
`run(R, max_runs, orders)`	Evaluate the first `max_runs` (AUTO_MAX_RUNS) of the plan, fit the

`runner/datago.py`

694 lines · 1 classes · 26 functions

Reading an order's Gamry sweeps and bench log from datago (Spark SQL).

Depends on	eis_local, settings
Used by	auto, session
Third-party	numpy, pandas, pyarrow
Standard library	contextlib, datetime, io, pathlib, re, shutil, tempfile

Class / function	What it does (first docstring line)
`current_label(token)`	'450' -> '450A', '7,5' -> '7.5A'.
`classify(name, mtype)`	(kind, condition) of one datago file from its uploaded name.
`orders(spark)`	Every order with a Gamry sweep in datago, as '2612086'.
`files(spark, leepa, refresh)`	Every datago file of the order, one row per file, classified.
`values(spark, file_ids, channels)`	{file_id: {channel: (t_s, value)}} in the order the samples were
`bench_row(spark, leepa)`	The bench log of the order (the longest .mf4 that is not an
`_starttime(row)`	(STARTTIME text, Time offset of the first point in s).
`_table(row, vals)`	—
`_write_dta(path, row, tab)`	—
`gamry_dir(spark, leepa, volume_root, say)`	A folder with EVERY Gamry sweep of the order from datago, as .dta, or
`is_datago_dir(p)`	—
`bench_channels(spark, leepa)`	(start time UTC, {channel: (t_s, v)}, file name) of the bench log, or
`famos_key(name)`	(condition, test, card) of a card upload, e.g.
`famos_cards(spark, leepa)`	{(condition, test): {card: row}} of the order's FAMOS uploads; when a
`famos_unparsed(spark, leepa)`	Names of the order's .DAT uploads that famos_key cannot read.
`card_channels(spark, file_id)`	({channel: samples}, fs, info) of one card, each channel in time order.
`default_slot_order(names)`	UC channels, segments by number, temperatures, the rest -- only when
`write_famos_v1(path, names, cols, fs, start)`	A FAMOS-v1 .DAT that eis_local.FamosV1 reads: float32 samples,
`_local_start(row)`	t_begin (UTC) as the naive local time a FAMOS |NT holds.
`compare_with_dat(fam_dg, dat, n_probe, probe_len)`	'' when the rebuilt card holds the samples of the .DAT `dat` under the
`famos_card(spark, row, out_path, slot_dat, same_dat, say)`	Rebuild one card from datago at `out_path` and check it.
`file_record(spark, file_id)`	Every column of the datago file record (metafiles_delta) of one file.
`class _Blank`	—
`_text(v)`	—
`raw_file_candidates(rec, order)`	[(where it came from, path or URI)] the original upload may be at.
`_fetch(spark, uri, dest)`	Copy `uri` to `dest`: a local / Volume / DBFS-fuse path directly, any
`raw_card(spark, row, out_path, order, say)`	(path, note) of the ORIGINAL upload of this card, copied to

`runner/results.py`

257 lines · 1 classes · 0 functions

The read-only side: the saved runs and the tables, never an evaluation.

Depends on	r2d2_geometry, session, settings, site_paths, tables, viewers
Used by	**init**
Third-party	pandas, plotly
Standard library	json, pathlib, re

Class / function	What it does (first docstring line)
`class Results`	Saved runs and tables only. Evaluate in "01 Local EIS - Process". Methods: _from_tables, _cond_dirs, _table_keys, _saved_keys, result_sets, _config_used, run_dirs, widgets, run, publish, _plate_stats, show_runs, show_trends.

`runner/session.py`

1,244 lines · 1 classes · 1 functions

Everything the runner notebook does, behind a handful of calls:

Depends on	bench_plots, bronze, config, csv_source, datago, eis_local, gamry_compare, gamry_dta, gamry_sync, hf_response, main, r2d2_geometry, settings, site_paths, tables, viewers, views
Used by	**init**, results
Third-party	numpy, pandas
Standard library	contextlib, datetime, gc, hashlib, io, json, os, pathlib, re, shutil, tempfile, time

Class / function	What it does (first docstring line)
`condition_key(cond)`	'45A' < '60A' < '150A' < '450A'.
`class Session`	The notebook's state: environment, selection, results. Methods: say, html, _w, _make, _famos_files, orders, _famos_datago_on, _datago_famos_cards, conditions_and_tests, plate_version, plate_hint, _volume_cards, _report_missing_cards, _why_rows_broken, _find_on_volume, _slot_source, famos_dir, _has_sweeps, _volume_gamry_root, gamry_dir, widgets, _chain_gain, _print_selection, order_dir, cond_name, saved_dir, plots_dir, _usable, _config, _hf_response_file, identity, run, select, fit_ecm, publish, _published, run_key_of, restore_run, summary, _identity_note, _save, run_dirs, label, _show, _by_condition, show_nyquist, show_median_nyquist, show_heatmaps, _copy, show_frequency_response, show_mf4, _bench_mf4, _sweeps_volume, _sweeps_datago, _famos_temps, show_ecm, build_hf_response.

`runner/settings.py`

206 lines · 0 classes · 0 functions

Every fixed setting of the runner notebook, in one place.

Depends on	only standard / third-party libraries
Used by	**init**, auto, datago, results, session, tables, views
Third-party	—
Standard library	pathlib

`runner/tables.py`

325 lines · 0 classes · 16 functions

The silver / gold layer as Delta tables -- one row set per run.

Depends on	settings
Used by	**init**, auto, results, session
Third-party	numpy, pandas
Standard library	datetime, json, pathlib, re

Class / function	What it does (first docstring line)
`table_name(suffix)`	—
`run_key(run_dir)`	The key columns of the run saved in `run_dir`
`_json(p)`	—
`run_record(run_dir)`	One row of the runs table, from the folder's identity and manifest.
`_clean_name(c)`	—
`tidy(df, key)`	Key columns first, column names Delta accepts, one type per column
`frames(run_dir)`	{table suffix: DataFrame} of one saved run (missing files skipped).
`exists(spark, name)`	—
`_sql_str(v)`	—
`_as_text(s)`	—
`_align(spark, pdf, name)`	Give every column the type it already has in the table.
`_write(spark, pdf, name, key)`	—
`publish(spark, run_dir, say)`	Write one saved run into every table; its earlier rows are replaced.
`delete_run(spark, key, say)`	Remove one run (its run_key) from every table. No undo.
`read(spark, suffix, where)`	A table as pandas ('' filter = all rows); empty when it does not exist.
`runs(spark)`	The runs table, newest first.

`runner/views.py`

605 lines · 0 classes · 19 functions

The plots of the runner notebook, built from a finished run folder. Pure functions of files on disk -- no widgets, no Spark -- so each one can be tested, and called from a script, without Databricks.

Depends on	bench_plots, csv_pipeline, ecm_drt, frequency_response, nyquist, settings, standard_eis, viewers
Used by	session
Third-party	matplotlib, numpy, pandas, plotly
Standard library	json, pathlib, time

Class / function	What it does (first docstring line)
`spectra(run_dir)`	—
`aggregate(run_dir)`	—
`gamry_of_run(run_dir)`	(DataFrame freq_hz/z_re/z_im in mOhm*cm2, sweep name) of the whole-cell
`segment_cards(run_dir)`	{segment: 'Karte_n'} from bronze/segment_meta.csv.
`nyquist_figure(run_dir, title, colour)`	—
`_median_by_freq(d)`	Median of Z' and Z'' over segments at each frequency (frequencies
`median_figure(run_dir, title)`	Nyquist, |Z| and phase: the Gamry sweep (black), the aggregate of the
`frequency_response(run_dir, gain_file, bode_dir, scratch)`	{'exsitu': png or None, 'insitu': png or None, 'checks': [...]}.
`channels_from_values(vals, max_points, include_others)`	bench_plots.Channel list from {name: (t_s, v)} (datago), whole time.
`bench_figure_whole(channels, title, sweeps, reference_lines)`	The whole bench recording, every channel (parameter drop-down of
`_clean(f, Z, s)`	—
`fit_standard(f, Z, sigma)`	R_ohmic + L + two ZARCs, complex NLLS (standard_eis.fit_spectrum, the
`fit_drt(f, Z, sigma, rs_fixed)`	DRT (Tikhonov, NNLS) -> its peaks -> the DRT-informed ECM
`ecm_fits(run_dir, refit, say)`	{'Aggregate' | segment: {'f','Z', 'std': fit, 'drt': fit}} for one
`_write_ecm_table(res, path)`	—
`_Z(pairs)`	—
`ecm_figure(entry, title)`	2 x 2: Nyquist, DRT gamma(tau), |Z|(f), residual. Every trace carries
`ecm_viewer(figs, title)`	{(condition, spectrum): figure} -> Condition / Spectrum / Fit viewer.
`ecm_png(entry, path, title)`	Static Nyquist + residual of both fits (the thesis figure).

core/

Shared foundations: the configuration, numerical utilities, plate geometry, plate statistics, plate conditions and the lookup of the plate calibration files.

`core/config.py`

1,566 lines · 1 classes · 1 functions

Every tunable number in the pipeline, in one place.

Depends on	only standard / third-party libraries
Used by	bronze, csv_pipeline, gold, main, plate_figure, plate_plotly, plate_style, session, silver, validate_hf (+ 29 test files)
Third-party	—
Standard library	argparse, dataclasses, json, os, pathlib

Class / function	What it does (first docstring line)
`class Config` `@dataclass(frozen=True)`	One run of the pipeline. Methods: replace, famos_pattern, famos_patterns, preset, f_hi, f_eval_max, f_known_hi, to_dict, save, from_json, from_env, _coerce, build_argparser, from_cli.
`heatmap_limits(param)`	Fixed (vmin, vmax) in display units for `param`, or None = automatic.

`core/plate_conditions.py`

486 lines · 2 classes · 9 functions

The bench measures the operating point at the PORTS: gas in and gas out. The plate is 252 mm long, and what happens between those two ports is most of what makes a local EIS map interesting -- the cathode dries at the inlet and floods at the outlet, and the impedance follows. This module turns port measurements into a per-segment field so the two can be looked at on the same geometry.

Depends on	utils
Used by	main
Third-party	numpy
Standard library	dataclasses

Class / function	What it does (first docstring line)
`saturation_pressure_pa(t_c)`	Saturation vapour pressure of water, Buck (1996).

Class / function	What it does (first docstring line)
`dew_point_to_mole_fraction(dew_c, p_total_pa)`	Vapour mole fraction from a dew point and a total pressure.
`cathode_stoichiometry(current_a, dry_air_nl_min, o2_fraction)`	Air supplied / air needed, lambda.
`class PortState @dataclass`	The operating point at the ports, in SI-ish units the bench reports.
`port_state_from_bench(state, plate_t)`	Turn a `gamry_compare.BenchLog.state_at()` reading into a PortState.
`class Field @dataclass`	One scalar field over the segments, and how much of it was measured. Methods: measured, summary.
`_flow_coordinate(centroids, axis, inlet)`	Position of each segment along the flow path, 0 at the inlet, 1 at the
`_current_fraction(xi, areas, j_dc)`	Fraction of the total current produced upstream of each segment.
`condition_fields(centroids, areas, ports, temp_sensor_x_mm, j_dc, axis, in…)`	Per-segment temperature, pressure and relative humidity.
`_humidity_field(xi, areas, ports, t_field, p_field, j_dc)`	—
`_selftest(log)`	—

`core/plate_stats.py`

177 lines · 0 classes · 9 functions

Plate-level mean and median over the 36 representative segments.

Depends on	r2d2_geometry
Used by	csv_pipeline, gold, plate_figure, plate_maps, plate_plotly, plausibility (+ 2 test files)
Third-party	numpy, pandas
Standard library	pathlib

Class / function	What it does (first docstring line)
`tile_of(seg)`	(col0, col1, row0, row1), 1-based inclusive pad indices of the tile
`tile_bounds_mm(seg)`	(x0, x1, y0, y1) of the tile in mm, origin at the top-left pad.
`covered(seg, plate_name)`	The other segments with pads inside this tile -- the ones whose
`_finite(values)`	—
`tile_values(values, classes, positive)`	segment -> value for the representative segments that count: finite,
`tile_stats(values, classes, positive)`	n, mean, median, sd, CV [%], min, max, span [%] over the 36 tiles.
`summary_line(values, classes, dec, unit, sep)`	'35 of 36 tiles · mean .. · median .. · sd .. · CV .. · min..max ..'
`statistics_rows(summary)`	One row per parameter of a plate_summary table (a DataFrame or the
`write_statistics(summary_csv, out_csv)`	plate_summary.csv -> plate_statistics.csv beside it (or `out_csv`).

`core/r2d2_geometry.py`

1,064 lines · 2 classes · 28 functions

Geometry of the Bosch R2-D2 72-segment current-density measurement plates: per-segment active area, centroid, bounding box and pad membership -- for **both plate generations**.

Depends on	plate_style
Used by	bronze, card_gain, csv_pipeline, csv_source, dc_closure, eis_local, frequency_response, gold, main, neighbours, nyquist, plate_figure, plate_maps, plate_plotly, plate_stats, plate_style, plausibility, results, session, silver, utils, viewers (+ 15 test files)
Third-party	matplotlib
Standard library	dataclasses, pathlib

Class / function	What it does (first docstring line)
`class Segment @dataclass(frozen=True)`	One measured segment: the set of pads wired to it. Methods: from_rect, from_pads, name, n_pads, area_cm2, col0, col1, row0, row1, is_rectangle, x0_mm, x1_mm, y0_mm, y1_mm, w_mm, h_mm, cx_mm, cy_mm, runs.
`_parse_matrix(text)`	—
`_segments_from_matrix(grid)`	—
`_label_pads(segs)`	A pad inside the segment to hang its number on.
`_build_gen1()`	—
`_build_gen2()`	—

Class / function	What it does (first docstring line)
`class Plate @dataclass(frozen=True)`	Methods: title.
`plate_key(name)`	Normalise any spelling of a plate to its registry key.
`orientation_key(name)`	Normalise any spelling of a mounting orientation.
`cell_pad(col, row, orientation)`	Where drawing pad (col, row) sits in the cell for this mounting.
`cell_sensor_x(orientation)`	Temperature sensor -> x (cell mm) for this mounting.
`_mounted(p, orientation)`	The drawn plate `p` with every pad and label moved to its cell
`plate(name)`	The plate as mounted in this run (see use_plate).
`plate_frame(name)`	The plate as DRAWN, whatever the mounting -- for anything stated in
`use_plate(name, orientation)`	Select the plate generation, and how it was mounted, for the whole
`active_plate()`	—
`active_orientation()`	—
`areas(plate_name)`	Segment name -> active area in cm^2 (active plate unless given).
`equal_areas(plate_name)`	Every segment given the same area, A_CELL/72 = 4.235 cm^2.
`label_xy(plate_name)`	Segment name -> (x, y) in mm of the centre of its label pad -- the
`centroids(plate_name)`	Segment name -> (x, y) centroid in mm, origin at the top-left pad.
`interface_boxes(plate_name)`	Segment name -> interface box ('r1'..'r6'); {} for a plate whose
`areas_from_csv(path)`	Read an override table. Accepts 'segment,area_cm2' with , or ; and
`temperature_at(x_mm, sensor_T)`	Linear interpolation of the four plate temperature sensors in x.
`segment_temperatures(sensor_T, plate_name)`	—
`renumbering(src, dst)`	Best-effort map src segment number -> dst segment number, by centroid.
`symmetry_report(plate_name)`	Does the pad map mirror onto itself, left-right and top-bottom?
`self_check(verbose, plate_name)`	Assert that the reconstruction tiles the plate exactly.
`write_csv(path, plate_name)`	—
`plot_map(path, value, label, title, plate_name, limits)`	Draw the plate. Colour = `value` per segment, default = area.

`core/site_paths.py`

186 lines · 0 classes · 9 functions

Where the plate-specific input files live, for ANY user.

Depends on	only standard / third-party libraries
Used by	auto, bronze, results, session (+ 2 test files)
Third-party	—
Standard library	os, pathlib

Class / function	What it does (first docstring line)
`config_root(root)`	The shared folder: `root`, else $EIS_CONFIG_ROOT, else the Volume.
`_plate(plate)`	—
`plate_dirs(plate, root, package_dir)`	The folders searched for this plate, in order.
`legacy_dirs(plate, user_home, package_dir)`	Pre-Volume locations, gen1 only: the current user's own workspace
`_legacy_names(kind, plate)`	—
`resolve(plate, root, package_dir, user_home)`	Find every plate file. Returns
`matches_plate(path, plate)`	True when nothing in `path` names the OTHER plate.
`describe(res)`	Human-readable lines for the notebook printout.
`missing_message(res)`	The error text when a required file is missing.

`core/utils.py`

1,119 lines · 2 classes · 35 functions

Everything shared by more than one layer: estimators, uncertainty, small numerics, logging and IO. No layer defines its own copy of any of this.

Depends on	r2d2_geometry
Used by	bronze, card_gain, csv_pipeline, csv_source, dc_closure, frequency_response, gamry_compare, gold, hf_response, main, neighbours, plate_conditions, plausibility, silver, validate_hf (+ 5 test files)
Third-party	numpy
Standard library	contextlib, csv, dataclasses, io, json, logging, pathlib, re, sys, time

Class / function	What it does (first docstring line)
`_console_stream()`	stdout, put into UTF-8 if the platform allows it.
`class _SafeStreamHandler`	A handler no console character can break. Methods: emit.
`_rules_for(stream)`	The heavy and light rule characters this stream can actually carry.
`get_logger(verbose)`	—
`banner(text, log)`	—
`section(text, log)`	—
`timed(text, log)`	—
`_design3(t, f, ramp)`	[cos, sin, 1] and optionally a centred ramp column.
`fit3(y, fs, f, detrend)`	Three-parameter fit y ~ a cos(wt) + b sin(wt) + c at KNOWN f.
`fit4(y, fs, f0, n_iter, detrend)`	Four-parameter fit: frequency solved for as well (IEEE Std 1057).
`class JointFit @dataclass`	Result of the seven-parameter two-channel fit. Methods: ratio, phase_diff_rad.
`fit7_joint(y_ref, y_sig, fs, f0, n_iter, tol, detrend)`	Seven-parameter sine fit of two channels sharing ONE frequency.
`background_at(xq, fs, f, n, n_win, off_bins)`	What a channel shows at frequency `f` while nothing is excited.
`harmonic_distortion(y, fs, f)`	THD from the 2nd and 3rd harmonic -- the linearity check.
`stationarity(y, fs, f, n_sub)`	Spread of the phasor over `n_sub` sub-windows -- the time-invariance
`crlb_phasor(snr_db, n, freq_known)`	Cramer-Rao lower bounds for a single tone in white Gaussian noise.
`sigma_rel_from_snr(snr_db, n, model, floor, ceiling)`	Per-point relative standard deviation of |Z|, for KK weighting.
`zmag_outliers(freq, Z, n_mad, win)`	Flag points whose |Z| departs from the segment's own local trend.
`combine_snr_db(snr_a, snr_b)`	SNR of a ratio of two independent phasors, in dB.
`unwrap_phase_monotone(freq, Z)`	Phase of Z, unwrapped in order of increasing frequency.
`apply_delay(freq, Z, dt)`	Undo an acquisition skew of `dt` seconds.
`all_pass_residual(freq, Z)`	How much all-pass content the spectrum still carries.
`signed_delay_from_phase(freq, Z, decades)`	Delay from the high-frequency phase slope, with its sign and quality.
`hf_phase_slope(freq, Z, decades)`	d(phase)/d(ln f) over the top `decades` of the band, in rad.
`moving_mean(x, win)`	—
`runs_of_true(mask)`	Contiguous True runs of a boolean array as (start, stop) pairs.
`geometric_grid_fit(freqs, tol)`	Recover a sweep's geometric grid f_k = f0 * r^-k from its
`interp_complex(f_new, f, Z)`	Log-frequency interpolation of a complex spectrum.
`extrap_complex(f_new, f, Z, extrapolate)`	—
`nan_safe_stats(x)`	—
`read_pairs(path)`	Read a two-column calibration file ('c0;c1' per line).
`load_gain(path)`	Ex-situ frequency response of the measuring chain.
`gain_at(gain, seg, freq)`	Interpolate the chain response for one segment onto `freq`.
`write_json(path, obj)`	—
`write_table(path, rows, columns)`	Minimal CSV writer -- no pandas dependency in the core path.
`_selftest(seed)`	Synthetic checks of the estimators. Run: python utils.py
`segment_areas(cfg)`	The area table this run should use.

pipeline/

The processing: the orchestrator (main.py), the layers bronze, silver, gold, the separate CSV pipeline and the signal-processing modules they call.

`pipeline/bronze.py`

2,594 lines · 4 classes · 36 functions

CONTRACT

Depends on	card_gain, config, eis_local, gamry_sync, hf_schedule, ladder_snap, r2d2_geometry, site_paths, tone_estimation, utils
Used by	gamry_sync, main, session, silver (+ 21 test files)
Third-party	numpy, scipy
Standard library	csv, dataclasses, datetime, fractions, hashlib, json, pathlib, re, sys

Class / function	What it does (first docstring line)
`class ChannelInfo @dataclass`	One physical channel on one card. Methods: slot_seconds.
`class BronzeSpectrum @dataclass`	Raw spectrum of one segment: what the instrument actually reported. Methods: n_points, slot_delta_seconds, j_dc.
`class CardInfo @dataclass`	—
`class BronzeRun @dataclass`	Everything bronze knows. Serialisable, cacheable, re-runnable. Methods: segments_measured, segments_missing, summary, coverage_summary.
`discover_files(cfg)`	FAMOS files for THIS RUN, sorted so card order is deterministic.
`_one_test(files, cfg)`	Never mix two tests of one condition in one run.
`_digest(items)`	—
`inventory_channels(files, cfg, log)`	Open every card once and record what is on it, including slot order.
`cfg_stride(cfg)`	Decimation used for cheap statistics (std, DC level).
`estimate_card_lags(files, cards, cfg, log)`	Sample offset of every card relative to the first, from the reference.
`card_ratio(info)`	—
`card_shift(info)`	—
`to_own(idx, info)`	Common-base sample index -> this card's own sample index.
`to_common(idx, info)`	This card's own sample index -> common-base sample index.
`fs_common(lags, fs_seen)`	Sample rate of the common base: the anchor's.
`_to_rate(x, fs_from, fs_to)`	Polyphase resampling of an alignment trace onto another rate.
`_guard_samples(cfg, fs)`	The prominence guard for this card, in samples.
`_alignment_trace(stem, cards, cfg)`	The card's reference, band-passed to the band the lag is measured in.
`_best_lag(x, ref, max_lag, guard)`	The lag of best agreement, its correlation, and its prominence.
`_prominence(mag, k, guard)`	How many robust sigma the peak stands above the rest of the curve.
`_agreeing_cards(stem, lag, candidates, cards, cfg)`	Which other cards independently returned the same offset.
`_blockwise_lag_diagnostic(x, ref, global_lag, fs, cfg)`	Does ONE constant lag hold for the whole record, or do the clocks drift?
`_pairwise_closure(stems, traces, lags, cards, cfg, guard)`	Do the anchor-relative lags agree with direct card-to-card lags?
`_pick_anchor(stems, traces, max_lag, log, guard)`	The card whose clock the others are shifted onto: the one they agree
`consensus_schedule(files, cards, cfg, log, lags)`	One excitation schedule for the whole plate.
`bronze_frequency_coverage(run_obj)`	How much of the plate actually carries each frequency.
`coverage_summary(rows, min_area_coverage)`	What the coverage table says in one line, for the manifest.
`_finite_median(values)`	Median of the finite values, or NaN when there are none.
`_gamry_guided(kept, tl, files, cards, cfg, lags, fs_seen, gs_mode, log)`	gamry_sync on the consensus schedule: report, exact frequencies, and
`_on_grid(f, grid, tol)`	—
`refine_frequencies(files, cards, schedule, grid, cfg, lags, log)`	One exact frequency per step, used by EVERY fit that follows.
`pooled_reference_phasors(files, cards, schedule, cfg, lags, log)`	One cell-voltage phasor per step, averaged over every card.
`_sensor_key(channel_name)`	FAMOS channel name -> calibration key.
`plate_temperatures(files, cards, cal, cfg, log)`	One temperature field for the whole plate, from every sensor on every
`_K_for(seg, cal, T, median_c0, median_c1, cfg)`	Transfer coefficient, imputing from the plate median if necessary.
`_fix_polarity(Z, freqs, snr_db, cfg)`	Return Z or -Z: a channel wired the other way round inverts its whole

Class / function	What it does (first docstring line)
`process_card(fp, cal, schedule, grid, cfg, log, T_seg, lag, ref_pool, …)`	Raw phasors for every segment on one card.
`run(cfg, log)`	Ingest every card into one BronzeRun.
`load(run_dir)`	Rebuild a BronzeRun from the tables `save` wrote.
`save(run_obj, cfg, log)`	Write bronze tables so silver can be re-run without touching .DAT.

`pipeline/channel_lag.py`

332 lines · 1 classes · 11 functions

Every segment's current-measurement chain, checked against the plate on every run: does it see the same high-frequency phase as the others, and if it lags, by how much? Removes the lag before R_ohmic is read.

Depends on	only standard / third-party libraries
Used by	csv_pipeline, gamry_dta, silver (+ 1 test files)
Third-party	numpy, scipy
Standard library	dataclasses

Class / function	What it does (first docstring line)
`class ChannelLag @dataclass`	Methods: row.
`_opt(cfg, name, default)`	—
`mode(cfg)`	—
`_key(f)`	—
`lag_model(cfg)`	—
`correction(freq, tau_s, model)`	The factor that removes the channel's lag: Z_true = Z_meas * this.
`predicted_phase(freq, tau_s, model)`	—
`plate_reference(items)`	Complex plate median per frequency, from the usable points only.
`fit_tau(freq, phase, tau_max, model)`	(tau, rms residual in deg) for phase = predicted_phase(f, tau),
`estimate(items, cfg, log)`	Lag per segment. `items` : segment -> (freq, Z, usable mask).
`_log_summary(out, m, f_lo, log)`	—
`card_summary(lags, card_of)`	Per card: how many segments, how many unreliable, median tau.

`pipeline/csv_pipeline.py`

1,715 lines · 1 classes · 25 functions

The evaluation path for `cfg.source_format == "csv"`. It produces the same products as bronze/silver/gold -- per-segment spectra, a cell aggregate, ECM parameters, Nyquist plots and plate heat maps -- from a completely different starting point, and it does NOT reuse the FAMOS stages. This module explains why in the places where the difference is a decision rather than an accident.

Depends on	channel_lag, config, csv_source, eis_local, eis_validation, gamry_compare, gold, nyquist, plate_figure, plate_stats, r2d2_geometry, silver, utils
Used by	main, views (+ 3 test files)
Third-party	matplotlib, numpy, scipy
Standard library	argparse, dataclasses, json, pathlib, time

Class / function	What it does (first docstring line)
`_uniform_view(t, y, uniform, fs)`	`y` on a uniform grid -- FOR PEAK DETECTION ONLY.
`_peaks(y, fs, f_lo, f_hi, snr_db, dyn_range_db)`	Local maxima of the windowed amplitude spectrum, above TWO thresholds.
`detect_tones(m, cfg, log)`	Find the excitation frequencies in the cell-voltage record.
`_detect_sweep_windows(t, y, fs, uniform, f_lo, f_hi, cfg)`	Split a stepped sweep into (start, stop, freq) dwells.
`deskew(A, tau, freq, conjugated)`	Move one channel's phasor back to the row's nominal instant.
`nyquist_zone(phasors, taus, f_alias, fs, max_zone)`	Which analogue frequency does the channel scan say this point was at?
`r2d2_point(m, cfg, log, f_true)`	One point file -> one frequency and one phasor per channel.
`sweep_grid_check(freqs)`	Do the recovered frequencies form a real sweep?
`r2d2_sweep_spectra(points, cfg, log)`	Assemble the per-segment spectra from a folder of point files.

Class / function	What it does (first docstring line)
`class SegmentSpectrum @dataclass`	—
`apply_scan_skew(freq, Z, seg_slot, ref_slot, scan_rate_hz)`	Remove a KNOWN channel-scan offset.
`segment_spectra(m, sched, cal, T_seg, cfg, log)`	Z_s(f) = U_cell(f) / j_s(f) for every segment, with uncertainty.
`from_frequency_file(m, cfg, log)`	A CSV that already holds spectra: no phasor estimation at all.
`z_zarc(w, R, tau, n)`	One ZARC (R in parallel with a CPE), parameterised by (R, tau, n).
`z_model(p, freq, n_arcs)`	Rs + jwL + sum of ZARCs.
`ecm_start(freq, Z, n_arcs)`	Starting values read off the spectrum, not guessed.
`fit_ecm(freq, Z, sigma_rel, n_arcs, weight)`	Weighted complex non-linear least squares with parameter intervals.
`capacitive_band(freq, Z)`	Mask of the points between the HF and the LF zero crossing of Z''.
`choose_n_arcs(freq, Z, sigma_rel)`	Fit one arc and two, keep the one the data supports.
`cell_aggregate(spectra, areas, a_cell)`	Z_cell = A_cell / sum_s (A_s / Z_s) -- the segments are in parallel.
`validate(sp, cfg)`	Linear Kramers-Kronig residuals -- is the spectrum a spectrum?
`write_outputs(spectra, ecm, agg, cfg, log, extra, reader_meta)`	—
`run_csv(cfg, stop_after)`	The CSV path, end to end.
`_apply_channel_lag(spectra, cfg, log)`	channel_lag.estimate on CSV spectra; Z corrected in place. Returns the
`_seg_key(s)`	—
`_finish_csv(spectra, sched, m, cfg, log, t0, stop_after, extra)`	Validation, ECM, aggregation and output — shared by every CSV route.

`pipeline/eis_local.py`

1,743 lines · 6 classes · 30 functions

Local electrochemical impedance spectroscopy on the Bosch R2-D2 72-segment plate -- evaluated WITHOUT any potentiostat file.

Depends on	eis_validation, hf_schedule, r2d2_geometry
Used by	abgleich, bronze, card_gain, csv_pipeline, datago, gamry_dta, gamry_sync, hf_schedule, session (+ 14 test files)
Third-party	matplotlib, numpy
Standard library	argparse, dataclasses, datetime, json, pathlib, re

Class / function	What it does (first docstring line)
`class FamosV1 @dataclass`	imc FAMOS **v1** binary reader -- one |CP carrying every channel name. Methods: channel, position, segment_names, uc_names, temp_names.
`class FamosV2`	imc FAMOS **v2**: one metadata block per channel, and float64 samples. Methods: channel, position, segment_names, uc_names, temp_names.
`_famos_plausible(fam)`	Empty string if this parse can be believed, else why it cannot.
`famos_start_time(path)`	The trigger time a FAMOS header states (|NT), as a naive datetime in
`class FamosFile`	Open a FAMOS recording, whichever dialect it is written in.
`pick_reference_channel(fam, prefer, stride, log)`	The UC channel to use as this card's reference. Fixed by name.
`class ChannelIntegrityError`	A card whose channels cannot be what their names say.
`channel_integrity(fam, ref_name, n_probe, probe_len, notes)`	Fatal problems with the channel CONTENT of one card ([] when sound).
`class PlateCalibration @dataclass`	j_s [A/cm^2] = u_s [V] / (c0 + 0.001 * c1 * T[degC]) per segment Methods: load, has_current_cal, K, temperature.
`_read_pairs(path)`	—
`load_gain(path)`	Optional dynamic response of the measuring chain, from the ex-situ
`gain_at(gain, seg, freq)`	—
`fit3(y, fs, f)`	Three-parameter fit y ~ a*cos(wt) + b*sin(wt) + c at known f.
`fit4(y, fs, f0, n_iter)`	Four-parameter fit: also solves for the frequency (IEEE Std 1057).
`harmonic_distortion(y, fs, f)`	THD from the 2nd and 3rd harmonic; the linearity check.
`_moving_mean(x, win)`	—
`demod_envelope(x, fs, f, n_cyc, min_win_s, max_win_s)`	|complex amplitude at f| as a function of window start time.
`dwell_window(x, fs, f, centre, frac, trim)`	Contiguous interval around `centre` where the response at f is above

Class / function	What it does (first docstring line)
`polish_window(x, fs, f, a, b, iters)`	Trim the window until the fit residual stops improving.
`_runs(mask)`	Contiguous True runs of a boolean array as (start, stop) pairs.
`fill_gaps(ref, fs, steps, min_snr_db, max_rounds)`	Recover steps that the log grid stepped over.
`_collapse_overlapping(steps, min_overlap)`	Two steps cannot occupy the same stretch of the record.
`_dedupe(steps, rel_tol, overlap_tol)`	One physical step must yield one entry.
`class Step @dataclass`	Methods: window_repaired, valid.
`_stationarity(x, fs, f, start, stop)`	—
`detect_schedule(ref, fs, ppd, f_lo, f_hi, min_snr_db, verbose)`	Find every stepped-sine dwell in the record without being told the
`segment_spectrum(fam, uc_name, seg, steps, K, gain, uc_gain)`	Z_s(f) in ohm*cm^2, with per-step SNR.
`deskew(freq, Z, fam, uc_name, dt_range, verbose)`	Remove the acquisition skew between the reference and the segment
`dc_closure(j_dc, areas, i_setpoint)`	sum_s j_s * A_s against the known cell current.
`aggregate(freq, Z, areas)`	Cell-level spectrum predicted from the local ones, in ohm*cm^2.
`_mpl()`	—
`plot_spectra(freq, Z, Z_cell, title, path)`	—
`plot_schedule(steps, fs, path)`	—
`plot_validation(val, path)`	—
`evaluate(dat_dir, out_dir, curr_cal, temp_cal, gain_file, uc_gain_…)`	—
`main()`	—

`pipeline/eis_measurement_model.py`

493 lines · 0 classes · 9 functions

The stage that turns a scattered local-EIS spectrum into a clean one, without any reference instrument and without cosmetic smoothing.

Depends on	only standard / third-party libraries
Used by	silver (+ 2 test files)
Third-party	numpy
Standard library	—

Class / function	What it does (first docstring line)
`has_inductive_tail(freq, Z, n_top)`	True when the spectrum is inductive at the TOP of its band: the mean
`_design(freq, taus, with_L)`	Complex design matrix [1, (jw), 1/(1+jw tau_k) ...].
`_weighted_lstsq(A, Z, w, ridge, n_free)`	Least squares on the complex system, stacked into real form.
`_mu(R)`	Schoenleber's mu: 1 - sum|R_neg| / sum R_pos. Falls towards 0 as the
`fit_kk_model(freq, Z, sigma_rel, M, mu_crit, with_L, tau_pad, ridge)`	Fit R0 + jwL + sum R_k/(1+jw tau_k) with fixed log-spaced tau.
`fit_delay_and_model(freq, Z, sigma_rel, dt_range, n_grid, refine)`	Find the acquisition delay by minimising the KK-model residual.
`fit_shared_delay(spectra, dt_range, n_grid, M_scan, verbose)`	ONE delay for a whole card, fitted from all its segments at once.
`clean_segment(freq, Z, snr_db, deskew, remove_L, dt_fixed, min_snr_db, …)`	Full cleaning chain for one segment. Z in ohm*cm^2.
`aggregate_asr(freq, Z_by_seg, areas, A_cell)`	Z_cell^ASR = A_cell / sum_s (A_s / Z_s^ASR).

`pipeline/eis_validation.py`

268 lines · 0 classes · 5 functions

Instrument-independent quality control for impedance spectra.

Depends on	only standard / third-party libraries
Used by	csv_pipeline, eis_local, silver (+ 1 test files)
Third-party	numpy
Standard library	—

Class / function	What it does (first docstring line)
`_linkk_fit(freq, Z, M, add_inductance, add_capacitance)`	Least-squares fit of R0 [+ jwL] [+ 1/jwC] + sum_k R_k/(1+jw tau_k).
`_mu(R)`	Schönleber's mu: 1 - sum|R_neg| / sum|R_pos|.
`lin_kk(freq, Z, mu_crit, max_M, add_inductance, tol)`	Run the lin-KK test. Returns residuals and a pass/fail verdict.
`z_hit(freq, Z, gamma, fit_band)`	Reconstruct ln|Z| from the phase (Ehm/Göhr approximation).
`estimate_delay(freq, Z, dt_range, n_grid, fit_band)`	Find the pure time delay hidden in a spectrum, using Z-HIT.

`pipeline/gamry_sync.py`

860 lines · 2 classes · 17 functions

Put the FAMOS recording on the Gamry's clock, and let the Gamry say which frequency was applied when.

Depends on	bronze, eis_local, gamry_compare, hf_schedule
Used by	bronze, dc_closure, gamry_compare, main, session, silver (+ 2 test files)
Third-party	numpy
Standard library	dataclasses, datetime, pathlib, re

Class / function	What it does (first docstring line)
`mode(cfg)`	—
`class Timeline @dataclass`	One Gamry sweep in the order it was run.
`_num(s)`	—
`read_timeline(path)`	Freq and Time columns of a Gamry .dta ZCURVE table.
`read_vdc(path)`	Median of the Vdc column of a Gamry .dta ZCURVE table (the cell's DC
`find_timeline(cfg, log)`	The sweep for cfg.condition under cfg.gamry_dir (same rules as the
`recording_window(cfg, files)`	(start, end) of this run's FAMOS recording from the cards' header
`clock_report(lags, files, fs_common, gamry_offset_s, gamry_started, sw…)`	What the header stamps say against what the signals say.
`class SyncResult @dataclass`	Methods: summary.
`_match(f, freqs, tol)`	—
`align(steps, tl, fs, f_lo, f_hi, tol, min_snr_db, max_spread_s,…)`	Offset between the FAMOS common time base and the Gamry clock, and a
`apply_frequencies(steps, sync, rebuild)`	Matched steps take the Gamry's exact frequency. Windows untouched.
`confine_windows(steps, sync, fs, rebuild, min_interval_s, guard_s, settle…)`	Put every long (low-frequency) step's window inside its Gamry step.
`separate_windows(steps, fs, rebuild, min_cycles, log)`	No two windows may overlap: one stretch of record holds one tone.
`_lockin_power(chans, fs, f, a, b, hop, win)`	Array power at f in sliding windows over [a, b): sum over channels of
`locate(chans, fs, f, lo, hi, win_cycles, min_dwell_cycles, trim_…)`	The dwell of a tone at f inside [lo, hi): the contiguous run around
`_dominates(chans, fs, f, a, b, ratio, min_db)`	Is the tone at f stronger, in [a, b), than the sweep's neighbours
`relocate(steps, sync, tl, chans, fs, lag, rebuild, make_step, log,…)`	Re-locate misplaced and missing steps in their Gamry slot -- and every
`corroborate_card_lags(per_card, lags, fs, tl, f_lo, f_hi, tol_s, fs_steps)`	Check every card's own clock against the Gamry's.

`pipeline/gold.py`

796 lines · 2 classes · 12 functions

Gold answers the question the whole rig exists to answer: WHERE on the plate is something wrong, and what is it.

Depends on	config, nyquist, plate_figure, plate_plotly, plate_stats, r2d2_geometry, silver, utils
Used by	csv_pipeline, main (+ 6 test files)
Third-party	matplotlib, numpy
Standard library	dataclasses, pathlib

Class / function	What it does (first docstring line)
`_run_title(cfg)`	' / ', plus the plate when it is not gen1, so a gen2

Class / function	What it does (first docstring line)
`class SegmentRecord @dataclass`	One segment on the finished plate, measured or inferred.
`class GoldRun @dataclass`	Methods: measured, inferred.
`split_processes(sp, cfg)`	Bucket the DRT by time constant into ohmic / charge-transfer / transport.
`spatial_trends(records, params, plate_w_mm)`	Is a pattern on the map a gradient, or is it the segment geometry?
`collect_parameters(sr, cfg)`	parameter -> {segment: value}, and the matching uncertainties.
`re_at(freq, Z, f0)`	Re Z at f0 from the kept points, interpolated in log f; nan when f0
`_kernel(X1, X2, lx, ly, kind)`	Anisotropic kernel on plate coordinates, in mm.
`fit_field(values, sds, cfg)`	Gaussian-process regression over the segment centroids.
`label_faults(records)`	Ratio-to-median rules, applied in place.
`plate_heatmap(records, param, cfg, title_extra)`	Static plate map with every segment drawn, measured or inferred.
`plate_heatmap_interactive(records, param, cfg)`	Plotly version: same map, with per-segment hover carrying the caveats.
`run(sr, cfg, log)`	—
`save(gr, sr, cfg, log)`	—

`pipeline/hf_schedule.py`

1,145 lines · 5 classes · 14 functions

THE PROBLEM

Depends on	eis_local
Used by	bronze, eis_local, gamry_sync
Third-party	numpy
Standard library	dataclasses, math

Class / function	What it does (first docstring line)
`crlb_usable(snr_db, n, sigma_rel_max, freq_known)`	Is a phasor from `n` samples at `snr_db` precise enough to keep?
`class LazyChannels`	Read-through view of one card's segment channels. Methods: keys, window.
`_names(chans)`	—
`_window(chans, name, a, b)`	One window out of one channel, without materialising the channel.
`_standardise(x)`	Zero mean, unit variance, NaNs removed.
`polarity_aligned_reference(chans, stride, log)`	Sum the segment channels into one virtual reference trace.
`class Ladder @dataclass`	f_k = f0 * ratio^-k, with dwell_s(f) = max(d_min, n_cyc / f). Methods: freq_of, index_of, tol_at, dwell_s.
`_snap_ppd(ppd_free, rel_tol)`	Round the spacing to the nearest integer points/decade when it is
`fit_ladder(freqs, starts, stops, fs, tol, snap)`	Recover the sweep's geometric ladder and dwell law.
`prune_off_ladder(freqs, lad, tol)`	Boolean mask: which of `freqs` sit on the ladder.
`_binom_sf_threshold(m, p, alpha)`	Smallest c with P(Binomial(m, p) >= c) <= alpha.
`class CfarResult @dataclass`	Outcome of the frequency-domain CFAR at one candidate rung. Methods: p_false_alarm, accept.
`cfar_channel_count(chans, fs, f, a, b, n_bg, guard)`	Frequency-domain CFAR at `f`, over the channel array, one snapshot.
`class Rank1Result @dataclass`	Outcome of the maximum-eigenvalue test at one candidate rung. Methods: usable, accept.
`rank1_statistic(chans, fs, f, a, b, n_snap)`	Maximum-eigenvalue detection across the channel array.
`class HFSchedule @dataclass`	What `recover_schedule` returns. Methods: as_steps, summary.
`_median_channel_snr_db(chans, fs, f, a, b, max_ch)`	SNR a SINGLE channel sees at this step, not the stack's.
`recover_schedule(chans, fs, f_lo, f_hi, ppd, min_snr_db, sigma_rel_max, ex…)`	Recover one card's excitation schedule from its segment channels.
`_verify_rung(chans, ref, fs, f_pred, lad, anchors, ladder_tol, sigma_r…)`	Locate one predicted rung in time, then accept or reject it.

`pipeline/ladder_snap.py`

364 lines · 0 classes · 7 functions

Snap a recovered excitation schedule onto its own geometric ladder.

Depends on	only standard / third-party libraries
Used by	bronze (+ 2 test files)
Third-party	numpy
Standard library	—

Class / function	What it does (first docstring line)
`fit_ladder(freq, ppd_candidates)`	Points per decade from the MEDIAN log-spacing of the detections.
`ladder_phase(freq, n_samples, amp, ppd, fs, min_cycles, dur_tol, outli…)`	Ladder offset in [0,1), from the cleanly resolved fixed-dwell steps.
`snap(freq, ppd, offset)`	Nearest rung frequency, distance to it in rung units, and rung index.
`snap_schedule(rows, ppd, offset, fs, amp_key)`	Collapse a detected schedule onto its ladder.
`snap_steps(steps, fs, ppd, offset, log)`	Snap a list of `eis_local.Step` onto the excitation ladder.
`_rebuild(cls, src)`	A copy of `src` with `changes` applied, keeping every other field.
`repair_windows(steps, fs, descending, min_dwell_frac, t_tol_s, max_repai…)`	Replace dwell windows that cannot belong to the sweep.

`pipeline/main.py`

688 lines · 0 classes · 10 functions

python main.py --dat /Volumes/.../Famos --curr-cal curr.csv --temp-cal temp.csv --leepa 2611976 --condition 450A --out ./results --current 450

Depends on	bronze, card_gain, config, csv_pipeline, csv_source, dc_closure, eis_paths, frequency_response, gamry_compare, gamry_sync, gold, plate_conditions, plausibility, r2d2_geometry, silver, utils, validate_hf
Used by	session (+ 2 test files)
Third-party	—
Standard library	argparse, datetime, json, pathlib, sys, time

Class / function	What it does (first docstring line)
`build_parser()`	—
`config_from_args(a)`	—
`geom_area(cfg)`	The cell area the comparison must use.
`run_pipeline(cfg, stop_after)`	bronze -> silver -> gold, with a manifest tying the three together.
`_recording_window(cfg)`	FAMOS header span of this run (gamry_sync.recording_window), or the
`reevaluate(run_dir, out_dir)`	Silver -> gold -> plausibility again, on a finished FAMOS run.
`_reapply_card_gain(br, cfg, log)`	Bring the saved Z to what cfg.card_gain asks for.
`whole_cell_and_plausibility(cfg, manifest, log, sr)`	Gamry cross-check, then plausibility -- shared by FAMOS and CSV paths.
`self_test()`	Everything that can be checked without measurement data.
`main(argv)`	—

`pipeline/neighbours.py`

460 lines · 2 classes · 9 functions

from neighbours import adjacency, scalar_outliers, spectrum_outliers

Depends on	r2d2_geometry, utils
Used by	silver (+ 2 test files)
Third-party	core, numpy
Standard library	dataclasses

Class / function	What it does (first docstring line)
`adjacency(plate_name)`	Segment -> the set of segments sharing a pad edge with it.
`_modified_z(x, ring)`	(z, median, MAD-as-sigma) of `x` against `ring`.
`class Finding @dataclass`	One segment, one parameter, one verdict. Methods: direction, as_row.
`_as_int(name)`	—

Class / function	What it does (first docstring line)
`scalar_outliers(values, param, plate_name, z_cut, adj)`	Segments whose value stands out from the ring that touches them.
`class SpectrumFinding @dataclass`	Methods: worst, band_summary.
`spectrum_outliers(spectra, plate_name, adj, min_points)`	Per-frequency departure of each segment from its neighbour ring.
`donor_ring(seg, available, plate_name, max_hops)`	The nearest ring of AVAILABLE segments around `seg`.
`fill_spectrum(seg, spectra, areas, plate_name, max_hops)`	An area-specific spectrum for `seg`, built from its neighbours.
`analyse(params, spectra, plate_name, z_cut)`	Run every scalar parameter and, when given, the spectra.
`_self_test()`	Plant one fault and check it is the segment that comes back.

`pipeline/silver.py`

2,728 lines · 4 classes · 38 functions

Silver turns raw phasors into spectra that can be believed, and attaches an uncertainty to every number so the gold layer can decide how much weight to give it. Nothing here is cosmetic: every step either removes a known instrumental effect or reports how well the data survives a test it could have failed.

Depends on	bronze, card_gain, channel_lag, config, eis_measurement_model, eis_validation, gamry_compare, gamry_sync, hf_response, neighbours, r2d2_geometry, standard_eis, utils
Used by	csv_pipeline, gold, main, plausibility, validate_hf (+ 12 test files)
Third-party	numpy, scipy
Standard library	dataclasses, pathlib

Class / function	What it does (first docstring line)
`class SkewModel @dataclass`	Structural acquisition-skew model for one card. Methods: slot_ratio, dt_for.
`class SilverSpectrum @dataclass`	One segment, corrected, modelled and graded. Methods: j_dc, R_ohmic_cv.
`class SilverRun @dataclass`	Methods: card_factors, reach, tiers.
`_kk_residual(freq, Z, sigma_rel, M)`	Weighted KK-model residual: how far the spectrum is from realisable.
`basis_value(basis, slot_seg, slot_ref)`	Channel-order term of the skew model, under either hardware hypothesis.
`skew_cost(prep, dt0, k, M, basis)`	Mean normalised KK residual of a card under dt = dt0 + k*basis.
`fs_prep(prep, default)`	Sampling rate implied by the prepared spectra (for delay bounds).
`optimise_skew(prep, k_nom, cfg, basis)`	Coarse-to-fine 2-D search for (dt0, k).
`fit_structural_skew(card, items, cfg, log, basis_override)`	Fit dt(slot) = dt0 + k * (slot_seg - slot_ref) for one card.
`_dt0_identifiability(prep, k, k_nom, cfg, basis)`	Is the common-mode delay visible above the noise in the cost?
`residual_skew_check(spectra, cfg, log)`	After correction, is there any channel-order structure LEFT?
`sp_slot(sp)`	—
`choose_global_basis(by_card, cfg, log)`	Decide the converter architecture ONCE for the whole plate.
`_prepare_card(items, cfg)`	Shared preparation used by both the basis vote and the per-card fit.
`fit_per_card_skew(card, items, cfg, log)`	Legacy single-delay-per-card model, kept for A/B comparison.
`_drt_design(freq, tau, with_L)`	[1, (jw), 1/(1+jw tau_m) ...] -- linear in every parameter.
`_se_kernel(x, sigma_f, ell)`	—
`bayesian_drt(freq, Z, sigma_rel, cfg)`	Gaussian-process DRT with a closed-form posterior.
`cfg_len(cfg)`	—
`r_ohmic_topband(freq, Z, sigma_rel, span_decades, n_min, n_max)`	R_inf as an inverse-variance weighted mean of Re Z at the top of band.
`r_ohmic_method(cfg)`	'intercept' or 'topband' -- config.r_ohmic_method, 'auto' resolved
`r_ohmic_intercept(freq, Z, sigma_rel, f_min_hz)`	The high-frequency intercept: Re Z where Im Z changes from negative
`r_ohmic_axis_fit(freq, Z, n_top)`	R_inf by extrapolating the top of the arc to Im Z = 0.
`extrapolate_hf(drt, f_hi, n)`	Evaluate the posterior above the measured band.
`_file_key(path)`	Path plus modification time and size: a file rebuilt in the same
`_hf_response(path)`	utils.load_gain of the frozen high-frequency response, read once.

Class / function	What it does (first docstring line)
`_hf_sources(path)`	{segment: 'segment' | 'card'}: which channels the response file gives
`gate_points(sp, cfg)`	The point-level gates of process_segment, on their own.
`process_segment(sp, skew, cfg, log, ledger, chain, card_phase, card_phase…)`	De-skew, weight, model, validate and grade one segment.
`channel_lag_items(spectra, skew, cfg)`	segment -> (freq, de-skewed Z, usable) for channel_lag.estimate.
`card_hf_phase(spectra, skew, lags, cfg, log)`	Per-card phase offset above the phase-coherent limit (full band only).
`_segment_phase_items(spectra, skew, lags, card_ph, cfg)`	(freq, corrected Z, keep, fs) per segment: the de-skew, the channel lag
`gamry_card_phase(spectra, skew, lags, cfg, log)`	Each card's phase calibrated against the condition's Gamry sweep.
`dc_closure(spectra, cfg)`	Sum(j_s * A_s) against the load current.
`cell_aggregate(spectra)`	Area-weighted harmonic mean -- what a cell-level instrument would see.
`class FilledSpectrum`	A segment reconstructed from its neighbours.
`build_filled_spectra(spectra, cfg, log)`	Rebuild substituted and (optionally) missing segments from neighbours.
`cell_aggregate_full(spectra, filled, a_cell)`	The cell curve, with the arithmetic written out and both normalisations.
`dc_share_faults(spectra, cfg)`	Segments whose DC current density says the CHANNEL is broken.
`run(bronze_run, cfg, log)`	—
`uc_gain_pct(card_ref)`	card -> gain in % of the UC channel its Z divides by, AFTER whatever
`save(sr, cfg, log)`	—

`pipeline/standard_eis.py`

283 lines · 0 classes · 9 functions

R_ohmic, R_ct, R_mt and R_pol the standard way: an equivalent-circuit fit.

Depends on	only standard / third-party libraries
Used by	silver, views (+ 1 test files)
Third-party	numpy, scipy
Standard library	—

Class / function	What it does (first docstring line)
`zarc(w, R, tau, n)`	R ‖ CPE, written with its time constant.
`z_model(p, freq)`	Z of R_ohmic + jωL + two ZARCs; p = (Rs, L, R1, tau1, n1, R2, tau2, n2).
`_to_p(x)`	Fit vector -> parameters. x = (Rs, L, R1, log10 τ1, n1, R2, Δlog10 τ, n2).
`graphical(freq, Z)`	Model-free numbers read straight off the spectrum.
`hf_intercept(freq, Z, n_top)`	R_ohmic from the high-frequency end, model-free.
`capacitive_band(freq, Z)`	Mask of the points the circuit describes: everything above the
`_starts(f, Z)`	Starting vectors read off the spectrum.
`fit_spectrum(freq, Z, sigma_rel, rs_fixed)`	Fit the standard circuit to one spectrum.
`choose_n_arcs(freq, Z, sigma_rel)`	Drop-in for csv_pipeline.choose_n_arcs: always the standard two-arc

`pipeline/tone_estimation.py`

297 lines · 1 classes · 9 functions

Card-level excitation-tone estimation for the local-EIS pipeline.

Depends on	only standard / third-party libraries
Used by	bronze (+ 2 test files)
Third-party	numpy, scipy
Standard library	dataclasses

Class / function	What it does (first docstring line)
`_design(n, fs, f, ramp)`	—
`phasors_at(Y, fs, f, ramp)`	Three-parameter (+ramp) LS of every row of Y at ONE frequency.
`_rss(Y, fs, f, ramp)`	—
`class ToneEstimate @dataclass`	—
`_hann(n)`	—
`_periodogram_peak(Y, fs, f_lo, f_hi)`	Multichannel IpDFT: SNR-normalised summed Hann periodogram, 2-point
`card_frequency(Y, fs, f_prior, search_bins, ramp, prior_band_bins, xtol_…)`	One frequency for all channels of one card in one dwell window.
`fit4(y, fs, f0, n_iter, detrend)`	IEEE-1057 four-parameter fit with the two defects removed.
`stationarity_test(y, fs, f, n_sub, alpha, ramp)`	Chi-square test of phasor constancy across n_sub sub-windows.
`_selftest(seed)`	—

checks/

Diagnostics that judge whether a finished result is believable. None of them can stop a run.

`checks/card_gain.py`

670 lines · 2 classes · 18 functions

Does every card measure the same cell voltage with the same gain? And does the current scale (K) close against the bench, at every operating point?

Depends on	eis_local, r2d2_geometry, utils
Used by	bronze, dc_closure, frequency_response, main, plausibility, silver (+ 3 test files)
Third-party	numpy
Standard library	csv, dataclasses, pathlib, re, warnings

Class / function	What it does (first docstring line)
`_opt(cfg, name, default)`	—
`mode(cfg)`	—
`estimate_snr_db(snr_sample_db, n)`	SNR of a fitted phasor from the per-sample SNR of its window: the
`short(card)`	—
`class CardReference @dataclass`	Methods: gain_pct, flat_pct, row.
`_band(freq, vals, lo, hi)`	—
`compare_phasors(freq, A, snr, dc, cfg, channel, used_for_Z)`	Card against card for ONE channel name.
`hf_response(freq, A, snr, cfg, ref_card)`	Each card's UC magnitude against ONE reference card, step by step.
`applied_hf(refs)`	card -> per-step |UC| response its Z is divided by (cfg.uc_hf_equalise).
`correctable(ref)`	—
`measure_reference(files, cards, schedule, cfg, lags, log)`	{channel: {card: CardReference}} for every UC channel the cards share.
`load_reference(run_dir)`	Read bronze/card_reference.csv back ({} when the run has none).
`applied_gains(refs)`	card -> what its Z is divided by (1.0 where nothing is applied).
`_log(out, m, log)`	—
`_fkey(f)`	—
`class CardFactor @dataclass`	Methods: spread_pct, level_pct, row.
`impedance_factors(spectra, card_of, voltage, cfg)`	Per card: how far its segments' |Z| sits from the plate median, per band.
`current_of(condition)`	"450A" -> 450.0; nan when the name carries no current.
`dc_closure_table(rows)`	Fit I_meas = a * I_ref + b over the conditions.
`plate_current(j_dc, areas, plate_key)`	(I over the measured area, measured area, I scaled to the plate).

`checks/dc_closure.py`

282 lines · 0 classes · 11 functions

Do the segment currents add up to the bench current -- at every operating point -- and is what is missing a SCALE (which sits in every impedance too) or an OFFSET (which does not)?

Depends on	card_gain, gamry_compare, gamry_sync, r2d2_geometry, utils
Used by	main (+ 1 test files)
Third-party	matplotlib, numpy
Standard library	argparse, csv, json, pathlib

Class / function	What it does (first docstring line)
`_rows(path)`	—
`_f(x)`	—
`find_runs(paths)`	—
`bench_current(run, bench_log, gamry_dir, cfg)`	(current, source) for this run's condition.
`uc2_dc(run)`	Median over the cards of the UC2 DC level (bronze/card_reference.csv).
`gamry_vdc(gamry_dir, condition)`	—
`sense_offset(rows, plate_key)`	Fit UC2 - Vdc = b - I * R_x over the conditions.
`analyse(runs, bench_log, gamry_dir, plate_key)`	—
`plot(res, path)`	—
`print_sense(res)`	—
`main(argv)`	—

`checks/frequency_response.py`

542 lines · 0 classes · 19 functions

How does the measuring chain of every segment respond to frequency -- on the bench (ex-situ) and during the measurement (in-situ)?

Depends on	card_gain, gamry_dta, plausibility, r2d2_geometry, utils
Used by	main, views (+ 1 test files)
Third-party	matplotlib, numpy, pandas
Standard library	argparse, json, pathlib

Class / function	What it does (first docstring line)
`_short(card)`	—
`exsitu_from_gain(path)`	G_d per segment from a pipeline gain file (gain = 1/H -> H = 1/gain).
`exsitu_from_bode(folder, f_ref_hz)`	G_d per segment straight from the Abgleich bode/ sweeps.
`_empty_exsitu()`	—
`_at(df, f0)`	Each segment's point nearest to f0 (log distance).
`exsitu_summary(df, f_top)`	—
`_relative(df, re, im, scale)`	—
`insitu(run_dir)`	Z_s / plate median per segment, raw (bronze) and clean (silver),
`insitu_summary(df)`	—
`_segment_steps(df)`	Per segment, after correction: median phase above F_HF, and how far
`card_residual(df)`	Per card: median phase and |H| step above F_HF (_segment_steps).
`box_residual(df, boxes)`	The same per interface box (r2d2_geometry.interface_boxes), with the
`_boxes_of(run_dir)`	The interface-box map of the run's plate, {} if unknown.
`checks(summary)`	—
`plot_exsitu(df, path, title, f_top)`	—
`plot_insitu(df, path, title)`	—
`run(run_dir, gain_file, bode_dir, out_dir, write_png, f_top, …)`	Both views for one finished run; writes RUN_DIR/frequency_response/.
`report(summary, say)`	—
`main(argv)`	—

`checks/gamry_compare.py`

1,365 lines · 3 classes · 26 functions

The local EIS measures 72 segments. Summed in parallel they must reproduce what an ordinary whole-cell EIS of the same cell measured at the same moment. That is the only end-to-end check the method has: it tests the calibration, the geometry, the synchronisation and the chain response all at once, against an instrument that shares none of them.

Depends on	gamry_dta, gamry_sync, mdf4_min, utils
Used by	csv_pipeline, dc_closure, gamry_sync, main, nyquist, session, silver (+ 8 test files)
Third-party	asammdf, matplotlib, numpy
Standard library	csv, dataclasses, datetime, importlib, pathlib, re, sys, tempfile

Class / function	What it does (first docstring line)
`class CellSweep @dataclass(frozen=True)`	One whole-cell reference spectrum. Methods: name, condition, asr, hfr_ohm.
`_hf_intercept(freq, Z)`	Z' where Z'' crosses zero on the way DOWN from the top of the band.
`_hf_extrapolated(freq, Z, n_top)`	Z' extrapolated to Z'' = 0 from the top of the band.
`read_cell_sweep(path)`	—
`is_sweep_mf4(name)`	A Gamry sweep exported as .mf4 (RO2612090-01_V26_112_HFR_101_CurrVal_450
`mf4_sweep_as_dta(path, cache_dir)`	The .mf4 Gamry sweep `path` written as a .dta (same columns, its
`find_cell_sweeps(root, pattern, order_id, version)`	Every whole-cell sweep under `root` that belongs to THIS cell.
`_far(g)`	—
`_hfr_number(sw)`	—
`select_sweep(hits, when, test_id, log, max_gap_s, match, order_id, ver…)`	The ONE sweep, among those of this current, that belongs to this
`_sweep_span(sw)`	(start, end) of a sweep from its STARTTIME and duration, or None.
`_stamp_gap(sw, when)`	Seconds between a sweep and the recording window (0 = overlap).
`_select_by_file(hits, when, test_id, log, max_gap_s)`	select_sweep(match="file"): by the file alone -- order / build, current,
`class BenchLog @dataclass`	An ASAM MDF4 bench recording, reduced to what the comparison needs. Methods: state_at.
`_asammdf_advice(exc)`	Why the import failed, and the command that fixes THIS interpreter.
`_ensure_zstd_importable()`	Make `from zstd import decompress` succeed without a C compiler.
`read_bench_log(path)`	Read an MF4 bench log. Requires `asammdf`; absent, this is skipped.
`_order_core(text)`	The digits of an order id, so RO2611976-01 and 2611976 compare equal.
`version_of(name)`	The build token in a file name, normalised to "V26_092", or None.
`names_version(name, version)`	Does this filename carry this build token?
`names_order(name, order_id)`	Does this filename name this cell?
`find_bench_log(root, order_id, version)`	The bench .mf4 for this cell, or None.
`class Comparison @dataclass`	One local aggregate against one whole-cell sweep. Methods: n_points, hfr_rel, hfr_fit_rel, mag_rel_median, phase_diff_deg, rms_rel, summary.
`compare(freq_local, Z_local, sweep, area_cm2, bench, chain_applied)`	Put a local aggregate and a whole-cell sweep on the same axis.
`read_cell_aggregate(path)`	Read the pipeline's own `cell_aggregate.csv` (mohm.cm2) as ohm.cm2.
`write_outputs(comparisons, out_dir, log)`	—
`plot(comparisons, path)`	Nyquist and residual, one column per condition.
`run(results_root, gamry_root, area_cm2, out_dir, bench_path, …)`	Compare every condition that has BOTH a local result and a sweep.
`_selftest(log)`	Round-trip a known cell through the comparison and check the metrics.

`checks/hf_response.py`

623 lines · 0 classes · 22 functions

The high-frequency response of every channel, measured across operating points and FROZEN -- the card and segment correction of the "Local EIS Complete Fix" plan (fixes 3 and 4), estimated the way that plan demands: from repeated measurements, validated on a current it was not fitted on.

Depends on	utils

Used by	session, silver (+ 1 test files)
Third-party	numpy, pandas
Standard library	argparse, hashlib, json, pathlib, shutil

Class / function	What it does (first docstring line)
`load_run(run_dir)`	(condition, spectra with segment / card / fk / Z, config) of one run.
`deviations(sp, exclude)`	Each point against the plate median at its frequency (complex),
`_quad(f, y)`	y smoothed by a quadratic in f (least squares): over 2-12.5 kHz the
`_smooth3(y)`	—
`_quad_at(f, y, f_at)`	The quadratic of _quad at f_at; above the last step it is continued
`lowpass_fit(f, lm)`	(a, fc, rms) of ln|H| = ln a - 0.5 ln(1 + (f/fc)^2), least squares;
`_log_mag(runs)`	ln|r| minus the segment's own 100-1000 Hz level, per run.
`rolloff(runs, f_lo)`	Per segment: |H| at its three highest steps against its own level
`flag_rolled_off(runs, f_lo, rolloff_min, rep_tol)`	Channels that roll off; the plate median re-taken without them once.
`estimate(runs, f_lo, mag, rolloff_min, rep_tol, info)`	{segment: (freq, h)} from deviations() of several runs.
`apply(sp, h)`	Divide each point at or above f_lo by its segment's h (nearest step).
`metrics(sp, f_card)`	Phase spread at the top common step, largest card |Z| step above
`validate(runs, f_lo, mag)`	Leave one run out: estimate on the others, apply to it, compare.
`write(h, path, f_lo, note)`	The gain-file format utils.load_gain reads; h = 1 below f_lo.
`_sha12(path)`	—
`top_step(sp)`	The highest step at least 60 % of the run's segments reach.
`compose(h, old, f_lo)`	old x residual on the union of both files' steps above f_lo.
`_backup(out)`	Copy the file about to be replaced (and its tables) to
`build(run_dirs, out_dir, f_lo, log, mag, prior, f_top_min)`	Estimate from all runs, validate leave-one-out, write the file.
`segment_sources(hf_file)`	{segment: 'segment' | 'card'} from the table build() writes beside
`_seg_key(s)`	—
`main(argv)`	—

`checks/plausibility.py`

1,248 lines · 2 classes · 26 functions

The gates in silver.py answer "is this POINT usable". This module answers a different and larger question: taking the surviving points at face value, does the resulting picture of the plate hold together as physics?

Depends on	card_gain, plate_stats, r2d2_geometry, silver, utils
Used by	frequency_response, main (+ 5 test files)
Third-party	numpy
Standard library	argparse, csv, dataclasses, math, pathlib, types

Class / function	What it does (first docstring line)
`class Check @dataclass(frozen=True)`	One plausibility question, its answer, and why that is the answer.
`class Report @dataclass`	Methods: verdict, by_verdict, rows.
`coverage(measured, plate_key)`	How much of the plate this run actually saw, by area.
`describe_block(measured, plate_key, max_missing)`	Name the measured set: whole plate, interior block, perimeter, or neither.
`_contiguous_runs(nums)`	[1,2,3,7,8] -> [(1,3),(7,8)].
`resolve_current(cfg, bench)`	The plate current to close against, and where it came from.
`current_closure(j_dc, areas, setpoint_a, plate_key, source)`	Sum(j_s * A_s) over the measured area, scaled to the whole plate.
`_positions(plate)`	Segment -> (x, y) mm of its label pad: where the drawing places it.
`_tile_or_all(values, pos)`	The 36 tile segments (plate_stats: 1..36, one per 5x5-pad tile, the
`_basis(good)`	—

Class / function	What it does (first docstring line)
`neighbour_smoothness(values, plate_key, param)`	Is the map spatially smooth, or does it look like shuffled labels?
`passivity(spectra)`	Re(Z) > 0 everywhere: a passive cell cannot generate energy.
`flow_trend(values, plate_key, param, axis)`	Does the map vary along the flow axis at all, and in which direction?
`parallel_series_resistance(r_ohmic, areas, r_sd)`	Combine per-segment R_s (ohm.cm2) in parallel, back into ohm.cm2.
`series_resistance_closure(r_ohmic, areas, reference_hfr_ohm_cm2, r_sd, area_fractio…)`	Parallel sum of the segment R_s against the whole cell's own R_s.
`_topband_re(freq, Z, span_decades)`	Inverse-variance mean of Re Z over the top third-decade of a band.
`matched_series_resistance(freq, Z_agg, ref_freq, ref_Z_asr, area_fraction)`	R_s of the local aggregate vs the whole cell, SAME estimator, SAME band.
`aggregate_vs_reference(freq, Z_agg, ref_freq, ref_Z_asr, area_fraction)`	The partial-plate aggregate against a whole-cell sweep, both in ohm.cm2.
`channel_lag_check(lags)`	Do the segment current chains see the same high-frequency phase?
`card_alignment_check(card_lags)`	Was every card shifted onto the common time base?
`card_voltage_check(card_ref)`	Do the cards read the same cell voltage on their UC channel?
`card_factor_check(factors)`	Is any card's impedance level a scale error rather than the cell?
`check_run(sr, cfg, plate_key, reference, reference_hfr, bench)`	Every check that the available data supports, on a finished SilverRun.
`check_from_disk(run_dir, cfg, plate_key, reference, reference_hfr, bench)`	The same checks as `check_run`, from files a finished run wrote.
`report(rep, log)`	—
`save(rep, out_dir)`	—
`_load_summary(path)`	—
`main(argv)`	—

`checks/validate_hf.py`

197 lines · 0 classes · 7 functions

Does the new high-frequency chain actually work?

Depends on	config, silver, utils
Used by	main
Third-party	numpy
Standard library	—

Class / function	What it does (first docstring line)
`synth_card(rng, n_seg, n_ch, fs, f_max, noise, ref_slot, dt0_true, r…)`	One card: n_seg segments, each at its own conversion slot.
`_prep(items)`	Pack the synthetic card the way silver.optimise_skew expects it.
`_cost(items, dt0, k, M)`	—
`fit_structural(items, k_nom)`	Calls silver's REAL optimiser, so this tests shipped code.
`fit_per_card(items)`	Legacy: one free delay for the whole card.
`report(name, est, true)`	—
`main()`	—

analysis/

The DRT and the DRT-informed equivalent-circuit fit used by the ECM cells.

`analysis/ecm_drt.py`

458 lines · 0 classes · 15 functions

DRT-informed equivalent-circuit fit (the notebook's "method B"), revised for the thesis. Model:

Depends on	only standard / third-party libraries
Used by	views (+ 1 test files)
Third-party	numpy, scipy
Standard library	—

Class / function	What it does (first docstring line)
`z_zarc(w, R, tau, n)`	—
`z_tlm(w, R_ion, R, tau, n)`	Finite catalyst-layer transmission line with a ZARC interface.
`_unpack(p, layout)`	layout = ('zarc',) | ('zarc','zarc') | ('tlm','zarc'); p after Rs, L.
`z_model(Rs, L, p_arcs, layout, freq)`	—
`mains_mask(freq, mains_hz, odd_only, tol, max_order)`	True where a point sits on a low mains harmonic (to be dropped).
`_start(freq, Z, Rs, n_arcs, peaks)`	—
`_fit_layout(f, Z, s, layout, Rs0, rs_fixed, peaks, tau_lo, tau_hi)`	—
`fit_drt_ecm(freq, Z, sigma_rel, peaks, rs_fixed, f_max_fit_hz, drop_m…)`	Fit one spectrum. Returns a dict (see module docstring).
`_report(b, f, Zf, s, weight, dropped_mains, rs_fixed, f_max_fit_h…)`	—
`model_curve(fit, freq)`	The fitted model on any frequency grid (e.g. a smooth line).
`summarize(fits)`	{key: (median, q25, q75, n)} over ALL fitted segments + counts.
`arcs_from_params(params, n_arcs)`	Method A (csv_pipeline) params -> the same fast/slow naming.
`_voigt_basis(freq, taus)`	Complex design matrix of a Voigt ladder with fixed time constants.
`drt_tikhonov(freq, Z, lam, n_tau_per_decade, extend_decades, use, nonn…)`	gamma(ln tau) by Tikhonov-regularised (non-negative) least squares.
`drt_peaks(res, min_rel_height)`	Peaks of gamma: [{'tau', 'f_peak', 'gamma', 'R'}], fastest first;

plotting/

Everything that draws: Nyquist/Bode, plate maps (static and interactive), the drop-down viewers, the common style, the bench log.

`plotting/bench_plots.py`

325 lines · 1 classes · 7 functions

The test-bench (MF4) parameters of an order as ONE interactive Plotly figure:

Depends on	only standard / third-party libraries
Used by	session, views (+ 2 test files)
Third-party	asammdf, numpy, plotly
Standard library	dataclasses

Class / function	What it does (first docstring line)
`class Channel @dataclass`	Methods: stats.
`decimate(t, v, max_points)`	At most ~max_points samples, keeping each bucket's min AND max so a
`_clean(t, v)`	—
`channels_from_mdf(mdf, groups, include_others, max_points)`	Every known channel present in an open asammdf.MDF, then (optionally)
`read_mf4(path, include_others, max_points)`	Read an MF4 bench log. Requires `asammdf`.
`mf4_start_time(path)`	The recording's start time (datetime) or None.
`_fmt(x)`	—
`bench_figure(channels, title, start, reference_lines, height, width)`	One figure, every channel a trace; a dropdown chooses what is shown.

`plotting/figure_panels.py`

217 lines · 0 classes · 6 functions

Show a multi-panel Plotly figure as separate, full-width figures, one below the other.

Depends on	only standard / third-party libraries
Used by	viewers (+ 1 test files)
Third-party	plotly
Standard library	copy, re

Class / function	What it does (first docstring line)
`_panels(layout)`	(xref, yref, xkey, ykey) for every x axis anchored to a y axis.
`_clean_axis(ax, own_ref, new_ref)`	—
`_remap_ref(ref, xref, yref)`	'x3' -> 'x', 'x3 domain' -> 'x domain'; None when it is another panel's.
`_title_text(layout)`	—
`_with_panel(title, sub)`	—
`split_subplots(fig, panel_height, panel_width, skip_empty)`	One figure per subplot panel, in row-major order. See module docstring.

`plotting/nyquist.py`

462 lines · 0 classes · 13 functions

The per-segment Nyquist / Bode plot, built ONCE and used everywhere: the runner shows it (interactive, Plotly) and the run saves the same plot (nyquist.html, and a PNG drawn the same way), so what is saved is what was looked at.

Depends on	gamry_compare, plate_style, r2d2_geometry
Used by	csv_pipeline, gold, views (+ 3 test files)
Third-party	matplotlib, numpy, pandas, plotly
Standard library	pathlib

Class / function	What it does (first docstring line)
`gamry_frame(src, area_cm2)`	The whole-cell Gamry sweep as a DataFrame (freq_hz, z_re_mohm_cm2,
`find_gamry(gamry_dir, condition, order_id, version, area_cm2)`	gamry_frame of the condition's sweep in `gamry_dir` (the same lookup
`_add_gamry(fig, g, name, width)`	The Gamry, black, in all three panels; one legend entry.
`air_inlet_x()`	x (plate mm) of the cathode (air) inlet end.
`flow_position(segments)`	0 at the cathode inlet .. 1 at the cathode outlet, per segment.
`_jet(t)`	—
`colours(segments, colour)`	—
`_seg_key(s)`	—
`figure(df, title, colour, rebuilt, note, height, width, gamry, g…)`	Nyquist + |Z| + phase of every segment (one subplot each).
`save_png(df, path, title, colour, rebuilt, dpi, gamry, gamry_name)`	The Nyquist panel of `figure`, as a PNG: same data, same colours.
`save_run(run_dir, title, colour, out_dir, gamry)`	nyquist.png + nyquist.html for one finished run (FAMOS or CSV).
`comparison_figure(aggregate, gamry, title, aggregate_name, gamry_name, note…)`	The aggregate of the segments against the whole-cell Gamry: Nyquist,
`comparison_png(aggregate, gamry, path, title, dpi)`	`comparison_figure` as a PNG (Nyquist, |Z|, phase in one row).

`plotting/plate_figure.py`

523 lines · 0 classes · 12 functions

Static plate heat map in the style of the interactive plate viewer: the metal frame with its bolts, the gasket border, the six manifold ports with their flow arrows -- a column at each end, gas outlet above the coolant and gas inlet below it (x = 0: AIR OUT / COOLANT / H2 IN; x = 252: H2 OUT / COOLANT / AIR IN; the coolant direction is config.COOLANT_INLET_END) -- the temperature-sensor pins T1..T4 on the top edge, the plate dimensions -- and every segment filled with its value.

Depends on	config, plate_stats, plate_style, r2d2_geometry
Used by	csv_pipeline, gold, viewers (+ 2 test files)
Third-party	matplotlib, numpy, scipy
Standard library	csv, pathlib, sys

Class / function	What it does (first docstring line)
`_cmap(name)`	—

Class / function	What it does (first docstring line)
`_limits(param, vals, limits)`	—
`draw_flow_plate(values, param, title, subtitle, classes, limits, label, u…)`	One plate heat map in the viewer's layout. Returns the figure.
`node_xy(p, n)`	Where segment n sits on the maps: the centre of its label pad, the red
`interpolate_field(p, vals, W, H, step)`	The value field over the active area: 2D linear interpolation between
`shaded_rgb(gx, gy, zi, cm, norm, W, H, gloss)`	Colour the field and give it the soft gloss: a light relief from the
`_draw_interpolated(ax, p, vals, classes, cm, norm, W, H, gloss)`	Fill the active area with the interpolated field and mark every
`_fixed(param)`	—
`renders(style_)`	The map styles a heatmap_style setting asks for.
`suffix(render)`	File-name suffix: plate_R_ohmic.png / plate_R_ohmic_interp.png.
`__getattr__(name)`	—
`write_condition_maps(summary_csv, out_dir, title, params, dpi, side, render)`	Draw plate_.png for every param from a gold plate_summary.csv

`plotting/plate_maps.py`

224 lines · 0 classes · 4 functions

Static plate heat maps with the VALUE written inside every segment.

Depends on	plate_stats, plate_style, r2d2_geometry
Used by	— (+ 2 test files)
Third-party	matplotlib, numpy
Standard library	—

Class / function	What it does (first docstring line)
`_finite(values)`	—
`uniformity_stats(values, classes)`	n, mean, median, sd, CV [%], min, max, span relative to the mean [%],
`robust_limits(values, pct)`	Colour limits from percentiles, widened so a flat plate is not noise.
`draw_value_map(values, label, unit, cmap, decimals, title, classes, pct,…)`	Plate map, one colour and one printed value per segment.

`plotting/plate_plotly.py`

284 lines · 1 classes · 9 functions

Interactive (Plotly) plate heat map: the true segment outlines, the value printed in every segment, hover on any pad for the segment's number, value and class, and a dropdown to switch between parameters.

Depends on	config, plate_stats, plate_style, r2d2_geometry
Used by	gold, viewers (+ 1 test files)
Third-party	numpy, plotly
Standard library	dataclasses

Class / function	What it does (first docstring line)
`class Field @dataclass`	One parameter on the plate. `values`: segment -> value, display units.
`_clean(values)`	—
`_limits(fd, vals)`	—
`_rgba(colorscale, t)`	Colour of the scale at t in [0, 1] as an RGB(A) tuple in 0..1.
`_boundary_xy(p)`	Line segments between pads owned by different segments, + outline.
`_label_xy(p, n, s)`	—
`_field_traces(fd, p, show_colorbar)`	Heatmap + number labels + value labels for one field.
`_subtitle(fd)`	Counts, then mean / median / sd over the 36 tile segments
`_end_annotations(W, H)`	Inlet / outlet / coolant labels beside the two ends, with arrows

Class / function	What it does (first docstring line)
`interactive_plate(fields, title, subtitle, plate_name, height, width)`	One Plotly figure; a dropdown switches between `fields`.

`plotting/plate_style.py`

275 lines · 0 classes · 16 functions

One look for every plate heat map: the colour scale, the text sizes, which way round the plate is drawn, and where the gas and coolant ports are.

Depends on	config, r2d2_geometry
Used by	nyquist, plate_figure, plate_maps, plate_plotly, r2d2_geometry, viewers (+ 2 test files)
Third-party	matplotlib
Standard library	—

Class / function	What it does (first docstring line)
`_rgb01(spec)`	—
`mpl_cmap(name)`	Matplotlib colormap for the heat maps (plotly Jet unless overridden).
`plotly_colorscale(name)`	The same scale as a Plotly colorscale (Jet; "parula" on request).
`ink(rgba)`	Text colour that stays legible on a fill (WCAG relative luminance).
`mirrored()`	True when the plate is drawn mirrored (read at call time, so a
`xlim(lo, hi)`	Matplotlib x limits: reversed when mirrored, so every patch, line and
`outward_ha(physical_side)`	Horizontal alignment for text set just OUTSIDE the x = 0 ("x0") or
`screen_side(physical_side)`	'left' or 'right': where the x = 0 / x = W edge lands on screen.
`coolant_inlet_end()`	'x0' or 'xW': the plate end the coolant enters at (config).
`flow_arrangement()`	—
`end_streams()`	Per plate end ('x0', 'xW'): (label, role, colour) of what enters or
`port_slot(label)`	Where a port sits in its end's column: the coolant in the middle, the
`port_kind(label)`	'anode', 'cathode' or 'coolant'.
`_arrow(stream_from)`	'left to right' / 'right to left' for a stream entering at an end.
`flow_note()`	One line describing the flows AS DRAWN (it changes with mirroring).
`annotate_ends(ax, width_mm, height_mm, gap_mm, fontsize)`	Write what enters and leaves at each end beside a plain plate map.

`plotting/viewers.py`

643 lines · 0 classes · 17 functions

ONE plot on screen, chosen from drop-downs, instead of a stack of figures.

Depends on	figure_panels, plate_figure, plate_plotly, plate_style, r2d2_geometry
Used by	results, session, views (+ 5 test files)
Third-party	matplotlib, numpy, pandas, plotly
Standard library	base64, gzip, html, io, json, pathlib, re, uuid

Class / function	What it does (first docstring line)
`_plotlyjs_url()`	—
`_slim(obj, sig)`	Plotly writes numeric arrays as binary blocks ({"dtype", "bdata"}),
`_payload(item)`	A figure as {"fig": json} or HTML as {"html": str}.
`_pack(obj)`	JSON -> gzip -> base64: the payload of a page. Plot data compresses
`_shell(dims, opts, payload, render_js, title, note, height)`	Drop-downs + one plot area; the browser unpacks the payload and calls
`ascii_html(fragment)`	The fragment with every non-ASCII character as an HTML entity, so the
`selector_html(items, dims, title, height, note)`	One drop-down per entry of `dims`, one plot area.
`panels(fig, panel_height, panel_width)`	A (multi-panel) figure as [(panel name, single-panel figure)].
`by_condition(figs, title, note, panel_dim)`	{condition: figure or html} -> selector; a multi-panel figure becomes
`img_html(path, width)`	A PNG as an inline , for selectors over saved figures.
`table_html(df, decimals)`	—

Class / function	What it does (first docstring line)
`plate_fields(summary_csv, fields)`	plate_plotly.Field per parameter present in a plate_summary.csv.
`_plate_geometry(p, gloss)`	Everything every map of the plate shares, sent once: the pad owner
`_plate_item(p, fd, head, step)`	One condition x parameter: the values, and the interpolated field on
`plate_maps(summaries, title, fields, views, gloss)`	{condition: plate_summary.csv or list[Field]} -> one heat-map viewer
`save_page(fragment, path, title)`	A viewer (what displayHTML shows) as a standalone .html file: the same
`unpack_payload(html)`	The selector's gzip+base64 payload, decoded (tests only).

readers/

Readers for everything that is not a FAMOS card: Gamry .DTA, MF4, the Abgleich calibration, CSV measurements, and the zstd shim for asammdf.

`readers/_vendor/zstd.py`

47 lines · 0 classes · 0 functions

Stand-in for the PyPI package "zstd", which asammdf imports unconditionally.

Depends on	only standard / third-party libraries
Used by	—
Third-party	zstandard
Standard library	—

`readers/abgleich.py`

324 lines · 1 classes · 7 functions

`eis_local.PlateCalibration` consumes two two-column files:

Depends on	eis_local
Used by	—
Third-party	numpy
Standard library	argparse, dataclasses, json, pathlib, re

Class / function	What it does (first docstring line)
`class StepFile @dataclass(frozen=True)`	Methods: segments.
`read_step(path)`	Parse one Step_Grad.csv.
`read_campaign(folder)`	Every Step file in a folder, ordered by its index.
`_slope(points)`	Least-squares u = R*i + u0 over one segment's DC sweep.
`fit_coefficients(steps, area_cm2)`	Rebuild the curr.csv / temp.csv coefficients from the Step files.
`implied_area(steps, curr_csv)`	The constant R(T)/K(T) a delivered curr.csv implies, per segment.
`verify(folder, curr_csv, temp_csv)`	Read a calibration folder and report whether it hangs together.
`write_coefficients(fit, curr_path, temp_path)`	—

`readers/csv_source.py`

1,409 lines · 1 classes · 35 functions

The FAMOS route and the CSV route are not two readers feeding one pipeline. They are two pipelines, and this module is the front half of the CSV one. The reason is in the hardware, not the file format:

Depends on	gamry_dta, r2d2_geometry, utils
Used by	csv_pipeline, main, session (+ 3 test files)
Third-party	numpy
Standard library	dataclasses, inspect, io, json, pathlib, re, sys, tempfile

Class / function	What it does (first docstring line)
`class CsvMeasurement @dataclass`	Whatever the CSV held, in the one shape csv_pipeline understands. Methods: segments, n_samples, fs_nominal, duration_s, summary.
`regularity(t)`	How uniform is the recorded time base?

Class / function	What it does (first docstring line)
`quantisation_step(x)`	Recover the printed resolution of a column from the data itself.
`fit_phasor_nonuniform(t, y, f, detrend)`	Least-squares sine fit at known f on ARBITRARY sample times.
`fit_phasor(t, y, f, uniform, fs, detrend)`	Dispatch to the uniform-grid fit when that is safe, else to the
`fit_phasors_multi(t, y, freqs, detrend)`	Fit EVERY tone of a multisine in one least-squares solve.
`_sniff(path, n_lines)`	—
`_split(line)`	Split a header/data row on the separator that yields the most fields.
`detect_dialect(path)`	Decide which layout `path` is, from its first lines.
`read_records(path)`	The bench record format: one `s<n>: ...` record per segment per instant.
`_read_table(path)`	Header, numeric block and the raw string rows (for non-numeric ids).
`read_wide_time(path)`	time_s, u_cell, s1..s72 [, temp1..temp4] -- one row per instant.
`read_long_time(path)`	time_s, segment, u_s [, u_cell] -- tidy, one row per sample per segment.
`read_freq(path, z_unit)`	A ready-made spectrum table: segment, freq_hz, and Z in some form.
`_r2d2_seconds(stamps)`	Seconds from the first row, from `YYYY.MM.DD HH:MM:SS,ffffff`.
`read_r2d2(path, meta_extra)`	One R2-D2 point file: `<name>.csv` with the `timeshifts` header row.
`read_r2d2_metadata(folder)`	The free-text `metadata.csv` sidecar, as key/value where it looks it.
`r2d2_point_files(folder)`	The point files of a sweep, in acquisition order.
`read_r2d2_sweep(folder)`	Every point file of one sweep, each carrying the shared metadata.
`read_gamry(path, area_cm2)`	A folder of per-segment Gamry .DTA sweeps, or one file.
`is_spectra_dir(folder)`	A folder of z.csv spectra files (the bench evaluation tool).
`operating_point(folder)`	Temperature and current from a folder name like Spectrum2_65degC_449A.
`_spectrum_lines(text)`	—
`read_spectra_dir(folder)`	Every z.csv of one operating point, in mOhm*cm2, with the segment
`mirror_map_x()`	Segment -> the segment at the mirrored x (x -> plate width - x, same
`mirror_x(m)`	Re-assign every segment of a spectra_dir to its left-right mirror.
`spectra_dirs(root)`	Every spectra_dir folder at or below `root` (one level of nesting).
`condition_current(condition)`	—
`match_condition(folders, condition, tol)`	The folder whose current is nearest to `condition` (e.g. "450A" ->
`is_condition_folder(folder)`	One operating point: a spectra_dir, or a sweep folder whose name
`condition_folders(root, depth)`	Every operating-point folder at or below `root`, `depth` levels deep.
`campaigns(root, depth)`	{campaign label: [operating-point folders]}. The campaign is the folder
`nominal_label(current_a)`	449 A -> "450A": the setpoint, when the measured current is within 2 %
`condition_map(folders)`	{"450A": folder, ...} for the folders that carry a current. Two folders
`read(path, dialect)`	Read a CSV measurement, detecting the layout unless told which.
`_selftest(log)`	Round-trip a known spectrum through each layout. Returns #failures.

`readers/gamry_dta.py`

477 lines · 1 classes · 11 functions

Two jobs:

Depends on	channel_lag, eis_local
Used by	csv_source, frequency_response, gamry_compare, session (+ 2 test files)
Third-party	numpy
Standard library	argparse, dataclasses, pathlib, re

Class / function	What it does (first docstring line)
`_f(s)`	—
`class GamrySweep @dataclass(frozen=True)`	Methods: sorted, z_lf, z_flat, normalised.
`read_dta(path)`	Parse the ZCURVE table of a Gamry .DTA file.

Class / function	What it does (first docstring line)
`read_bode_folder(folder, skip_raw)`	Read every per-segment sweep in a `bode/` folder -> {segment: sweep}.
`find_abgleich_dirs(roots, max_depth)`	Folders that look like an Abgleich delivery: a `bode/` holding
`cross_check_abgleich(sweeps, curr_csv)`	Do the bode sweeps and the DC Abgleich describe the same segments?
`_normalised(sw, f_ref_hz)`	—
`median_response(sweeps, f_ref_hz)`	The plate's shared chain response: median |H| and median arg H.
`write_gain_csv(sweeps, path, convention, f_ref_hz, curr_csv, force, shared)`	Write `segment,freq_hz,gain_real,gain_imag` for utils.load_gain.
`chain_tau(sweeps, f_lo, f_hi)`	Per segment: the first-order time constant tau [s] of its chain,
`compare_chain_tau(bode_tau, insitu_tau)`	Does the ex-situ chain explain the in-situ per-segment lag?
`chain_summary(sweeps, at_hz)`	Median |H| and phase across segments at a few frequencies.

`readers/mdf4_min.py`

150 lines · 1 classes · 4 functions

A minimal MDF 4.x reader for the small, sorted files the Gamry export writes (RO2612090-01_V26_112_HFR_101_CurrVal_450.mf4: one data group, one channel group, float64/uint channels, an XML header comment with STARTTIME).

Depends on	only standard / third-party libraries
Used by	gamry_compare (+ 1 test files)
Third-party	numpy
Standard library	pathlib, re, struct, zlib

Class / function	What it does (first docstring line)
`class MDF4Error`	—
`_block(buf, at)`	(id, links, data bytes) of the block at byte `at`.
`_text(buf, at)`	—
`_data_bytes(buf, at)`	Concatenated record bytes of a ##DT / ##DL / ##DZ chain.
`read(path)`	({channel name: values}, {header key: value}) of a sorted MDF4 file.

docs/

Tools that build this documentation.

`docs/tools/make_file_index.py`

218 lines · 0 classes · 5 functions

Writes docs/TRAINING_GUIDE_FILE_INDEX.md: for every Python file of the package its size, what it imports from the package, who imports it, the libraries it uses, the first paragraph of its docstring and an index of its top-level classes and functions (first docstring line each).

Depends on	only standard / third-party libraries
Used by	—
Third-party	—
Standard library	ast, pathlib, re, sys

Class / function	What it does (first docstring line)
`folder_of(rel)`	—
`summary(doc, limit)`	The first paragraph of a module docstring, without its title.
`md(s)`	—
`scan()`	—
`write(data)`	—

tests/

The pytest suite, its synthetic-data generators and fixtures.

Test file	Tests	What it checks (first docstring line)
conftest.py	0	Make the pipeline's folders importable for every test.
make_synth_famos.py	0	Write a synthetic FAMOS card set so the pipeline can be run end to end.
make_synth_sweep.py	0	Synthetic raw data: a descending stepped sweep on several cards, and its Gamry .DTA.
test_aggregate_and_fill.py	14	The cell aggregate, written as an admittance sum, and the gaps in it.
test_alignment_diagnostics.py	14	The alignment band, the clock-drift diagnostic and the closure check.
test_audit_corrections.py	5	The corrections that came out of the raw-signal audit of 2612030 450 A:
test_bench_plots.py	4	The MF4 bench parameters: one figure, a dropdown per parameter, legend
test_card_alignment.py	6	Cross-card alignment: the thing that cost 43 of 68 segments on the 45 A set.
test_card_alignment_corroboration.py	8	A weak alignment peak that another card reproduces is a confirmed one.
test_card_gain.py	16	The per-card voltage-gain check and the DC closure, on data with a known answer.
test_channel_integrity.py	7	One signal per channel: the RO2612085 Karte_3 failure, as a test.
test_channel_lag_stage.py	12	The channel-lag stage inside the pipeline, on a plate with a known answer.
test_console_encoding.py	6	The log must survive a console that cannot spell.
test_coverage_and_provenance.py	9	A frequency in the schedule is not a frequency that was measured.
test_csv_pipeline.py	5	test_csv_pipeline.py -- end-to-end checks for the CSV evaluation path
test_datago_original_card.py	8	RO2612091 45 A Karte_3: datago's rows hold UC2 under every channel.
test_drt_nonnegativity.py	9	A distribution of relaxation times cannot be negative.
test_ecm_drt.py	7	ecm_drt: the revised DRT-informed ECM fit (method B).
test_famos_dialects.py	9	Two FAMOS dialects, and the one that fails silently.
test_figure_panels.py	8	One plot per figure.
test_file_discovery.py	7	Which card files belong to THIS run?
test_frequency_refine.py	2	Blind runs: the ladder snap puts every step on a perfect 10/decade grid,
test_frequency_response.py	8	frequency_response: card delays show before the timing correction and not
test_full_band.py	10	Full band (a developer diagnostic since the shipped evaluation stops at
test_gamry_by_bench_name.py	2	The bench .mf4 name ties an order to its build, and the build to its
test_gamry_compare.py	22	The whole-cell cross-check: does it recover an error we planted?
test_gamry_match_by_file.py	4	RO2612091 45 A (2026-10-21): the Gamry sweep was there and was not used.
test_gamry_plotting.py	4	The Gamry is drawn exactly as recorded: every point of the .DTA at the
test_gamry_selection.py	8	Which cell's reference sweep is this, and how would you know?
test_gamry_sweep_choice.py	7	Which Gamry sweep is a run's reference -- RO2612090 450 A (2026-10-21).
test_gamry_sync.py	12	The Gamry clock: read its timeline, measure the offset, fix what the blind
test_gold_fill_and_silver_files.py	4	What the display cells draw, and what they leave out.
test_heatmap_limits.py	3	Fixed heat-map colour scales (config.HEATMAP_LIMITS).
test_hf_response.py	12	The two fixes of the October 2026 multi-condition review:
test_hf_schedule.py	3	One stretch of the record is one step of the sweep.
test_hfr_estimators.py	8	How should the high-frequency intercept be measured?
test_plate_figure.py	5	plate_figure: the viewer-style plate map draws, and uses the fixed scale.
test_plate_maps.py	6	Plate maps with the value in every segment, on a scale that shows spread.
test_plate_orientation.py	13	A plate mounted turned round: the segments move, the ports do not.
test_plate_stats.py	6	Plate mean / median over the 36 representative segments: segment 1..36
test_plate_style.py	13	One look for every plate map: Jet colours, value over number, the view
test_plausibility.py	32	Plausibility checks: do they catch the faults they claim to catch?
test_prominence_guard.py	5	The prominence guard is a duration, and widening it costs nothing.
test_reference_channel.py	8	The shared reference is UC2 on every card, by name, not by argmax.
test_rejection_ledger.py	8	Why did a point, or a whole segment, not make it into the result?
test_runner_datago.py	4	runner.datago against a stand-in for Spark: every sweep of a current is

Test file	Tests	What it checks (first docstring line)
`test_runner_datago_famos.py`	16	FAMOS_FROM_DATAGO: a card is rebuilt from datago's rows, checked, and
`test_runner_session.py`	12	The runner package behind the notebook: selection, saved results, plots.
`test_segment_exclusion.py`	12	Leaving a segment out has to mean leaving it out.
`test_site_paths_and_plate.py`	9	Shared plate files, the plate drop-down, and a cache that is found again.
`test_spectra_dir.py`	8	The bench tool's spectra folders (Spectrum*degCA/z.csv) end to*
`test_standard_eis.py`	5	The standard evaluation (standard_eis.py, config.param_method = "ecm"):
`test_sync_timestamps.py`	4	Header time stamps: used to pick the sweep and the test, never to align.
`test_tables_delta.py`	1	runner/tables.py on a real Delta Lake (local Spark). Skipped unless
`test_tables_results_auto.py`	14	The spp_eval-style layer: Delta tables per run, the read-only Results
`test_tables_spark_types.py`	2	Every frame tables.frames() makes must be accepted by a real Spark as it
`test_tone_estimation.py`	5	Card-level tone estimation (tone_estimation.py) and its wiring into bronze.
`test_viewers.py`	5	viewers: one plot chosen from drop-downs; the plate viewer offers every
`test_window_confine.py`	5	Low-frequency windows must sit inside their own Gamry step.